烟幕干扰弹投放策略的时空协同优化

摘要：

针对无人机投放烟幕干扰弹以遮蔽圆柱形真目标的问题，本文建立导弹、无人机、烟幕弹与烟幕云团的统一三维运动学模型，并以“来袭导弹到真目标的全部视线均被烟幕阻断”作为有效遮蔽判据。区别于只考察目标中心视线的近似方法，本文将真目标视为具有实际尺寸的圆柱体：单云团情形利用烟幕球对导弹视点形成的切锥，将整圆柱遮蔽转化为上、下底圆周上的连续极值问题；多云团情形采用“对每条目标视线，至少存在一个有效云团与之相交”的联合覆盖口径，即 $\forall$-ray $\exists$-cloud，从而允许不同云团共同覆盖目标视域。有效遮蔽时长由连续时间区间的并集测度计算，重叠时段不重复计时，不连续时段可以累加。

对于问题一，在 $g = 9.8\ \mathrm{m/s^2}$ 下求得投放点为 $(17620, 0, 1800)\ \mathrm{m}$，起爆点为 $(17188, 0, 1736.496)\ \mathrm{m}$，严格整圆柱遮蔽区间为 $[8.056445, 9.448088]\ \mathrm{s}$，有效时长为 $1.391643\ \mathrm{s}$。目标中心视线近似给出 $1.435082\ \mathrm{s}$，比严格结果多计约 3.121%。对于问题二，采用中心视线代理筛选、差分进化广域搜索、多起点无梯度局部精修和严格几何统一复算，得到当前最佳已知时长 $4.542880\ \mathrm{s}$。对于问题三，在同机三弹及投弹间隔约束下，得到联合遮蔽区间 $[2.764945, 8.278596]\ \mathrm{s}$，总时长 $5.513651\ \mathrm{s}$；当前候选实质由前两枚弹完成，不能据此断言第三枚弹在全局最优策略中无用。对于问题四，三架无人机各投一弹形成三段接力遮蔽，联合总时长为 $10.138857\ \mathrm{s}$。对于问题五，以三枚导弹遮蔽时长之和为主目标，通过单弹候选库、分配组合、逐机块精修和边际填充构成 40 维分层求解，得到 $(D_1, D_2, D_3) = (10.250038, 1.912852, 1.741134)\ \mathrm{s}$，$J_{\mathrm{sum}} = 13.904024\ \mathrm{s}$，公平性诊断量 $J_{\min} = 1.741134\ \mathrm{s}$；最终五架无人机各使用一枚有效弹。各问题均进行了可行性、网格收敛、边界探针、独立视线—球相交和内核回归检查。由于问题二至问题五的优化均受评估预算限制，本文只将其结果称为经过严格复算的最佳已知策略，不作全局最优声明。

关键词：烟幕干扰；视线遮蔽；圆柱目标；联合覆盖；区间并集；非光滑优化

AI 辅助标注：本文第 1–13 节的题面整理、模型规格检查、代码实现、数值验证与文字初稿使用了 OpenAI Codex（GPT-5）辅助 [2]；判据选择、参数口径和最终取舍由使用者确认，数值由随文程序复算。关键交互与人工修改情况见支撑材料《AI 工具使用详情》。

1 问题重述

本题参数与任务定义依据 2025 年全国大学生数学建模竞赛 A 题 [1]。

警戒雷达发现三枚来袭导弹后，控制中心需要指挥无人机在来袭导弹与真目标之间投放烟幕干扰弹。导弹以 300 m/s 的速度直指位于坐标原点的假目标；无人机受领任务后可瞬时改变航向，随后以 70—140 m/s 的速度等高度直线飞行。烟幕弹脱离无人机后作抛体运动，起爆时瞬间形成半径为 10 m 的球形有效烟幕区；云团在起爆后 20 s 内以 3 m/s 的速度竖直下沉。同一架无人机相邻两次投弹的时间间隔不得小于 1 s。

真目标为半径 7 m、高 10 m 的圆柱，下底面圆心为 $(0, 200, 0)$。需要依次解决：固定策略下单枚烟幕弹的有效遮蔽时长；单机单弹最优投放策略；单机三弹协同策略；三机各一弹协同策略；以及五机、每机至多三弹对三枚导弹的综合干扰策略。

本题的关键并非只求烟幕球与一条代表视线的距离，而是回答以下三个问题：其一，何时整座圆柱目标均被遮住；其二，多朵烟幕能否共同覆盖不同的目标视线；其三，多段遮蔽在时间上重叠或分离时如何正确计时。为此，本文先建立统一运动学和严格遮蔽几何，再将各小问表达为连续时间区间测度的计算或优化问题。

2 模型假设

1. 烟幕弹在投放瞬间继承无人机的水平速度，初始竖直速度为零；投放后忽略空气阻力，仅受重力作用。
2. 无人机接到任务后可瞬时完成转向，此后以固定航向、固定速度等高度直线飞行，不考虑转向和加速过程。
3. 烟幕弹起爆后瞬时形成半径固定为 10 m 的均匀有效球形区域；不考虑云团扩散、浓度渐变与成云时间。
4. 无风条件下，云团起爆后的水平坐标保持不变，仅以 3 m/s 竖直下沉，并按题意持续有效 20 s。
5. 烟幕弹须在触地前起爆；导弹到达假目标原点后停止计时。
6. “有效遮蔽” 定义为导弹到真目标的全部视线均在到达目标前穿过至少一朵有效烟幕，而非只遮挡圆柱中心。
7. 主计算采用 $g = 9.8\ \mathrm{m/s^2}$；问题一另用 $g = 9.81\ \mathrm{m/s^2}$ 检查敏感性。

上述假设与题面给定的理想烟幕模型一致，但不代表真实风场、扩散和制导反馈已被描述。数值结果的适用范围受这些假设限制。

3 符号说明

符号	含义	单位或范围
O	假目标原点	m
$\mathcal{T}$	真目标圆柱实体	—
M_{j0}，$M_j(t)$	导弹 j 的初始位置、时刻 t 的位置	m
U_{i0}，$U_i(t)$	无人机 i 的初始位置、时刻 t 的位置	m
θ_i	无人机 i 的水平航向角，从 $+x$ 轴逆时针计	rad
v_i	无人机 i 的速度	$[70, 140]$ m/s
t_{ik}^{r}	无人机 i 第 k 枚弹的投放时刻	s
δ_{ik}	投放到起爆的引信时长	s
t_{ik}^{e}	起爆时刻，$t_{ik}^{\mathrm{r}} + \delta_{ik}$	s
P_{ik}^{r}，P_{ik}^{e}	投放点、起爆点	m
$C_{ik}(t)$	起爆后云团中心位置	m
R，T_C	有效烟幕半径、有效寿命	10 m，20 s
$I_j(t)$	导弹 j 在时刻 t 是否被有效遮蔽	0 或 1
E_j，D_j	导弹 j 的有效遮蔽时间集合及其总长度	s
$T^{(q)}$	问题 q（$q = 1, 2, 3, 4$）的总遮蔽时长	s
J_{sum}	三枚导弹遮蔽时长之和，问题五主目标	s
$J_{\min}$	三枚导弹遮蔽时长的最小值，公平性诊断量	s

真目标集合写为

$$\mathcal{T} = \left\{ (x, y, z) : x^2 + (y - 200)^2 \le 7^2,\ 0 \le z \le 10 \right\}.$$

4 统一运动学模型

4.1 导弹与无人机轨迹

导弹始终直指假目标。令

$$\hat{d}_j = -\frac{M_{j0}}{\|M_{j0}\|_2},$$

则导弹轨迹为

$$M_j(t) = M_{j0} + 300t\hat{d}_j = \left(1 - \frac{300t}{\|M_{j0}\|_2}\right) M_{j0}, \qquad 0 \le t \le t_j^{\mathrm{hit}},$$

其中

$$t_j^{\mathrm{hit}} = \frac{\|M_{j0}\|_2}{300}.$$

定义无人机 i 的水平单位航向向量

$$h_i = (\cos\theta_i, \sin\theta_i, 0),$$

其轨迹为

$$U_i(t) = U_{i0} + v_i t h_i.$$

4.2 烟幕弹与云团轨迹

第 k 枚弹的投放点为

$$P_{ik}^{\mathrm{r}} = U_i(t_{ik}^{\mathrm{r}}).$$

在投放到起爆之间，弹体位置为

$$B_{ik}(t) = P_{ik}^{\mathrm{r}} + v_i(t - t_{ik}^{\mathrm{r}})h_i - \frac{1}{2}g(t - t_{ik}^{\mathrm{r}})^2 e_z,$$

其中 $e_z = (0, 0, 1)$。于是

$$t_{ik}^{\mathrm{e}} = t_{ik}^{\mathrm{r}} + \delta_{ik},$$

$$P_{ik}^{\mathrm{e}} = U_{i0} + v_i t_{ik}^{\mathrm{e}} h_i - \frac{1}{2}g\delta_{ik}^2 e_z.$$

云团在有效期内的中心位置为

$$C_{ik}(t) = P_{ik}^{\mathrm{e}} - 3(t - t_{ik}^{\mathrm{e}})e_z, \qquad t \in [t_{ik}^{\mathrm{e}}, t_{ik}^{\mathrm{e}} + 20].$$

起爆可行性要求

$$t_{ik}^{\mathrm{r}} \ge 0, \qquad \delta_{ik} \ge 0, \qquad (P_{ik}^{\mathrm{e}})_z \ge 0, \qquad t_{ik}^{\mathrm{e}} \le t_j^{\mathrm{hit}}.$$

同一无人机的相邻投弹还需满足

$$t_{i,k+1}^{\mathrm{r}} - t_{ik}^{\mathrm{r}} \ge 1.$$

5 严格整圆柱遮蔽模型

5.1 单条视线与烟幕球相交

对目标点 $q \in \mathcal{T}$，导弹 $M_j(t)$ 到 q 的视线线段为

$$L_j(q,t) = \{M_j(t) + \lambda(q - M_j(t)) : 0 \le \lambda \le 1\}.$$

云团中心到该线段的最短距离为

$$\lambda^* = \mathrm{clip}_{[0,1]} \frac{[C_{ik}(t) - M_j(t)]^{\mathsf{T}}[q - M_j(t)]}{\|q - M_j(t)\|_2^2},$$

$$d_{jik}(q,t) = \|C_{ik}(t) - [M_j(t) + \lambda^*(q - M_j(t))]\|_2 .$$

当 $d_{jik}(q,t) \le R$ 时，该视线在导弹与目标之间穿过烟幕球。使用线段而非无限直线，可避免把导弹后方或目标后方的烟幕误判为有效遮蔽。

5.2 单云团的连续圆柱极值

单云团严格遮住整个圆柱要求

$$\max_{q \in \mathcal{T}} d_{jik}(q,t) \le R.$$

为避免以少量代表点代替整座目标，本文使用烟幕球相对导弹视点形成的切锥。记同一时刻的导弹位置与云团中心为 M, C，$D = \|C - M\|_2$。当 $D > R$ 时，令

$$a = \frac{C - M}{D},$$

并定义切锥裕量

$$m(q) = \big\|(q - M) - [(q - M)^{\mathsf{T}} a]a\big\|_2 - \frac{R}{\sqrt{D^2 - R^2}}(q - M)^{\mathsf{T}} a.$$

$m(q) \le 0$ 表示点 q 位于烟幕球的遮蔽切锥内，同时还需检查目标位于球后切平面之后。由于 $m(q)$ 对 q 为凸函数，而圆柱为凸集，最大裕量可在圆柱的极点集合取得，即上、下底面的两条圆周。由此将三维连续极值化为两个周期角度上的一维最大化。若 $D \le R$，导弹位于烟幕球内，所有由导弹发出的目标视线均已与烟幕相交，程序单独处理该边界情形。

5.3 多云团联合覆盖

多云团不能简单要求“存在一朵云单独遮住整个圆柱”。本文采用

$$I_j^{\mathrm{joint}}(t) = 1 \iff \forall q \in \partial\mathcal{T},\ \exists (i,k) \in \mathcal{K}(t) : d_{jik}(q,t) \le R,$$

其中 $\mathcal{K}(t)$ 为时刻 t 仍处于有效期内的云团集合。该量词次序允许不同云团分别遮住目标视域的不同部分。若以 $m_{jik}(q,t)$ 表示第 (i,k) 朵云对视线的带符号裕量，则联合最坏裕量为

$$G_j(t) = \max_{q \in \partial \mathcal{T}} \min_{(i,k) \in \mathcal{K}(t)} m_{jik}(q,t),$$

并有

$$I_j^{\text{joint}}(t) = 1 \iff G_j(t) \le 0.$$

由于多个切锥的并集不再保持单个凸函数的极值结构，本文对圆柱表面进行逐级加密，并利用距离函数的 Lipschitz 上界检查网格之间是否可能存在未覆盖小孔；临界时刻再进行连续局部精修。人工构造的双云互补案例中，两朵云单独均不能完整遮住目标，而联合后能够完整覆盖，验证了该实现没有退化为单云区间并集。

5.4 遮蔽区间并集

对导弹 j，有效时刻集合为

$$E_j = \{t \in [0, t_j^{\text{hit}}] : I_j(t) = 1\},$$

总遮蔽时长为

$$D_j = \mu(E_j),$$

其中 μ 表示一维时间长度。数值上先用固定时间网格定位状态变化区间，再用 Brent 法求连续边界；最后合并相交或相邻区间并求总长度。时间网格只用于寻找根的括区，不用“有效网格点数乘步长”估计时长。多枚弹同时有效时只计算区间并集，不能把各弹时长直接相加。

6 优化方法与验证框架

遮蔽时长随决策变量变化时会出现区间生成、消失、合并和分裂，因此目标函数非凸且不保证光滑。本文采用分阶段无梯度优化：先使用较低分辨率几何代理和结构化可行解扩大候选覆盖，再用差分进化进行广域搜索；随后对多个可行候选采用 Nelder–Mead 局部精修；最终统一使用严格整圆柱判据、自适应表面细化与连续时间求根复算并排名。代理时间网格固定锚定于任务绝对时间 $t = 0$，同时加入起爆、失效等事件时刻，避免候选有效窗口变化导致采样相位改变。

正式验证包括：

1. 检查速度、投放时刻、引信、投弹间隔、起爆高度和命中前起爆等可行性条件；
2. 提高时间和目标表面分辨率，比较遮蔽时长的收敛性；
3. 在每个区间起点前、起点、内部、终点和终点后检查遮蔽状态；
4. 不调用主切锥公式，直接以视线线段—球相交进行高密度独立复算；
5. 检查云团标签置换不变性、单云内核回归和多云互补构造；
6. 记录随机种子、求解预算、停止状态和近优候选。

问题二至问题四的差分进化与部分局部精修均达到预设评估预算上限，问题五的分层候选与块精修同样受固定预算限制。因此，问题二至问题五的结果均称为“当前最佳

已知策略”；其可行性和目标值经过严格复算，但没有获得全局最优证明。

7 问题一：固定策略下的遮蔽时长

FY1 以 120 m/s 朝原点方向飞行，故 $h_1 = (-1, 0, 0)$。投放时刻为 1.5 s，引信时长为 3.6 s，起爆时刻为 5.1 s。由运动学模型得到

$$P_{11}^{\mathrm{r}} = (17620, 0, 1800)\ \mathrm{m},$$

$$P_{11}^{\mathrm{e}} = (17188, 0, 1736.496)\ \mathrm{m}.$$

严格整圆柱与目标中心视线两种口径的结果如下。

判据	遮蔽区间 / s	总时长 / s
严格整圆柱	$[8.0564454904, 9.4480881586]$	1.3916426682
中心视线近似	$[8.0130060567, 9.4480881586]$	1.4350821019

因此问题一的有效遮蔽时长为

$$\boxed{T^{(1)} = 1.392\ \mathrm{s}}.$$

中心视线近似多计 0.043439 s，相对严格结果高约 3.121%，说明目标的有限尺寸会显著影响遮蔽开始时刻。采用 $g = 9.81\ \mathrm{m/s^2}$ 时，严格时长为 1.405034 s，比主值增加约 0.962%。

圆周初始采样数从 256 增至 4096 时结果保持一致；时间扫描步长从 0.10 s 减至 0.01 s 时，时长最大差约 4.1×10^{-11} s。对圆柱表面独立采样 349,920 点所作的视线线段距离检查，与区间边界前后状态全部一致。导弹与云团最近距离为 4.716707 m，表明导弹在遮蔽过程中进入烟幕球，因此必须保留 $D \le R$ 的特殊几何分支。

8 问题二：单机单弹策略优化

8.1 优化模型

决策变量为

$$x = (\theta, v, t^{\mathrm{r}}, \delta),$$

目标是在可行域内最大化严格整圆柱遮蔽区间并集长度：

$$\max_{x \in \mathcal{X}_2} T^{(2)}(x).$$

其中 $0 \le \theta < 2\pi$，$70 \le v \le 140$，$t^{\mathrm{r}} \ge 0$，$\delta \ge 0$，且烟幕弹须在触地前、M1 到达假目标前起爆。中心视线结果仅用于生成搜索候选和结果对照，不参与最终排名。

8.2 最佳已知策略与结果

项目	数值
航向角	176.640993°
飞行速度	70.000000 m/s
投放时刻	0 s
投放点	(17800, 0, 1800) m
引信时长	2.496785640 s
起爆时刻	2.496785640 s
起爆点	(17625.525268, 10.240445, 1769.453701) m
严格遮蔽区间	[2.496787690, 7.039667637] s
严格总时长	4.542879947 s
中心视线时长	4.572673761 s

故问题二当前最佳已知严格遮蔽时长为

$$\boxed{T^{(2)} = 4.543 \text{ s}}.$$

该策略位于速度下界和投放时刻下界，说明在当前几何位置与单弹约束下，尽早释放并保持较低飞行速度更有利于把起爆点布置在有效视线附近。四档精度结果的最大差为 7.2×10^{-12} s；88,560 个圆柱表面点的独立视线检查通过；对航向、速度、投放时刻和引信的六项可行局部扰动均未改善当前目标值。将问题一策略输入该通用内核，时长误差仅为 1.78×10^{-15} s。

9 问题三：FY1 三弹协同策略

9.1 投弹时序参数化

三枚弹共享 FY1 的航向 θ 与速度 v。为自动满足投放顺序和至少 1 s 的间隔，引入非负变量 s_0, s_1, s_2：

$$t_1^{\mathrm{r}} = s_0, \qquad t_2^{\mathrm{r}} = s_0 + 1 + s_1, \qquad t_3^{\mathrm{r}} = s_0 + 2 + s_1 + s_2.$$

每枚弹配置独立引信 $\delta_1, \delta_2, \delta_3$，故共有八个决策变量：

$$x = (\theta, v, s_0, s_1, s_2, \delta_1, \delta_2, \delta_3).$$

目标为最大化三朵云对 M1 的 ∀-ray ∃-cloud 联合遮蔽时间：

$$\max_{x \in \mathcal{X}_3} T^{(3)}(x).$$

9.2 最佳已知策略

FY1 的共享航向角为 177.891490°，共享速度为 75.537408 m/s。三枚弹的策略如下。

弹号	投放时刻 / s	投放点 (x, y, z) / m	引信 / s	起爆时刻 / s	起爆点 (x, y, z) / m
1	0.0000866	$(17799.9935, 0.0002, 1800)$	2.7648584	2.7649450	$(17591.2846, 7.6843, 1762.5422)$
2	1.0001937	$(17724.4991, 2.7797, 1800)$	2.5816390	3.5818327	$(17529.6208, 9.9546, 1767.3422)$
3	2.0002754	$(17649.0067, 5.5591, 1800)$	9.3343535	11.3346290	$(16944.3912, 31.5010, 1373.0622)$

相邻投放间隔分别为 1.0001071 s 和 1.0000818 s，满足题面约束。联合遮蔽区间为

$$[2.7649450441, 8.2785964073] \text{ s},$$

总时长为

$$\boxed{T^{(3)} = 5.514 \text{ s}}.$$

第一、二枚弹的单独严格遮蔽区间分别为 $[2.764945, 6.913706]$ s 和 $[5.043020, 8.278596]$ s，单独时长分别为 4.148761 s 和 3.235576 s。它们重叠约 1.870686 s，合并后延长了连续遮蔽窗口。第三枚弹在当前候选中的单独严格时长为零，删除后的联合时长变化仅为 6.2×10^{-13} s；在固定前两枚的条件搜索中也未发现超过 10^{-4} s 的实质改进。因此，只能说明当前最佳已知候选没有有效利用第三枚弹，不能证明三弹问题的全局最优策略必然只需要两枚弹。

当前策略的联合时长与“单云完整遮蔽区间并集”只相差约 4.1×10^{-10} s，即该数值候选主要体现时间接力，而非目标视域互补。三档精度的时长最大差为 3.25×10^{-13} s；110,160 条独立视线检查、六种云团标签排列、问题二单云回归及人工互补覆盖案例均通过。

10 问题四：三机各一弹协同策略

10.1 优化模型

FY1、FY2、FY3 各投放一枚弹。每架无人机拥有独立的

$$x_i = (\theta_i, v_i, t_i^{\mathrm{r}}, \delta_i),$$

共构成 12 维决策向量。由于每机只投一弹，同机投弹间隔约束不激活。目标仍为最大化对 M1 的联合遮蔽区间并集长度：

$$\max_{x \in \mathcal{X}_4} T^{(4)}(x).$$

10.2 最佳已知策略

无人机	航向角 / °	速度 / m/s	投放时刻 / s	投放点 (x, y, z) / m	引信 / s	起爆时刻 / s	起爆点 (x, y, z) / m
FY1	179.301490	136.942948	0.815997	$(17688.263, 1.362, 1800)$	3.963735	4.779732	$(17145.498, 7.980, 1723.015)$
FY2	290.170336	112.126363	7.471102	$(12288.852, 613.668, 1400)$	5.477371	12.948473	$(12500.621, 37.175, 1252.992)$
FY3	120.730428	138.527165	19.053573	$(4651.249, -731.190, 700)$	6.818714	25.872287	$(4168.570, 80.751, 472.175)$

三段联合遮蔽区间为

$$[5.1449298089, 9.5689345086],$$

$$[12.9484732632, 16.4222046726],$$

$$[42.4765836426, 44.7177048596].$$

其时长分别为 4.424005 s、3.473731 s 和 2.241121 s，总时长为

$$\boxed{T^{(4)} = 10.139 \text{ s}}.$$

删除 FY1、FY2、FY3 所对应云团后，联合时长分别减少 4.423935 s、3.473656 s 和 2.241112 s，说明三枚弹均有实质贡献。该策略形成三段相互分离的时间接力，没有产生可报告的目标视域互补增益。三档精度计算得到 10.138972 s、10.138934 s 和 10.138857 s，最大差约 1.15×10^{-4} s，支持毫秒级结果报告。每段区间均通过 110,160 条独立视线的边界检查；云团标签置换不改变总时长，问题三几何内核回归误差为零。

当前生成的 `result1.xlsx` 与 `result2.xlsx` 仅为标注了 `provisional-schema` 的临时工作簿，因为题面所述官方模板未随当前资料提供。正式提交前应在不重新求解的前提下，将已计算字段映射到官方模板。

11 问题五：五机多弹对三枚导弹的综合干扰

11.1 多导弹目标函数与 40 维决策

问题五同时使用 FY1—FY5，每架无人机至多投放三枚烟幕弹，对 M1、M2、M3 实施干扰。对导弹 j，仍按第 5.3 节的联合视线判据分别求得有效集合 E_j 和时长 $D_j = \mu(E_j)$。主目标取三枚导弹的总遮蔽导弹—秒数：

$$\max_{x \in \mathcal{X}_5} J_{\text{sum}}(x), \qquad J_{\text{sum}}(x) = D_1(x) + D_2(x) + D_3(x).$$

该定义允许不同导弹在同一物理时刻各自贡献遮蔽时长，但同一导弹被多朵云重复遮挡时仍只计一次。为显示总和目标可能掩盖的分配不均衡，另报告

$$J_{\min}(x) = \min\{D_1(x), D_2(x), D_3(x)\}.$$

$J_{\min}$ 只作为公平性诊断量，不参与当前候选排名。若任务目标改为优先保障受遮蔽最少的导弹，可将候选按 $(J_{\min}, J_{\text{sum}})$ 的字典序重新搜索和排名。

每架无人机的三枚弹共享航向 θ_i 和速度 v_i。沿用问题三的等待变量参数化，令

$$t_{i1}^{\text{r}} = s_{i0}, \qquad t_{i2}^{\text{r}} = s_{i0} + 1 + s_{i1}, \qquad t_{i3}^{\text{r}} = s_{i0} + 2 + s_{i1} + s_{i2},$$

并为三枚弹设置独立引信 $\delta_{i1}, \delta_{i2}, \delta_{i3}$。因此每架无人机有

$$x_i = (\theta_i, v_i, s_{i0}, s_{i1}, s_{i2}, \delta_{i1}, \delta_{i2}, \delta_{i3})$$

八个连续变量，五架无人机合计 40 维。由于增加一朵可行云不会缩小任何导弹的遮蔽集合，求解阶段先为每架无人机保留三个弹位；正式结果再根据删除贡献，把数值零贡献弹位标记为未使用，从而满足“每架至多三枚”的要求。

11.2 分层求解与 persistent incumbent

直接在 40 维非光滑空间中进行一次全局搜索难以兼顾覆盖范围和严格几何计算成本，因此采用以下分层策略：

1. 对 5 架无人机与 3 枚导弹的 15 个组合分别开展单弹搜索，形成候选库。每个组合使用 64 次评估；其中 13 个组合在当前预算内找到正遮蔽，FY1–M2 与 FY1–M3 仅能记为“当前预算未找到正覆盖”，不能解释为物理不可行。
2. 从候选库中组装使三枚导弹均受到干扰的分配方案，共评估 243 个有效分配子集，并始终保留问题四在 M1 上的已验证三云策略作为可行下界。
3. 固定其余无人机后，依次对每架无人机的八维变量作一次块坐标精修，每个块使用 16 次评估；代理时间网格继续锚定于绝对任务时间 $t = 0$。
4. 首轮严格复算显示仅五个弹位具有正删除贡献。为检验其余弹位能否填补三枚导弹的未覆盖时段，又按无人机完成 5 组、每组 16 次评估的边际填充搜索。
5. 边际填充产生的新候选经严格复算后，$J_{\mathrm{sum}} = 13.8693042$ s，低于此前已经正式验证的 13.9040240 s。因此保留后者作为 persistent incumbent，防止后续有预算搜索使正式结果降级。

上述流程是有预算的分层启发式搜索。persistent incumbent 表示始终保留并重新严格复算的最佳已知可行解，不意味着它已被证明为 40 维全局最优。

11.3 最佳已知投放策略

删除贡献诊断表明，最终五架无人机各使用一枚有效弹，其余十个弹位的总目标删除损失为数值零，故标记为未使用。实际使用策略如下。

无人机	主要干扰导弹	航向角 / °	速度 / m/s	投放时刻 / s	投放点 (x, y, z) / m	引信 / s	起爆时刻 / s	起爆点 (x, y, z) / m
FY1	M1	179.301490	136.942948	0.815997	$(17688.263, 1.362, 1800)$	4.013282	4.829279	$(17138.713, 8.062, 1721.078)$
FY2	M1	290.170336	112.126363	7.471102	$(12288.852, 613.668, 1400)$	5.477371	12.948473	$(12500.621, 37.175, 1252.992)$
FY3	M1	121.013390	138.340570	19.098230	$(4638.709, -735.631, 700)$	6.946564	26.044794	$(4143.569, 87.984, 463.552)$
FY4	M2	303.831378	118.628302	6.375676	$(11421.090, 1371.727, 1800)$	9.749407	16.125083	$(12065.003, 411.000, 1334.250)$
FY5	M3	135.527992	114.522224	15.757616	$(11712.252, -735.770, 1300)$	4.584975	20.342591	$(11337.558, -367.919, 1196.992)$

在删除贡献所用的诊断精度下，FY1、FY2、FY3 对 D_1 的删除损失分别为 4.465017 s、3.473521 s 和 2.311075 s；FY4 对 D_2 的删除损失为 1.912852 s，FY5 对 D_3 的删除损失为 1.741134 s。五枚实际使用弹均有正贡献。

11.4 三导弹遮蔽区间与目标值

各导弹的严格联合遮蔽区间为：

导弹	区间序号	联合遮蔽区间 / s	区间时长 / s
M1	1	$[4.9626149841, 9.4278445555]$	4.4652295714
M1	2	$[12.9484732632, 16.4222046750]$	3.4737314118
M1	3	$[40.9072599455, 43.2183371370]$	2.3110771915
M2	1	$[16.1250830957, 18.0379351471]$	1.9128520514
M3	1	$[20.3425906024, 22.0837243332]$	1.7411337307

因此

$$(D_1, D_2, D_3) = (10.2500381747, 1.9128520514, 1.7411337307)\ \text{s},$$

$$\boxed{J_{\text{sum}} = 13.904024\ \text{s}}, \qquad \boxed{J_{\min} = 1.741134\ \text{s}}.$$

M1 获得三段接力遮蔽，M2 与 M3 各获得一段。D_1 不低于问题四的 M1 可行下界 10.1388573 s，但 $J_{\min}$ 仅为 1.7411337 s，说明按总和排名的策略明显偏向 M1；这正是同时报告 $J_{\min}$ 的必要性。

11.5 数值验证与结论边界

40 维策略的可行性违反为零；全部 15 个参数化弹位均满足同机相邻投放间隔不小于 1 s，且起爆高度均为正。M1、M2、M3 在三档时间—表面精度下的时长跨度分别为 2.13×10^{-4} s、6.90×10^{-5} s 和 9.05×10^{-8} s。对五个遮蔽区间分别检查 44,280 条独立视线，区间中点均完整遮挡，起点前和终点后均存在未遮挡视线。

云团标签排列抽样的最大时长偏差为零；问题四在 M1 上的几何回归误差为 9.92×10^{-10} s；J_{sum} 与三个分量之和完全一致，且 M1 时长满足问题四单调下界。11 个事件测试、编译、格式和静态检查均通过。

上述证据支持当前策略的可行性、时长复算和几何实现一致性，但不能证明全局最优。候选库、分配组合、块精修和边际填充均有明确预算，40 维目标还包含区间生成与消失带来的非光滑性。若更强调三枚导弹的最低保障，应改用 $J_{\min}$ 优先口径重新优化，而不能把当前 $J_{\min}$ 诊断值误解为公平目标的最优值。

当前生成的 result3.xlsx 与问题三、问题四对应工作簿一样，使用带“provisional-schema”标记的临时字段结构，因为官方模板未随当前资料提供。获得官方模板后，只需映射已有策略和区间字段，无需重新求解。

12 模型评价

12.1 模型优点

1. **严格考虑目标尺寸**。本文没有用目标中心点替代圆柱，而是从导弹观察几何出发，判断整座圆柱是否处于烟幕球的遮蔽视域内。问题一中中心视线近似多计约3.121%，说明该改进具有实际数值意义。
2. **联合覆盖量词符合多烟幕物理含义**。多云团判据采用 $\forall$-ray $\exists$-cloud，允许不同烟幕共同覆盖目标视域，而不是要求某一朵云独立完成全遮蔽。人工互补案例验证了这一能力。
3. **时间统计避免重复计数**。所有结果均以连续遮蔽区间的并集长度计算，既能处理不连续时段，也不会把重叠烟幕时长重复相加。
4. **求解与正式评估分离**。低分辨率代理用于扩大候选覆盖，最终结果统一通过严格几何和连续时间求根复算，减少了粗网格直接充当答案的误差。
5. **分层方法适应高维多导弹调度**。问题五将 40 维联合搜索拆成单弹候选库、分配组合、逐机块精修和边际填充，并通过 persistent incumbent 保留已严格验证的最佳已知可行解，避免后续低预算搜索造成结果退化。
6. **验证链条较完整**。模型通过网格收敛、边界前后状态、独立视线—球相交、标签置换、互补构造和前序问题回归检查，能够区分软件数值正确性与模型现实有效性。

12.2 模型局限

1. 烟幕被理想化为半径固定、内部均匀有效的球体，未考虑风场、扩散、浓度衰减、地面效应和成云过程。
2. 导弹始终直指假目标，不考虑制导系统在受干扰后的航迹变化；无人机也忽略转向和加速时间。
3. 多阶段无梯度优化只能在有限预算内搜索复杂的非光滑目标。问题二至问题五均已严格复算，但求解器没有给出全局收敛证据。
4. 问题三当前候选未有效利用第三枚弹，且问题三、四的最佳已知候选没有表现出数值可见的视域互补增益；这属于当前搜索结果的特征，不应上升为一般结论。
5. 题面所述三个官方 Excel 模板当前缺失，现有工作簿只能作为临时字段结构，提交前仍需做格式映射。
6. 问题五的 40 维分层搜索同样没有全局最优证明；15 个单弹组合中有两个在当前预算内未找到正覆盖，不能据此判定其物理不可行。
7. 问题五按 J_{sum} 排名，结果明显偏向 M1，而 $J_{\min} = 1.741134$ s。若任务更重视最低保障或导弹间公平性，需要更换目标函数并重新搜索，不能仅对现有候选作文字解释。

13 结论

本文围绕“烟幕何时真正遮住具有实际尺寸的圆柱目标”建立统一的三维运动学—视线几何模型。单云团情形通过切锥和圆柱极点归约实现严格连续判定，多云团情形通过 $\forall$-ray $\exists$-cloud 联合覆盖描述视域协同，并以连续遮蔽区间并集作为统一目标。

问题一固定策略的严格遮蔽时长为 1.392 s，目标中心视线近似会产生约 3.121% 的高估。问题二单机单弹当前最佳已知时长为 4.543 s。问题三同机三弹当前最佳已知时长为 5.514 s，其中前两枚弹形成连续接力，第三枚弹在当前候选中无实质贡献。问题四三机各一弹形成三段不连续接力，当前最佳已知总时长为 10.139 s。问题五的五枚实际使用弹分别由 FY1—FY5 投放，三枚导弹的遮蔽时长为 $(10.250038, 1.912852, 1.741134)$ s，总遮蔽导弹—秒数为 13.904024 s，最短单导弹遮蔽时长为 1.741134 s。该结果说明总和目标能够形成可行的多导弹分配，但会把更多资源配置给边际收益较高的 M1；若强调均衡保障，应改用 $J_{\min}$ 优先目标重新求解。

五个问题的正式结果均通过相应精度复算和独立几何检查。问题一是固定输入下的可复算结果；问题二至问题五属于有限预算搜索所得的最佳已知策略，数值验证不能替代全局最优证明，也不能消除理想球形烟幕、无风和无制导反馈等模型假设带来的现实局限。

参考文献

参考文献

[1] 全国大学生数学建模竞赛组委会. 2025 年全国大学生数学建模竞赛 A 题：烟幕干扰弹的投放策略 [Z]. 2025.

[2] OpenAI Codex, GPT-5, OpenAI, 使用日期：2026-08-20–2026-08-21.

A 支撑材料文件列表

本论文的支撑材料与论文电子版分开提交。主要文件如下：

文件或目录	内容说明
`problem.md`	经页面核对的结构化题面
`model-spec.md`	模型边界、变量、约束、判据与需求追踪
`configs/`	问题一至问题五的正式运行配置
`src/q1.py–src/q5.py`	五个问题的完整可运行源程序
`run.py`	统一运行入口
`tests/`	联合覆盖、事件边界与回归测试
`outputs/manifest.json`	正式运行种子和产物清单
`outputs/logs/`	优化状态、边界、收敛和独立几何验证记录
`outputs/tables/`	结果表、区间表及三个结果工作簿

B 完整可运行源程序

以下程序与支撑材料中的源文件一致。运行环境和命令见支撑材料中的 README.md。为满足论文附录包含完整程序的要求，正式提交稿默认展开全部源代码；若仅进行正文排版预览，可将导言区的 \fullsourceappendixtrue 改为 \fullsourceappendixfalse。

B.1 统一运行入口

```python
"""Project-level entry point for the current CUMCM 2025 Problem A checkpoint."""

import json
from pathlib import Path

from src.q5 import run_question_5

import mmkit
from mmkit.artifacts import create_run_manifest, prepare_output_paths, write_manifest
from mmkit.experiments import set_random_seed

PROJECT_ROOT = Path(__file__).resolve().parent
PROBLEM_PDF = PROJECT_ROOT / "data/raw/CUMCM-2025-problem+A-Chinese.pdf"
Q5_CONFIG = PROJECT_ROOT / "configs/q5.json"


def main() -> None:
    """Run the currently approved Question 5 checkpoint and write its manifest."""
    output_paths = prepare_output_paths(PROJECT_ROOT / "outputs")
    config = json.loads(Q5_CONFIG.read_text(encoding="utf-8"))
    seed_state = set_random_seed(int(config["seed"]))
    artifacts = run_question_5(PROJECT_ROOT, config)
    manifest = create_run_manifest(
        run_name=PROJECT_ROOT.name,
        seed=seed_state.seed,
        config={"status": "question-5", "problem": "CUMCM-2025-A", "q5": config},
        input_files=[PROBLEM_PDF, Q5_CONFIG],
        artifacts=artifacts,
    )
    manifest_path = write_manifest(
        manifest,
        output_paths.root / "manifest.json",
        overwrite=True,
    )
    print(f"Question 5 complete; mmkit={mmkit.__version__}; manifest={manifest_path}")


if __name__ == "__main__":
    main()
```

B.2 问题一程序

```python
"""Question 1 kinematics, full-cylinder shadow test, and validation outputs."""

from __future__ import annotations

import json
from dataclasses import asdict, dataclass
```

```python
from pathlib import Path
from typing import Any

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.optimize import brentq, minimize_scalar

from mmkit.artifacts import configure_plotting, prepare_output_paths, save_figure, save_table

MISSILE_INITIAL = np.array([20000.0, 0.0, 2000.0])
UAV_INITIAL = np.array([17800.0, 0.0, 1800.0])
TARGET_CENTER = np.array([0.0, 200.0, 5.0])
MISSILE_SPEED = 300.0
UAV_SPEED = 120.0
RELEASE_TIME = 1.5
FUSE_DELAY = 3.6
EXPLOSION_TIME = RELEASE_TIME + FUSE_DELAY
CLOUD_RADIUS = 10.0
CLOUD_DESCENT_SPEED = 3.0
CLOUD_LIFETIME = 20.0
TARGET_RADIUS = 7.0
TARGET_HEIGHT = 10.0


@dataclass(frozen=True, slots=True)
class OcclusionInterval:
    """One continuous effective-occlusion interval."""

    start: float
    end: float

    @property
    def duration(self) -> float:
        """Return interval length in seconds."""
        return self.end - self.start


@dataclass(frozen=True, slots=True)
class Q1Result:
    """Question 1 geometry and both requested occlusion interpretations."""

    gravity: float
    release_point: tuple[float, float, float]
    explosion_point: tuple[float, float, float]
    strict_intervals: tuple[OcclusionInterval, ...]
    center_intervals: tuple[OcclusionInterval, ...]

    @property
    def strict_duration(self) -> float:
        """Total full-cylinder occlusion time."""
        return sum(interval.duration for interval in self.strict_intervals)

    @property
    def center_duration(self) -> float:
        """Total center-line approximation time."""
        return sum(interval.duration for interval in self.center_intervals)


def missile_position(time: float) -> np.ndarray:
    """Return M1 position at time measured from radar detection."""
    direction = -MISSILE_INITIAL / np.linalg.norm(MISSILE_INITIAL)
```

```python
    return MISSILE_INITIAL + MISSILE_SPEED * time * direction


def release_point() -> np.ndarray:
    """Return the fixed Question 1 release point."""
    return UAV_INITIAL + np.array([-UAV_SPEED * RELEASE_TIME, 0.0, 0.0])


def explosion_point(gravity: float) -> np.ndarray:
    """Return the fixed Question 1 explosion point for a declared gravity."""
    return release_point() + np.array(
        [-UAV_SPEED * FUSE_DELAY, 0.0, -0.5 * gravity * FUSE_DELAY**2]
    )


def cloud_position(time: float, gravity: float) -> np.ndarray:
    """Return the cloud center during its effective lifetime."""
    if not EXPLOSION_TIME <= time <= EXPLOSION_TIME + CLOUD_LIFETIME:
        raise ValueError("time is outside the effective cloud lifetime")
    return explosion_point(gravity) + np.array(
        [0.0, 0.0, -CLOUD_DESCENT_SPEED * (time - EXPLOSION_TIME)]
    )


def _cone_margin(points: np.ndarray, missile: np.ndarray, cloud: np.ndarray) -> np.ndarray:
    """Return signed tangent-cone margins; non-positive points lie in the shadow cone."""
    cloud_vector = cloud - missile
    cloud_distance = float(np.linalg.norm(cloud_vector))
    if cloud_distance <= CLOUD_RADIUS:
        return np.full(len(points), cloud_distance - CLOUD_RADIUS)
    axis = cloud_vector / cloud_distance
    point_vectors = np.asarray(points, dtype=float) - missile
    axial = point_vectors @ axis
    perpendicular = point_vectors - np.outer(axial, axis)
    radial = np.linalg.norm(perpendicular, axis=1)
    tangent = CLOUD_RADIUS / np.sqrt(cloud_distance**2 - CLOUD_RADIUS**2)
    return radial - tangent * axial


def _minimum_target_axial(missile: np.ndarray, cloud: np.ndarray) -> float:
    """Return the exact minimum axial coordinate over the solid target cylinder."""
    axis = (cloud - missile) / np.linalg.norm(cloud - missile)
    centered = float((TARGET_CENTER - missile) @ axis)
    horizontal_support = TARGET_RADIUS * float(np.hypot(axis[0], axis[1]))
    vertical_support = 0.5 * TARGET_HEIGHT * abs(float(axis[2]))
    return centered - horizontal_support - vertical_support


def _rim_point(angle: float, height: float) -> np.ndarray:
    return np.array(
        [
            TARGET_RADIUS * np.cos(angle),
            200.0 + TARGET_RADIUS * np.sin(angle),
            height,
        ]
    )


def _maximum_rim_margin(
    missile: np.ndarray,
    cloud: np.ndarray,
    *,
```

```
    rim_samples: int,
) -> tuple[float, np.ndarray]:
    """Maximize cone margin over the two extreme circular rims of the cylinder."""
    if rim_samples < 32:
        raise ValueError("rim_samples must be at least 32")
    angles = np.linspace(0.0, 2.0 * np.pi, rim_samples, endpoint=False)
    step = 2.0 * np.pi / rim_samples
    best_margin = -np.inf
    best_point = np.zeros(3)

    for height in (0.0, TARGET_HEIGHT):
        points = np.column_stack(
            (
                TARGET_RADIUS * np.cos(angles),
                200.0 + TARGET_RADIUS * np.sin(angles),
                np.full_like(angles, height),
            )
        )
        margins = _cone_margin(points, missile, cloud)
        candidate_indices = np.argpartition(margins, -min(4, rim_samples))[-min(4, rim_samples) :]
        for index in candidate_indices:
            center_angle = float(angles[index])

            def negative_margin(angle: float, fixed_height: float = height) -> float:
                point = _rim_point(angle, fixed_height)
                return -float(_cone_margin(point[None, :], missile, cloud)[0])

            optimized = minimize_scalar(
                negative_margin,
                bounds=(center_angle - step, center_angle + step),
                method="bounded",
                options={"xatol": 1e-13},
            )
            margin = -float(optimized.fun)
            if margin > best_margin:
                best_margin = margin
                best_point = _rim_point(float(optimized.x), height)
    return best_margin, best_point


def strict_shadow_margin(time: float, gravity: float, *, rim_samples: int) -> float:
    """Return full-cylinder shadow margin; the target is occluded iff it is non-positive."""
    missile = missile_position(time)
    cloud = cloud_position(time, gravity)
    cloud_distance = float(np.linalg.norm(cloud - missile))
    if cloud_distance <= CLOUD_RADIUS:
        return cloud_distance - CLOUD_RADIUS
    cone_margin, _ = _maximum_rim_margin(missile, cloud, rim_samples=rim_samples)
    tangent_plane_distance = np.sqrt(cloud_distance**2 - CLOUD_RADIUS**2)
    behind_margin = tangent_plane_distance - _minimum_target_axial(missile, cloud)
    return max(cone_margin, behind_margin)


def center_shadow_margin(time: float, gravity: float) -> float:
    """Return center-point shadow margin; non-positive means its sightline is occluded."""
    missile = missile_position(time)
    cloud = cloud_position(time, gravity)
    cloud_distance = float(np.linalg.norm(cloud - missile))
    if cloud_distance <= CLOUD_RADIUS:
        return cloud_distance - CLOUD_RADIUS
    cone_margin = float(_cone_margin(TARGET_CENTER[None, :], missile, cloud)[0])
    axis = (cloud - missile) / cloud_distance
```

```
    target_axial = float((TARGET_CENTER - missile) @ axis)
    behind_margin = np.sqrt(cloud_distance**2 - CLOUD_RADIUS**2) - target_axial
    return max(cone_margin, behind_margin)


def _dense_target_surface(angle_count: int, level_count: int) -> np.ndarray:
    """Sample the complete cylinder surface for an independent segment-distance check."""
    if angle_count < 32 or level_count < 3:
        raise ValueError("surface validation requires at least 32 angles and 3 levels")
    angles = np.linspace(0.0, 2.0 * np.pi, angle_count, endpoint=False)
    heights = np.linspace(0.0, TARGET_HEIGHT, level_count)
    angle_grid, height_grid = np.meshgrid(angles, heights, indexing="ij")
    side = np.column_stack(
        (
            TARGET_RADIUS * np.cos(angle_grid).ravel(),
            200.0 + TARGET_RADIUS * np.sin(angle_grid).ravel(),
            height_grid.ravel(),
        )
    )

    radii = np.linspace(0.0, TARGET_RADIUS, level_count)
    angle_grid, radius_grid = np.meshgrid(angles, radii, indexing="ij")
    cap_x = radius_grid.ravel() * np.cos(angle_grid).ravel()
    cap_y = 200.0 + radius_grid.ravel() * np.sin(angle_grid).ravel()
    bottom = np.column_stack((cap_x, cap_y, np.zeros_like(cap_x)))
    top = np.column_stack((cap_x, cap_y, np.full_like(cap_x, TARGET_HEIGHT)))
    return np.vstack((side, bottom, top))


def _maximum_segment_distance(
    points: np.ndarray,
    missile: np.ndarray,
    cloud: np.ndarray,
) -> tuple[float, np.ndarray]:
    """Independently maximize distance from the cloud center to sightline segments."""
    directions = points - missile
    squared_lengths = np.einsum("ij,ij->i", directions, directions)
    parameters = ((cloud - missile) @ directions.T) / squared_lengths
    parameters = np.clip(parameters, 0.0, 1.0)
    closest = missile + parameters[:, None] * directions
    distances = np.linalg.norm(closest - cloud, axis=1)
    index = int(np.argmax(distances))
    return float(distances[index]), points[index]


def _closest_missile_cloud_approach(gravity: float) -> tuple[float, float]:
    """Return closest approach time and distance during the cloud lifetime."""

    def squared_distance(time: float) -> float:
        difference = missile_position(time) - cloud_position(time, gravity)
        return float(difference @ difference)

    optimized = minimize_scalar(
        squared_distance,
        bounds=(EXPLOSION_TIME, EXPLOSION_TIME + CLOUD_LIFETIME),
        method="bounded",
        options={"xatol": 1e-13},
    )
    return float(optimized.x), float(np.sqrt(optimized.fun))


def _find_intervals(
```

```
    margin_function: Any,
    *,
    start: float,
    end: float,
    scan_step: float,
    root_tolerance: float,
) -> tuple[OcclusionInterval, ...]:
    """Locate non-positive intervals using a scan only for brackets and Brent roots for edges."""
    count = int(np.ceil((end - start) / scan_step))
    times = np.linspace(start, end, count + 1)
    margins = np.array([margin_function(float(time)) for time in times])
    inside = margins <= 0.0
    boundaries: list[tuple[float, bool]] = []
    for index in np.flatnonzero(inside[1:] != inside[:-1]):
        left = float(times[index])
        right = float(times[index + 1])
        root = float(brentq(margin_function, left, right, xtol=root_tolerance, rtol=1e-14))
        boundaries.append((root, bool(inside[index + 1])))

    intervals: list[OcclusionInterval] = []
    current_start = start if inside[0] else None
    for boundary, enters in boundaries:
        if enters:
            current_start = boundary
        elif current_start is not None:
            intervals.append(OcclusionInterval(current_start, boundary))
            current_start = None
    if current_start is not None:
        intervals.append(OcclusionInterval(current_start, end))
    return tuple(intervals)


def solve_question_1(config: dict[str, Any], gravity: float) -> Q1Result:
    """Solve Question 1 for one declared gravity value."""
    start = EXPLOSION_TIME
    end = EXPLOSION_TIME + CLOUD_LIFETIME
    rim_samples = int(config["rim_samples"])
    strict_intervals = _find_intervals(
        lambda time: strict_shadow_margin(time, gravity, rim_samples=rim_samples),
        start=start,
        end=end,
        scan_step=float(config["time_scan_step"]),
        root_tolerance=float(config["root_tolerance"]),
    )
    center_intervals = _find_intervals(
        lambda time: center_shadow_margin(time, gravity),
        start=start,
        end=end,
        scan_step=float(config["time_scan_step"]),
        root_tolerance=float(config["root_tolerance"]),
    )
    return Q1Result(
        gravity=gravity,
        release_point=tuple(float(value) for value in release_point()),
        explosion_point=tuple(float(value) for value in explosion_point(gravity)),
        strict_intervals=strict_intervals,
        center_intervals=center_intervals,
    )


def _interval_rows(result: Q1Result, label: str) -> list[dict[str, float | str]]:
    rows: list[dict[str, float | str]] = []
```

```
    for criterion, intervals in (
        ("strict-full-cylinder", result.strict_intervals),
        ("center-line", result.center_intervals),
    ):
        for index, interval in enumerate(intervals, start=1):
            rows.append(
                {
                    "scenario": label,
                    "gravity_m_s2": result.gravity,
                    "criterion": criterion,
                    "interval": index,
                    "start_s": interval.start,
                    "end_s": interval.end,
                    "duration_s": interval.duration,
                }
            )
    return rows


def _validation_record(config: dict[str, Any], result: Q1Result) -> dict[str, Any]:
    convergence: list[dict[str, float | int]] = []
    gravity = result.gravity
    for rim_samples in config["validation_rim_samples"]:
        intervals = _find_intervals(
            lambda time, samples=int(rim_samples): strict_shadow_margin(
                time, gravity, rim_samples=samples
            ),
            start=EXPLOSION_TIME,
            end=EXPLOSION_TIME + CLOUD_LIFETIME,
            scan_step=float(config["time_scan_step"]),
            root_tolerance=float(config["root_tolerance"]),
        )
        convergence.append(
            {
                "rim_samples": int(rim_samples),
                "duration_s": sum(interval.duration for interval in intervals),
                "interval_count": len(intervals),
            }
        )

    time_scan_convergence: list[dict[str, float | int]] = []
    for scan_step in config["validation_time_steps"]:
        intervals = _find_intervals(
            lambda time: strict_shadow_margin(
                time, gravity, rim_samples=int(config["rim_samples"])
            ),
            start=EXPLOSION_TIME,
            end=EXPLOSION_TIME + CLOUD_LIFETIME,
            scan_step=float(scan_step),
            root_tolerance=float(config["root_tolerance"]),
        )
        time_scan_convergence.append(
            {
                "scan_step_s": float(scan_step),
                "duration_s": sum(interval.duration for interval in intervals),
                "interval_count": len(intervals),
            }
        )

    boundary_checks: list[dict[str, float | bool | str]] = []
    boundary_tolerance = float(config["boundary_margin_tolerance"])
    for criterion, intervals, margin_function in (
```

```
        (
            "strict-full-cylinder",
            result.strict_intervals,
            lambda time: strict_shadow_margin(
                time, gravity, rim_samples=int(config["rim_samples"])
            ),
        ),
        ("center-line", result.center_intervals, lambda time: center_shadow_margin(time, gravity)),
    ):
        for interval in intervals:
            midpoint = 0.5 * (interval.start + interval.end)
            probe = float(config["boundary_probe_seconds"])
            for location, time, expected_state in (
                ("before-start", interval.start - probe, "outside"),
                ("start", interval.start, "boundary"),
                ("midpoint", midpoint, "inside"),
                ("end", interval.end, "boundary"),
                ("after-end", interval.end + probe, "outside"),
            ):
                margin = margin_function(time)
                if expected_state == "boundary":
                    check_passed = abs(margin) <= boundary_tolerance
                elif expected_state == "inside":
                    check_passed = margin < -boundary_tolerance
                else:
                    check_passed = margin > boundary_tolerance
                boundary_checks.append(
                    {
                        "criterion": criterion,
                        "location": location,
                        "time_s": time,
                        "margin_m": margin,
                        "expected_state": expected_state,
                        "check_passed": check_passed,
                    }
                )

    surface_points = _dense_target_surface(
        int(config["surface_validation_angles"]),
        int(config["surface_validation_levels"]),
    )
    surface_checks: list[dict[str, Any]] = []
    for interval in result.strict_intervals:
        midpoint = 0.5 * (interval.start + interval.end)
        probe = float(config["boundary_probe_seconds"])
        for location, time in (
            ("before-start", interval.start - probe),
            ("start", interval.start),
            ("midpoint", midpoint),
            ("end", interval.end),
            ("after-end", interval.end + probe),
        ):
            missile = missile_position(time)
            cloud = cloud_position(time, gravity)
            maximum_distance, worst_point = _maximum_segment_distance(
                surface_points, missile, cloud
            )
            surface_checks.append(
                {
                    "location": location,
                    "time_s": time,
                    "sampled_surface_points": int(len(surface_points)),
```

```
                    "maximum_segment_distance_m": maximum_distance,
                    "distance_margin_m": maximum_distance - CLOUD_RADIUS,
                    "occluded": maximum_distance <= CLOUD_RADIUS,
                    "worst_sampled_target_point": worst_point.tolist(),
                }
            )

    closest_time, closest_distance = _closest_missile_cloud_approach(gravity)
    return {
        "model": "q1-shadow-cone-v1",
        "criterion": "full target cylinder contained in the missile-view shadow cone",
        "target_extreme_set": "top and bottom circular rims",
        "gravity_m_s2": gravity,
        "time_scan_step_s": float(config["time_scan_step"]),
        "root_tolerance_s": float(config["root_tolerance"]),
        "boundary_margin_tolerance_m": boundary_tolerance,
        "rim_convergence": convergence,
        "time_scan_convergence": time_scan_convergence,
        "boundary_checks": boundary_checks,
        "independent_surface_segment_checks": surface_checks,
        "derived_checks": {
            "missile_hit_time_s": float(np.linalg.norm(MISSILE_INITIAL) / MISSILE_SPEED),
            "release_time_s": RELEASE_TIME,
            "explosion_time_s": EXPLOSION_TIME,
            "cloud_effective_end_s": EXPLOSION_TIME + CLOUD_LIFETIME,
            "explosion_above_ground": bool(result.explosion_point[2] > 0.0),
            "closest_missile_cloud_time_s": closest_time,
            "closest_missile_cloud_distance_m": closest_distance,
            "missile_enters_cloud": closest_distance < CLOUD_RADIUS,
        },
    }


def run_question_1(project_root: Path, config: dict[str, Any]) -> list[Path]:
    """Run Question 1 and write tables, plot, and a validation record."""
    paths = prepare_output_paths(project_root / "outputs")
    primary = solve_question_1(config, float(config["gravity"]))
    sensitivity = solve_question_1(config, float(config["gravity_sensitivity"]))

    summary = pd.DataFrame(
        [
            {
                "scenario": "primary",
                "gravity_m_s2": primary.gravity,
                "release_x_m": primary.release_point[0],
                "release_y_m": primary.release_point[1],
                "release_z_m": primary.release_point[2],
                "explosion_x_m": primary.explosion_point[0],
                "explosion_y_m": primary.explosion_point[1],
                "explosion_z_m": primary.explosion_point[2],
                "strict_duration_s": primary.strict_duration,
                "center_duration_s": primary.center_duration,
            },
            {
                "scenario": "gravity-sensitivity",
                "gravity_m_s2": sensitivity.gravity,
                "release_x_m": sensitivity.release_point[0],
                "release_y_m": sensitivity.release_point[1],
                "release_z_m": sensitivity.release_point[2],
                "explosion_x_m": sensitivity.explosion_point[0],
                "explosion_y_m": sensitivity.explosion_point[1],
                "explosion_z_m": sensitivity.explosion_point[2],
```

```
                "strict_duration_s": sensitivity.strict_duration,
                "center_duration_s": sensitivity.center_duration,
            },
        ]
    )
    intervals = pd.DataFrame(
        _interval_rows(primary, "primary") + _interval_rows(sensitivity, "gravity-sensitivity")
    )
    summary_path = save_table(summary, paths.tables / "q1_summary.csv", overwrite=True)
    intervals_path = save_table(intervals, paths.tables / "q1_intervals.csv", overwrite=True)

    times = np.linspace(
        EXPLOSION_TIME,
        EXPLOSION_TIME + CLOUD_LIFETIME,
        int(config["plot_points"]),
    )
    strict_margins = np.array(
        [
            strict_shadow_margin(
                float(time), primary.gravity, rim_samples=int(config["rim_samples"])
            )
            for time in times
        ]
    )
    center_margins = np.array(
        [center_shadow_margin(float(time), primary.gravity) for time in times]
    )
    configure_plotting()
    figure, axis = plt.subplots(figsize=(8.5, 4.8))
    axis.plot(times, strict_margins, label="Full cylinder", linewidth=2)
    axis.plot(times, center_margins, label="Center-line approximation", linestyle="--")
    axis.axhline(0.0, color="black", linewidth=1)
    axis.set_xlabel("Time since task assignment (s)")
    axis.set_ylabel("Shadow-cone margin (m)")
    axis.set_title("Question 1 occlusion margins")
    axis.legend()
    figure_path = save_figure(figure, paths.figures / "q1_occlusion_margin.svg", overwrite=True)
    plt.close(figure)

    validation = _validation_record(config, primary)
    validation["primary_result"] = {
        **asdict(primary),
        "strict_duration": primary.strict_duration,
        "center_duration": primary.center_duration,
    }
    validation["gravity_sensitivity_result"] = {
        **asdict(sensitivity),
        "strict_duration": sensitivity.strict_duration,
        "center_duration": sensitivity.center_duration,
    }
    validation_path = paths.logs / "q1_validation.json"
    validation_path.write_text(
        json.dumps(validation, ensure_ascii=False, indent=2) + "\n",
        encoding="utf-8",
    )
    return [summary_path, intervals_path, figure_path, validation_path]
```

B.3 问题二程序

```
"""Question 2 optimization using the audited Question 1 occlusion geometry."""

```

```python
from __future__ import annotations

import json
from collections.abc import Callable
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd
from scipy.optimize import differential_evolution, minimize

from mmkit.artifacts import prepare_output_paths, save_table

from .q1 import (
    CLOUD_DESCENT_SPEED,
    CLOUD_LIFETIME,
    CLOUD_RADIUS,
    MISSILE_INITIAL,
    MISSILE_SPEED,
    TARGET_CENTER,
    TARGET_HEIGHT,
    TARGET_RADIUS,
    UAV_INITIAL,
    OcclusionInterval,
    _cone_margin,
    _dense_target_surface,
    _find_intervals,
    _maximum_rim_margin,
    _maximum_segment_distance,
    _minimum_target_axial,
    missile_position,
)

MINIMUM_SPEED = 70.0
MAXIMUM_SPEED = 140.0
UAV_ALTITUDE = float(UAV_INITIAL[2])


@dataclass(frozen=True, slots=True)
class Q2Strategy:
    """One feasible FY1 release strategy."""

    theta_rad: float
    speed_m_s: float
    release_time_s: float
    fuse_delay_s: float

    @property
    def explosion_time_s(self) -> float:
        """Return explosion time from task assignment."""
        return self.release_time_s + self.fuse_delay_s


@dataclass(frozen=True, slots=True)
class Q2Evaluation:
    """Formal strict and comparison evaluation for one feasible strategy."""

    strategy: Q2Strategy
    release_point: tuple[float, float, float]
    explosion_point: tuple[float, float, float]
    strict_intervals: tuple[OcclusionInterval, ...]
```

```
    center_intervals: tuple[OcclusionInterval, ...]
    minimum_cloud_center_height_m: float
    cloud_center_below_ground: bool
    cloud_center_ground_crossing_time_s: float | None

    @property
    def strict_duration_s(self) -> float:
        """Return union length of strict full-cylinder intervals."""
        return sum(interval.duration for interval in self.strict_intervals)

    @property
    def center_duration_s(self) -> float:
        """Return union length of center-line comparison intervals."""
        return sum(interval.duration for interval in self.center_intervals)


def missile_hit_time() -> float:
    """Return time at which M1 reaches the false target at the origin."""
    return float(np.linalg.norm(MISSILE_INITIAL) / MISSILE_SPEED)


def _heading(theta_rad: float) -> np.ndarray:
    return np.array([np.cos(theta_rad), np.sin(theta_rad), 0.0])


def release_point(strategy: Q2Strategy) -> np.ndarray:
    """Return the strategy's release point."""
    return UAV_INITIAL + strategy.speed_m_s * strategy.release_time_s * _heading(strategy.theta_rad)


def explosion_point(strategy: Q2Strategy, gravity: float) -> np.ndarray:
    """Return the ballistic explosion point."""
    horizontal = strategy.speed_m_s * strategy.explosion_time_s * _heading(strategy.theta_rad)
    return (
        UAV_INITIAL + horizontal + np.array([0.0, 0.0, -0.5 * gravity * strategy.fuse_delay_s**2])
    )


def cloud_position(time: float, strategy: Q2Strategy, gravity: float) -> np.ndarray:
    """Return cloud center during the declared 20-second effective window."""
    if not strategy.explosion_time_s <= time <= strategy.explosion_time_s + CLOUD_LIFETIME:
        raise ValueError("time is outside the effective cloud lifetime")
    return explosion_point(strategy, gravity) + np.array(
        [0.0, 0.0, -CLOUD_DESCENT_SPEED * (time - strategy.explosion_time_s)]
    )


def _strategy_from_array(values: np.ndarray) -> Q2Strategy:
    return Q2Strategy(*(float(value) for value in values))


def _feasibility_violation(strategy: Q2Strategy, gravity: float) -> float:
    """Return a scaled nonnegative violation measure."""
    hit_time = missile_hit_time()
    explosion_height = float(explosion_point(strategy, gravity)[2])
    violation = max(0.0, strategy.explosion_time_s - hit_time) / hit_time
    violation += max(0.0, -explosion_height) / UAV_ALTITUDE
    return violation


def _strict_margin(
    time: float,
```

```
    strategy: Q2Strategy,
    gravity: float,
    *,
    rim_samples: int,
    refine_rim: bool,
) -> float:
    """Return the generalized full-cylinder margin for one strategy."""
    missile = missile_position(time)
    cloud = cloud_position(time, strategy, gravity)
    cloud_distance = float(np.linalg.norm(cloud - missile))
    if cloud_distance <= CLOUD_RADIUS:
        return cloud_distance - CLOUD_RADIUS
    if refine_rim:
        cone_margin, _ = _maximum_rim_margin(missile, cloud, rim_samples=rim_samples)
    else:
        angles = np.linspace(0.0, 2.0 * np.pi, rim_samples, endpoint=False)
        x = TARGET_RADIUS * np.cos(angles)
        y = 200.0 + TARGET_RADIUS * np.sin(angles)
        points = np.vstack(
            (
                np.column_stack((x, y, np.zeros_like(x))),
                np.column_stack((x, y, np.full_like(x, TARGET_HEIGHT))),
            )
        )
        cone_margin = float(np.max(_cone_margin(points, missile, cloud)))
    tangent_plane_distance = np.sqrt(cloud_distance**2 - CLOUD_RADIUS**2)
    behind_margin = tangent_plane_distance - _minimum_target_axial(missile, cloud)
    return max(cone_margin, behind_margin)


def _center_margin(time: float, strategy: Q2Strategy, gravity: float) -> float:
    """Return the center-line comparison margin for one strategy."""
    missile = missile_position(time)
    cloud = cloud_position(time, strategy, gravity)
    cloud_distance = float(np.linalg.norm(cloud - missile))
    if cloud_distance <= CLOUD_RADIUS:
        return cloud_distance - CLOUD_RADIUS
    cone_margin = float(_cone_margin(TARGET_CENTER[None, :], missile, cloud)[0])
    axis = (cloud - missile) / cloud_distance
    target_axial = float((TARGET_CENTER - missile) @ axis)
    behind_margin = np.sqrt(cloud_distance**2 - CLOUD_RADIUS**2) - target_axial
    return max(cone_margin, behind_margin)


def _effective_window(strategy: Q2Strategy) -> tuple[float, float]:
    start = strategy.explosion_time_s
    return start, min(start + CLOUD_LIFETIME, missile_hit_time())


def _sampled_metrics(
    strategy: Q2Strategy,
    margin_function: Callable[[float], float],
    *,
    time_step: float,
) -> tuple[float, float]:
    start, end = _effective_window(strategy)
    if end <= start:
        return 0.0, float("inf")
    count = max(1, int(np.ceil((end - start) / time_step)))
    times = np.linspace(start, end, count + 1)
    margins = np.array([margin_function(float(time)) for time in times])
    inside = (margins <= 0.0).astype(float)
```

```
    return float(np.trapezoid(inside, times)), float(np.min(margins))


def _objective(
    values: np.ndarray,
    gravity: float,
    *,
    criterion: str,
    time_step: float,
    rim_samples: int = 96,
) -> float:
    strategy = _strategy_from_array(values)
    violation = _feasibility_violation(strategy, gravity)
    if violation > 0.0:
        return 100.0 + 100.0 * violation
    if criterion == "center":

        def margin(time: float) -> float:
            return _center_margin(time, strategy, gravity)
    else:

        def margin(time: float) -> float:
            return _strict_margin(
                time,
                strategy,
                gravity,
                rim_samples=rim_samples,
                refine_rim=False,
            )

    duration, minimum_margin = _sampled_metrics(strategy, margin, time_step=time_step)
    if duration > 0.0:
        return -duration
    return max(minimum_margin, 0.0) / 100.0


def _root_refined_objective(
    values: np.ndarray,
    gravity: float,
    *,
    time_step: float,
    rim_samples: int,
    root_tolerance: float,
) -> float:
    """Return a locally useful duration objective with root-refined boundaries."""
    strategy = _strategy_from_array(values)
    violation = _feasibility_violation(strategy, gravity)
    if violation > 0.0:
        return 100.0 + 100.0 * violation
    start, end = _effective_window(strategy)

    def margin(time: float) -> float:
        return _strict_margin(
            time,
            strategy,
            gravity,
            rim_samples=rim_samples,
            refine_rim=False,
        )

    sampled_duration, minimum_margin = _sampled_metrics(strategy, margin, time_step=time_step)
    if sampled_duration <= 0.0:
```

```
        return max(minimum_margin, 0.0) / 100.0
    intervals = _find_intervals(
        margin,
        start=start,
        end=end,
        scan_step=time_step,
        root_tolerance=root_tolerance,
    )
    return -sum(interval.duration for interval in intervals)


def _formal_evaluation(
    strategy: Q2Strategy,
    gravity: float,
    config: dict[str, Any],
) -> Q2Evaluation:
    settings = config["formal_evaluation"]
    start, end = _effective_window(strategy)
    strict_intervals = _find_intervals(
        lambda time: _strict_margin(
            time,
            strategy,
            gravity,
            rim_samples=int(settings["rim_samples"]),
            refine_rim=True,
        ),
        start=start,
        end=end,
        scan_step=float(settings["time_scan_step_s"]),
        root_tolerance=float(settings["root_tolerance_s"]),
    )
    center_intervals = _find_intervals(
        lambda time: _center_margin(time, strategy, gravity),
        start=start,
        end=end,
        scan_step=float(settings["time_scan_step_s"]),
        root_tolerance=float(settings["root_tolerance_s"]),
    )
    release = release_point(strategy)
    explosion = explosion_point(strategy, gravity)
    minimum_height = float(explosion[2] - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME)
    ground_crossing_time = (
        strategy.explosion_time_s + float(explosion[2]) / CLOUD_DESCENT_SPEED
        if minimum_height < 0.0
        else None
    )
    return Q2Evaluation(
        strategy=strategy,
        release_point=tuple(float(value) for value in release),
        explosion_point=tuple(float(value) for value in explosion),
        strict_intervals=strict_intervals,
        center_intervals=center_intervals,
        minimum_cloud_center_height_m=minimum_height,
        cloud_center_below_ground=minimum_height < 0.0,
        cloud_center_ground_crossing_time_s=ground_crossing_time,
    )


def _bounds(config: dict[str, Any]) -> list[tuple[float, float]]:
    configured = config["bounds"]
    return [
        tuple(configured["theta_rad"]),
```

```
        tuple(configured["speed_m_s"]),
        tuple(configured["release_time_s"]),
        tuple(configured["fuse_delay_s"]),
    ]


def _initial_population(bounds: list[tuple[float, float]], popsize: int, seed: int) -> np.ndarray:
    """Build a seeded population containing a known positive-duration baseline."""
    rng = np.random.default_rng(seed)
    population_count = max(5, popsize * len(bounds))
    lower = np.array([bound[0] for bound in bounds])
    upper = np.array([bound[1] for bound in bounds])
    population = rng.uniform(lower, upper, size=(population_count, len(bounds)))
    population[0] = np.array([np.pi, 120.0, 1.5, 3.6])
    population[1] = np.array([np.pi, 140.0, 0.0, 3.6])
    population[2] = np.array([np.pi, 100.0, 0.0, 5.0])
    return population


def _deduplicate(candidates: list[np.ndarray]) -> list[np.ndarray]:
    unique: list[np.ndarray] = []
    for candidate in candidates:
        if not any(np.linalg.norm(candidate - existing) < 1e-7 for existing in unique):
            unique.append(np.asarray(candidate, dtype=float))
    return unique


def optimize_question_2(config: dict[str, Any]) -> tuple[list[Q2Evaluation], dict[str, Any]]:
    """Run center-screened and strict derivative-free optimization."""
    gravity = float(config["gravity"])
    seed = int(config["seed"])
    bounds = _bounds(config)
    center_settings = config["center_search"]
    strict_settings = config["strict_search"]
    center_init = _initial_population(bounds, int(center_settings["popsize"]), seed)

    center_result = differential_evolution(
        lambda values: _objective(
            values,
            gravity,
            criterion="center",
            time_step=float(center_settings["time_step_s"]),
        ),
        bounds,
        seed=seed,
        maxiter=int(center_settings["maxiter"]),
        popsize=int(center_settings["popsize"]),
        tol=float(center_settings["tol"]),
        polish=bool(center_settings["polish"]),
        init=center_init,
        workers=1,
    )
    center_population = np.asarray(center_result.population, dtype=float)
    center_order = np.argsort(center_result.population_energies)
    strict_init = center_population[center_order]

    strict_result = differential_evolution(
        lambda values: _objective(
            values,
            gravity,
            criterion="strict",
            time_step=float(strict_settings["time_step_s"]),
```

```
            rim_samples=int(strict_settings["rim_samples"]),
        ),
        bounds,
        seed=seed + 1,
        maxiter=int(strict_settings["maxiter"]),
        popsize=int(strict_settings["popsize"]),
        tol=float(strict_settings["tol"]),
        polish=bool(strict_settings["polish"]),
        init=strict_init,
        workers=1,
    )

    boundary_settings = config["boundary_refinement"]

    def boundary_objective(values: np.ndarray) -> float:
        full_values = np.array(
            [
                values[0],
                float(boundary_settings["speed_m_s"]),
                float(boundary_settings["release_time_s"]),
                values[1],
            ]
        )
        return _root_refined_objective(
            full_values,
            gravity,
            time_step=float(boundary_settings["time_step_s"]),
            rim_samples=int(boundary_settings["rim_samples"]),
            root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
        )

    boundary_result = differential_evolution(
        boundary_objective,
        [
            tuple(boundary_settings["theta_rad"]),
            tuple(boundary_settings["fuse_delay_s"]),
        ],
        seed=int(boundary_settings["seed"]),
        maxiter=int(boundary_settings["maxiter"]),
        popsize=int(boundary_settings["popsize"]),
        tol=float(boundary_settings["tol"]),
        polish=True,
        workers=1,
    )
    boundary_candidate = np.array(
        [
            boundary_result.x[0],
            float(boundary_settings["speed_m_s"]),
            float(boundary_settings["release_time_s"]),
            boundary_result.x[1],
        ]
    )
    warm_start = boundary_settings["independent_warm_start"]
    independent_warm_start = np.array(
        [
            float(warm_start["theta_rad"]),
            float(boundary_settings["speed_m_s"]),
            float(boundary_settings["release_time_s"]),
            float(warm_start["fuse_delay_s"]),
        ]
    )

```

```
    raw_candidates = [
        center_result.x,
        strict_result.x,
        boundary_candidate,
        independent_warm_start,
    ]
    strict_population = np.asarray(strict_result.population, dtype=float)
    strict_order = np.argsort(strict_result.population_energies)
    retained_population = max(8, int(config["local_refinement"]["starts"]))
    raw_candidates.extend(strict_population[strict_order[:retained_population]])

    local_results = []
    local_settings = config["local_refinement"]
    for start_values in _deduplicate(raw_candidates)[: int(local_settings["starts"])]:
        result = minimize(
            lambda values: _root_refined_objective(
                values,
                gravity,
                time_step=float(strict_settings["time_step_s"]),
                rim_samples=int(strict_settings["rim_samples"]),
                root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
            ),
            start_values,
            method="Nelder-Mead",
            bounds=bounds,
            options={
                "maxfev": int(local_settings["maxfev"]),
                "xatol": float(local_settings["xtol"]),
                "fatol": float(local_settings["ftol"]),
            },
        )
        local_results.append(result)
        raw_candidates.append(result.x)

    feasible_candidates = [
        candidate
        for candidate in _deduplicate(raw_candidates)
        if _feasibility_violation(_strategy_from_array(candidate), gravity) <= 1e-12
    ]
    ranked_coarse = sorted(
        feasible_candidates,
        key=lambda values: _root_refined_objective(
            values,
            gravity,
            time_step=float(strict_settings["time_step_s"]),
            rim_samples=int(strict_settings["rim_samples"]),
            root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
        ),
    )
    formal_count = min(int(config["formal_evaluation"]["candidate_count"]), len(ranked_coarse))
    formal = [
        _formal_evaluation(_strategy_from_array(values), gravity, config)
        for values in ranked_coarse[:formal_count]
    ]
    formal.sort(key=lambda evaluation: evaluation.strict_duration_s, reverse=True)

    solver_record = {
        "seed": seed,
        "variable_order": ["theta_rad", "speed_m_s", "release_time_s", "fuse_delay_s"],
        "bounds": config["bounds"],
        "candidate_initialization": {
            "mechanism": "seeded uniform population with Q1 and two heading baselines injected",
```

```
            "reason": "positive-duration strategies occupy a sparse subset of the box bounds",
            "baseline_q1": [float(np.pi), 120.0, 1.5, 3.6],
        },
        "center_differential_evolution": {
            "success": bool(center_result.success),
            "message": str(center_result.message),
            "iterations": int(center_result.nit),
            "evaluations": int(center_result.nfev),
            "best_sampled_duration_s": -float(center_result.fun),
            "settings": center_settings,
        },
        "strict_differential_evolution": {
            "success": bool(strict_result.success),
            "message": str(strict_result.message),
            "iterations": int(strict_result.nit),
            "evaluations": int(strict_result.nfev),
            "best_sampled_duration_s": -float(strict_result.fun),
            "settings": strict_settings,
        },
        "independent_boundary_refinement": {
            "success": bool(boundary_result.success),
            "message": str(boundary_result.message),
            "iterations": int(boundary_result.nit),
            "evaluations": int(boundary_result.nfev),
            "root_refined_duration_s": -float(boundary_result.fun),
            "theta_rad": float(boundary_result.x[0]),
            "fuse_delay_s": float(boundary_result.x[1]),
            "settings": boundary_settings,
        },
        "local_refinements": [
            {
                "success": bool(result.success),
                "message": str(result.message),
                "evaluations": int(result.nfev),
                "feasible": _feasibility_violation(_strategy_from_array(result.x), gravity)
                <= 1e-12,
                "root_refined_duration_s": (
                    -float(result.fun) if float(result.fun) <= 0.0 else None
                ),
            }
            for result in local_results
        ],
        "formal_evaluation": config["formal_evaluation"],
        "feasible_candidate_count": len(feasible_candidates),
        "formal_candidate_count": len(formal),
        "convergence_claimed": bool(center_result.success and strict_result.success),
        "status": (
            "converged"
            if center_result.success and strict_result.success
            else "budget-exhausted-with-feasible-candidates"
        ),
    }
    return formal, solver_record


def _summary_row(evaluation: Q2Evaluation, rank: int) -> dict[str, Any]:
    strategy = evaluation.strategy
    return {
        "rank": rank,
        "theta_rad": strategy.theta_rad,
        "theta_deg": float(np.degrees(strategy.theta_rad) % 360.0),
        "speed_m_s": strategy.speed_m_s,
```

```
        "release_time_s": strategy.release_time_s,
        "release_x_m": evaluation.release_point[0],
        "release_y_m": evaluation.release_point[1],
        "release_z_m": evaluation.release_point[2],
        "fuse_delay_s": strategy.fuse_delay_s,
        "explosion_time_s": strategy.explosion_time_s,
        "explosion_x_m": evaluation.explosion_point[0],
        "explosion_y_m": evaluation.explosion_point[1],
        "explosion_z_m": evaluation.explosion_point[2],
        "strict_duration_s": evaluation.strict_duration_s,
        "center_duration_s": evaluation.center_duration_s,
        "minimum_cloud_center_height_m": evaluation.minimum_cloud_center_height_m,
        "cloud_center_below_ground": evaluation.cloud_center_below_ground,
        "cloud_center_ground_crossing_time_s": (evaluation.cloud_center_ground_crossing_time_s),
    }


def _strict_intervals_with_settings(
    strategy: Q2Strategy,
    gravity: float,
    *,
    time_scan_step: float,
    rim_samples: int,
    root_tolerance: float,
) -> tuple[OcclusionInterval, ...]:
    start, end = _effective_window(strategy)
    return _find_intervals(
        lambda time: _strict_margin(
            time,
            strategy,
            gravity,
            rim_samples=rim_samples,
            refine_rim=True,
        ),
        start=start,
        end=end,
        scan_step=time_scan_step,
        root_tolerance=root_tolerance,
    )


def _q2_validation_record(best: Q2Evaluation, config: dict[str, Any]) -> dict[str, Any]:
    """Build numerical, boundary, independent-geometry, and perturbation checks."""
    gravity = float(config["gravity"])
    validation = config["validation"]
    root_tolerance = float(config["formal_evaluation"]["root_tolerance_s"])
    convergence = []
    for settings in validation["convergence_settings"]:
        intervals = _strict_intervals_with_settings(
            best.strategy,
            gravity,
            time_scan_step=float(settings["time_scan_step_s"]),
            rim_samples=int(settings["rim_samples"]),
            root_tolerance=root_tolerance,
        )
        convergence.append(
            {
                **settings,
                "interval_count": len(intervals),
                "strict_duration_s": sum(interval.duration for interval in intervals),
            }
        )
```

```
623
624     boundary_checks = []
625     probe = float(validation["boundary_probe_s"])
626     for interval in best.strict_intervals:
627         midpoint = 0.5 * (interval.start + interval.end)
628         explosion_time = best.strategy.explosion_time_s
629         locations = [
630             ("pre-explosion", explosion_time - probe, "inactive"),
631             ("start", interval.start, "boundary"),
632             ("midpoint", midpoint, "inside"),
633             ("end", interval.end, "boundary"),
634             ("after-end", interval.end + probe, "outside"),
635         ]
636         if interval.start > explosion_time:
637             active_left_probe = interval.start - min(probe, 0.5 * (interval.start - explosion_time))
638             locations.insert(1, ("before-start", active_left_probe, "outside"))
639         for location, time, expected in locations:
640             active = (
641                 best.strategy.explosion_time_s
642                 <= time
643                 <= (best.strategy.explosion_time_s + CLOUD_LIFETIME)
644             )
645             margin = (
646                 _strict_margin(
647                     time,
648                     best.strategy,
649                     gravity,
650                     rim_samples=int(config["formal_evaluation"]["rim_samples"]),
651                     refine_rim=True,
652                 )
653                 if active
654                 else None
655             )
656             boundary_checks.append(
657                 {
658                     "location": location,
659                     "time_s": time,
660                     "expected_state": expected,
661                     "strict_margin_m": margin,
662                     "cloud_active": active,
663                 }
664             )
665
666     surface_points = _dense_target_surface(
667         int(validation["surface_angles"]), int(validation["surface_levels"])
668     )
669     independent_checks = []
670     for boundary in boundary_checks:
671         time = float(boundary["time_s"])
672         if not boundary["cloud_active"]:
673             independent_checks.append(
674                 {
675                     "location": boundary["location"],
676                     "time_s": time,
677                     "cloud_active": False,
678                     "occluded": False,
679                 }
680             )
681             continue
682         maximum_distance, worst_point = _maximum_segment_distance(
683             surface_points,
684             missile_position(time),
```

```
            cloud_position(time, best.strategy, gravity),
        )
        independent_checks.append(
            {
                "location": boundary["location"],
                "time_s": time,
                "cloud_active": True,
                "sampled_surface_points": len(surface_points),
                "maximum_segment_distance_m": maximum_distance,
                "distance_margin_m": maximum_distance - CLOUD_RADIUS,
                "occluded": maximum_distance <= CLOUD_RADIUS,
                "worst_sampled_target_point": worst_point.tolist(),
            }
        )

    perturbations = {
        "theta-minus": (-float(validation["perturbation_theta_rad"]), 0.0, 0.0, 0.0),
        "theta-plus": (float(validation["perturbation_theta_rad"]), 0.0, 0.0, 0.0),
        "speed-plus": (0.0, float(validation["perturbation_speed_m_s"]), 0.0, 0.0),
        "release-plus": (0.0, 0.0, float(validation["perturbation_release_time_s"]), 0.0),
        "fuse-minus": (0.0, 0.0, 0.0, -float(validation["perturbation_fuse_delay_s"])),
        "fuse-plus": (0.0, 0.0, 0.0, float(validation["perturbation_fuse_delay_s"])),
    }
    baseline = np.array(list(asdict(best.strategy).values()), dtype=float)
    perturbation_checks = []
    for label, offset in perturbations.items():
        candidate = baseline + np.asarray(offset)
        objective = _root_refined_objective(
            candidate,
            gravity,
            time_step=float(validation["perturbation_time_step_s"]),
            rim_samples=int(validation["perturbation_rim_samples"]),
            root_tolerance=root_tolerance,
        )
        perturbation_checks.append(
            {
                "perturbation": label,
                "strategy": asdict(_strategy_from_array(candidate)),
                "root_refined_strict_duration_s": -objective if objective <= 0.0 else 0.0,
                "feasible": _feasibility_violation(_strategy_from_array(candidate), gravity)
                <= 1e-12,
            }
        )

    q1_baseline = _formal_evaluation(Q2Strategy(np.pi, 120.0, 1.5, 3.6), gravity, config)
    return {
        "convergence": convergence,
        "boundary_checks": boundary_checks,
        "independent_full_surface_segment_checks": independent_checks,
        "local_perturbation_checks": perturbation_checks,
        "q1_kernel_regression": {
            "strategy": asdict(q1_baseline.strategy),
            "strict_duration_s": q1_baseline.strict_duration_s,
            "expected_q1_strict_duration_s": 1.3916426681980933,
            "absolute_difference_s": abs(q1_baseline.strict_duration_s - 1.3916426681980933),
        },
    }


def run_question_2(project_root: Path, config: dict[str, Any]) -> list[Path]:
    """Optimize Question 2 and write auditable tables and solver metadata."""
    paths = prepare_output_paths(project_root / "outputs")
```

```
    evaluations, solver_record = optimize_question_2(config)
    if not evaluations:
        raise RuntimeError("Question 2 optimization produced no feasible formal candidate")

    near_count = min(int(config["near_optimal_count"]), len(evaluations))
    candidate_rows = [
        _summary_row(item, rank) for rank, item in enumerate(evaluations[:near_count], 1)
    ]
    summary_path = save_table(
        pd.DataFrame(candidate_rows[:1]), paths.tables / "q2_summary.csv", overwrite=True
    )
    candidates_path = save_table(
        pd.DataFrame(candidate_rows), paths.tables / "q2_candidates.csv", overwrite=True
    )

    interval_rows: list[dict[str, Any]] = []
    for rank, evaluation in enumerate(evaluations[:near_count], 1):
        for criterion, intervals in (
            ("strict-full-cylinder", evaluation.strict_intervals),
            ("center-line", evaluation.center_intervals),
        ):
            for number, interval in enumerate(intervals, 1):
                interval_rows.append(
                    {
                        "rank": rank,
                        "criterion": criterion,
                        "interval": number,
                        "start_s": interval.start,
                        "end_s": interval.end,
                        "duration_s": interval.duration,
                    }
                )
    intervals_path = save_table(
        pd.DataFrame(interval_rows), paths.tables / "q2_intervals.csv", overwrite=True
    )

    best = evaluations[0]
    record = {
        "model": "q2-two-stage-derivative-free-v1",
        "objective": "maximize union duration of strict full-cylinder occlusion",
        "gravity_m_s2": float(config["gravity"]),
        "best": {
            **_summary_row(best, 1),
            "strict_intervals": [asdict(interval) for interval in best.strict_intervals],
            "center_intervals": [asdict(interval) for interval in best.center_intervals],
        },
        "near_optimal_candidates": candidate_rows,
        "solver": solver_record,
        "ground_rule": (
            "Cloud continues descending for 20 seconds as stated; no additional ground cutoff."
        ),
        "feasibility_checks": {
            "speed_within_bounds": MINIMUM_SPEED <= best.strategy.speed_m_s <= MAXIMUM_SPEED,
            "release_time_nonnegative": best.strategy.release_time_s >= 0.0,
            "fuse_delay_nonnegative": best.strategy.fuse_delay_s >= 0.0,
            "explosion_before_missile_hit": best.strategy.explosion_time_s <= missile_hit_time(),
            "explosion_above_ground": best.explosion_point[2] >= 0.0,
            "cloud_center_below_ground_during_effective_window": best.cloud_center_below_ground,
        },
    }
    log_path = paths.logs / "q2_optimization.json"
    log_path.write_text(json.dumps(record, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
```

```
    validation_record = _q2_validation_record(best, config)
    validation_path = paths.logs / "q2_validation.json"
    validation_path.write_text(
        json.dumps(validation_record, ensure_ascii=False, indent=2) + "\n",
        encoding="utf-8",
    )
    return [summary_path, candidates_path, intervals_path, log_path, validation_path]
```

B.4 问题三程序

```
"""Question 3 joint line-of-sight coverage by three smoke clouds."""

from __future__ import annotations

import json
from collections.abc import Callable, Sequence
from dataclasses import asdict, dataclass
from functools import lru_cache
from itertools import permutations
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd
from scipy.optimize import brentq, differential_evolution, minimize

from mmkit.artifacts import prepare_output_paths, save_table

from .q1 import (
    CLOUD_DESCENT_SPEED,
    CLOUD_LIFETIME,
    CLOUD_RADIUS,
    TARGET_HEIGHT,
    TARGET_RADIUS,
    UAV_INITIAL,
    OcclusionInterval,
    _dense_target_surface,
    missile_position,
)
from .q2 import Q2Strategy, missile_hit_time
from .q2 import _formal_evaluation as q2_formal_evaluation
from .q2 import explosion_point as single_explosion_point
from .q2 import release_point as single_release_point


@dataclass(frozen=True, slots=True)
class Q3Strategy:
    """Shared FY1 flight controls and three release/fuse controls."""

    theta_rad: float
    speed_m_s: float
    s0_s: float
    s1_s: float
    s2_s: float
    fuse1_s: float
    fuse2_s: float
    fuse3_s: float

    @property
    def release_times_s(self) -> tuple[float, float, float]:
        """Return release times under the automatic one-second-gap parameterization."""
```

```
        return (
            self.s0_s,
            self.s0_s + 1.0 + self.s1_s,
            self.s0_s + 2.0 + self.s1_s + self.s2_s,
        )

    @property
    def fuse_delays_s(self) -> tuple[float, float, float]:
        """Return the three independently selected fuse delays."""
        return (self.fuse1_s, self.fuse2_s, self.fuse3_s)

    @property
    def explosion_times_s(self) -> tuple[float, float, float]:
        """Return explosion times; their ordering is deliberately unconstrained."""
        return tuple(
            release + fuse
            for release, fuse in zip(self.release_times_s, self.fuse_delays_s, strict=True)
        )


@dataclass(frozen=True, slots=True)
class BombGeometry:
    """Derived geometry for one bomb."""

    bomb: int
    release_time_s: float
    fuse_delay_s: float
    explosion_time_s: float
    release_point: tuple[float, float, float]
    explosion_point: tuple[float, float, float]


@dataclass(frozen=True, slots=True)
class Q3Evaluation:
    """Formal joint and conservative evaluation of one three-bomb strategy."""

    strategy: Q3Strategy
    bombs: tuple[BombGeometry, ...]
    joint_intervals: tuple[OcclusionInterval, ...]
    individual_strict_intervals: tuple[tuple[OcclusionInterval, ...], ...]
    conservative_union_intervals: tuple[OcclusionInterval, ...]
    deletion_durations_s: tuple[float, float, float]
    ordered_incremental_durations_s: tuple[float, float, float]

    @property
    def joint_duration_s(self) -> float:
        return _duration(self.joint_intervals)

    @property
    def conservative_duration_s(self) -> float:
        return _duration(self.conservative_union_intervals)


@dataclass(frozen=True, slots=True)
class CoverageAssessment:
    """Adaptive surface certificate or boundary-level unresolved assessment."""

    status: str
    signed_margin_m: float
    sampled_lower_bound_m: float
    lipschitz_upper_bound_m: float
    surface_angles: int
```

```
    surface_levels: int
    refinement_trace: tuple[dict[str, float | int], ...]


def _strategy_from_array(values: np.ndarray) -> Q3Strategy:
    return Q3Strategy(*(float(value) for value in values))


def _bomb_strategies(strategy: Q3Strategy) -> tuple[Q2Strategy, ...]:
    return tuple(
        Q2Strategy(
            strategy.theta_rad,
            strategy.speed_m_s,
            release_time,
            fuse,
        )
        for release_time, fuse in zip(strategy.release_times_s, strategy.fuse_delays_s, strict=True)
    )


def _bomb_geometries(strategy: Q3Strategy, gravity: float) -> tuple[BombGeometry, ...]:
    rows = []
    for index, bomb in enumerate(_bomb_strategies(strategy), 1):
        release = single_release_point(bomb)
        explosion = single_explosion_point(bomb, gravity)
        rows.append(
            BombGeometry(
                bomb=index,
                release_time_s=bomb.release_time_s,
                fuse_delay_s=bomb.fuse_delay_s,
                explosion_time_s=bomb.explosion_time_s,
                release_point=tuple(float(value) for value in release),
                explosion_point=tuple(float(value) for value in explosion),
            )
        )
    return tuple(rows)


def _feasibility_violation(strategy: Q3Strategy, gravity: float) -> float:
    hit_time = missile_hit_time()
    violation = 0.0
    values = asdict(strategy).values()
    if any(value < 0.0 for value in values):
        violation += sum(max(0.0, -value) for value in values)
    for bomb in _bomb_strategies(strategy):
        violation += max(0.0, bomb.explosion_time_s - hit_time) / hit_time
        height = float(single_explosion_point(bomb, gravity)[2])
        violation += max(0.0, -height) / float(UAV_INITIAL[2])
    return violation


def _cloud_center(time: float, bomb: BombGeometry) -> np.ndarray:
    return np.asarray(bomb.explosion_point) + np.array(
        [0.0, 0.0, -CLOUD_DESCENT_SPEED * (time - bomb.explosion_time_s)]
    )


def _active_bombs(time: float, bombs: Sequence[BombGeometry]) -> tuple[BombGeometry, ...]:
    return tuple(
        bomb
        for bomb in bombs
        if bomb.explosion_time_s <= time <= bomb.explosion_time_s + CLOUD_LIFETIME
```

```
    )


@lru_cache(maxsize=24)
def _surface_points(angle_count: int, level_count: int) -> np.ndarray:
    points = _dense_target_surface(angle_count, level_count)
    points.setflags(write=False)
    return points


def _point_cloud_distances(
    points: np.ndarray, missile: np.ndarray, cloud: np.ndarray
) -> np.ndarray:
    """Return cloud-center distances to missile-to-target-point segments."""
    directions = points - missile
    squared_lengths = np.einsum("ij,ij->i", directions, directions)
    parameters = ((cloud - missile) @ directions.T) / squared_lengths
    parameters = np.clip(parameters, 0.0, 1.0)
    closest = missile + parameters[:, None] * directions
    return np.linalg.norm(closest - cloud, axis=1)


def _coverage_margins(
    points: np.ndarray,
    missile: np.ndarray,
    active_bombs: Sequence[BombGeometry],
    time: float,
) -> np.ndarray:
    """Return pointwise best-cloud margins; nonpositive means jointly covered."""
    if not active_bombs:
        return np.full(len(points), np.inf)
    distances = np.vstack(
        [
            _point_cloud_distances(points, missile, _cloud_center(time, bomb))
            for bomb in active_bombs
        ]
    )
    return np.min(distances, axis=0) - CLOUD_RADIUS


def _target_point(surface: str, first: float, second: float) -> np.ndarray:
    if surface == "side":
        angle, height = first, second
        radius = TARGET_RADIUS
        z = height
    else:
        angle, radius = first, second
        z = 0.0 if surface == "bottom" else TARGET_HEIGHT
    return np.array([radius * np.cos(angle), 200.0 + radius * np.sin(angle), z], dtype=float)


def _continuous_worst_margin(
    missile: np.ndarray,
    active: Sequence[BombGeometry],
    time: float,
    sampled_points: np.ndarray,
    sampled_margins: np.ndarray,
    *,
    starts: int,
    maxfev: int,
) -> float:
    """Refine the worst sampled ray continuously on side and both caps."""
```

```
    block_size = len(sampled_points) // 3
    order_parts = [
        offset + np.argsort(sampled_margins[offset : offset + block_size])[-starts:]
        for offset in (0, block_size, 2 * block_size)
    ]
    order = np.concatenate(order_parts)
    seeds: list[tuple[str, np.ndarray]] = []
    for index in order:
        point = sampled_points[int(index)]
        angle = float(np.arctan2(point[1] - 200.0, point[0]) % (2.0 * np.pi))
        radial = float(np.clip(np.hypot(point[0], point[1] - 200.0), 0.0, TARGET_RADIUS))
        seeds.append(("side", np.array([angle, float(np.clip(point[2], 0, TARGET_HEIGHT))])))
        if abs(point[2]) < 1e-9:
            seeds.append(("bottom", np.array([angle, radial])))
        if abs(point[2] - TARGET_HEIGHT) < 1e-9:
            seeds.append(("top", np.array([angle, radial])))

    best = float(np.max(sampled_margins))
    for surface, seed in seeds:
        upper_second = TARGET_HEIGHT if surface == "side" else TARGET_RADIUS

        def negative_margin(parameters: np.ndarray, fixed_surface: str = surface) -> float:
            point = _target_point(fixed_surface, float(parameters[0]), float(parameters[1]))
            margin = _coverage_margins(point[None, :], missile, active, time)[0]
            return -float(margin)

        result = minimize(
            negative_margin,
            seed,
            method="Nelder-Mead",
            bounds=[(0.0, 2.0 * np.pi), (0.0, upper_second)],
            options={"maxfev": maxfev, "xatol": 1e-9, "fatol": 1e-9},
        )
        best = max(best, -float(result.fun))
    return best


def _surface_covering_radius(angle_count: int, level_count: int) -> float:
    """Return a Euclidean covering-radius bound for the sampled cylinder surface."""
    angular_half_chord = 2.0 * TARGET_RADIUS * np.sin(np.pi / (2.0 * angle_count))
    linear_half_step = max(TARGET_HEIGHT, TARGET_RADIUS) / (2.0 * (level_count - 1))
    return float(np.hypot(angular_half_chord, linear_half_step))


def joint_coverage_margin(
    time: float,
    bombs: Sequence[BombGeometry],
    *,
    surface_angles: int,
    surface_levels: int,
    continuous_refinement: bool,
    refinement_starts: int = 0,
    refinement_maxfev: int = 0,
    adaptive_max_surface_angles: int | None = None,
    adaptive_max_surface_levels: int | None = None,
) -> float:
    """Return max-over-rays min-over-active-clouds coverage margin."""
    return joint_coverage_assessment(
        time,
        bombs,
        surface_angles=surface_angles,
        surface_levels=surface_levels,
```

```
        continuous_refinement=continuous_refinement,
        refinement_starts=refinement_starts,
        refinement_maxfev=refinement_maxfev,
        adaptive_max_surface_angles=adaptive_max_surface_angles,
        adaptive_max_surface_levels=adaptive_max_surface_levels,
    ).signed_margin_m


def joint_coverage_assessment(
    time: float,
    bombs: Sequence[BombGeometry],
    *,
    surface_angles: int,
    surface_levels: int,
    continuous_refinement: bool,
    refinement_starts: int = 0,
    refinement_maxfev: int = 0,
    adaptive_max_surface_angles: int | None = None,
    adaptive_max_surface_levels: int | None = None,
) -> CoverageAssessment:
    """Adaptively certify covered/uncovered rays using a Lipschitz grid bound."""
    active = _active_bombs(time, bombs)
    if not active:
        return CoverageAssessment(
            "uncovered-no-active-cloud",
            float("inf"),
            float("inf"),
            float("inf"),
            surface_angles,
            surface_levels,
            (),
        )
    missile = missile_position(time)
    max_angles = adaptive_max_surface_angles or surface_angles
    max_levels = adaptive_max_surface_levels or surface_levels
    current_angles = surface_angles
    current_levels = surface_levels
    trace: list[dict[str, float | int]] = []
    while True:
        points = _surface_points(current_angles, current_levels)
        margins = _coverage_margins(points, missile, active, time)
        sampled_maximum = float(np.max(margins))
        covering_radius = _surface_covering_radius(current_angles, current_levels)
        upper_bound = sampled_maximum + covering_radius
        trace.append(
            {
                "surface_angles": current_angles,
                "surface_levels": current_levels,
                "covering_radius_m": covering_radius,
                "sampled_lower_bound_m": sampled_maximum,
                "lipschitz_upper_bound_m": upper_bound,
            }
        )
        if not continuous_refinement or sampled_maximum > 0.0:
            status = "uncovered-certified" if sampled_maximum > 0.0 else "sampled-covered"
            return CoverageAssessment(
                status,
                sampled_maximum,
                sampled_maximum,
                upper_bound,
                current_angles,
                current_levels,
```

```
                tuple(trace),
            )
        if upper_bound <= 0.0:
            return CoverageAssessment(
                "covered-certified",
                upper_bound,
                sampled_maximum,
                upper_bound,
                current_angles,
                current_levels,
                tuple(trace),
            )
        if current_angles >= max_angles and current_levels >= max_levels:
            break
        current_angles = min(max_angles, current_angles * 2)
        current_levels = min(max_levels, 2 * (current_levels - 1) + 1)

    local_margin = _continuous_worst_margin(
        missile,
        active,
        time,
        points,
        margins,
        starts=refinement_starts,
        maxfev=refinement_maxfev,
    )
    status = "uncovered-certified-local-witness" if local_margin > 0.0 else "boundary-uncertain"
    return CoverageAssessment(
        status,
        local_margin,
        sampled_maximum,
        upper_bound,
        current_angles,
        current_levels,
        tuple(trace),
    )


def _merge_intervals(intervals: Sequence[OcclusionInterval]) -> tuple[OcclusionInterval, ...]:
    if not intervals:
        return ()
    ordered = sorted(intervals, key=lambda item: item.start)
    merged = [ordered[0]]
    for interval in ordered[1:]:
        previous = merged[-1]
        if interval.start <= previous.end + 1e-9:
            merged[-1] = OcclusionInterval(previous.start, max(previous.end, interval.end))
        else:
            merged.append(interval)
    return tuple(merged)


def _duration(intervals: Sequence[OcclusionInterval]) -> float:
    return sum(interval.duration for interval in intervals)


def _find_joint_intervals(
    margin: Callable[[float], float],
    bombs: Sequence[BombGeometry],
    *,
    scan_step: float,
    root_tolerance: float,
```

```
) -> tuple[OcclusionInterval, ...]:
    """Find union intervals, preserving cloud activation/deactivation breakpoints."""
    start = min(bomb.explosion_time_s for bomb in bombs)
    end = min(missile_hit_time(), max(bomb.explosion_time_s + CLOUD_LIFETIME for bomb in bombs))
    regular = np.linspace(start, end, int(np.ceil((end - start) / scan_step)) + 1)
    events = [
        event
        for bomb in bombs
        for event in (bomb.explosion_time_s, bomb.explosion_time_s + CLOUD_LIFETIME)
        if start <= event <= end
    ]
    event_probes = []
    probe = max(root_tolerance * 10.0, scan_step * 1e-5)
    for event in events:
        event_probes.extend((max(start, event - probe), min(end, event + probe)))
    times = np.unique(np.concatenate((regular, np.asarray(events), np.asarray(event_probes))))
    margins = np.array([margin(float(time)) for time in times])
    inside = margins <= 0.0
    intervals: list[OcclusionInterval] = []
    current = float(times[0]) if inside[0] else None
    for index in np.flatnonzero(inside[1:] != inside[:-1]):
        left, right = float(times[index]), float(times[index + 1])
        midpoint = 0.5 * (left + right)
        left_active = tuple(bomb.bomb for bomb in _active_bombs(left, bombs))
        middle_active = tuple(bomb.bomb for bomb in _active_bombs(midpoint, bombs))
        right_active = tuple(bomb.bomb for bomb in _active_bombs(right, bombs))
        if left_active != middle_active and middle_active == right_active:
            boundary = left
        elif left_active == middle_active and middle_active != right_active:
            boundary = right
        else:
            boundary = float(brentq(margin, left, right, xtol=root_tolerance, rtol=1e-14))
        if inside[index + 1]:
            current = boundary
        elif current is not None:
            intervals.append(OcclusionInterval(current, boundary))
            current = None
    if current is not None:
        intervals.append(OcclusionInterval(current, float(times[-1])))
    return _merge_intervals(intervals)


def _settings_margin(
    bombs: Sequence[BombGeometry], settings: dict[str, Any], *, refine: bool
) -> Callable[[float], float]:
    return lambda time: joint_coverage_margin(
        time,
        bombs,
        surface_angles=int(settings["surface_angles"]),
        surface_levels=int(settings["surface_levels"]),
        continuous_refinement=refine,
        refinement_starts=int(settings.get("continuous_refinement_starts", 0)),
        refinement_maxfev=int(settings.get("continuous_refinement_maxfev", 0)),
        adaptive_max_surface_angles=int(
            settings.get("adaptive_max_surface_angles", settings["surface_angles"])
        ),
        adaptive_max_surface_levels=int(
            settings.get("adaptive_max_surface_levels", settings["surface_levels"])
        ),
    )


```

```
def _sampled_metrics(
    bombs: Sequence[BombGeometry], margin: Callable[[float], float], time_step: float
) -> tuple[float, float]:
    start = min(bomb.explosion_time_s for bomb in bombs)
    end = min(missile_hit_time(), max(bomb.explosion_time_s + CLOUD_LIFETIME for bomb in bombs))
    anchored = np.arange(0.0, missile_hit_time() + 0.5 * time_step, time_step)
    anchored = anchored[(anchored >= start) & (anchored <= end)]
    events = np.array(
        [
            event
            for bomb in bombs
            for event in (bomb.explosion_time_s, bomb.explosion_time_s + CLOUD_LIFETIME)
            if start <= event <= end
        ]
    )
    times = np.unique(np.concatenate(([start, end], anchored, events)))
    margins = np.array([margin(float(time)) for time in times])
    inside = (margins <= 0.0).astype(float)
    return float(np.trapezoid(inside, times)), float(np.min(margins))


def _objective(
    values: np.ndarray, gravity: float, settings: dict[str, Any], *, roots: bool
) -> float:
    strategy = _strategy_from_array(values)
    violation = _feasibility_violation(strategy, gravity)
    if violation > 0.0:
        return 100.0 + 100.0 * violation
    bombs = _bomb_geometries(strategy, gravity)
    margin = _settings_margin(bombs, settings, refine=False)
    duration, minimum = _sampled_metrics(bombs, margin, float(settings["time_step_s"]))
    if duration <= 0.0:
        return max(minimum, 0.0) / 100.0
    if not roots:
        return -duration
    intervals = _find_joint_intervals(
        margin,
        bombs,
        scan_step=float(settings["time_step_s"]),
        root_tolerance=1e-8,
    )
    return -_duration(intervals)


def _bounds(config: dict[str, Any]) -> list[tuple[float, float]]:
    bounds = config["bounds"]
    fuse = tuple(bounds["fuse_delay_s"])
    return [
        tuple(bounds["theta_rad"]),
        tuple(bounds["speed_m_s"]),
        tuple(bounds["s0_s"]),
        tuple(bounds["s1_s"]),
        tuple(bounds["s2_s"]),
        fuse,
        fuse,
        fuse,
    ]


def _warm_vector(config: dict[str, Any]) -> np.ndarray:
    warm = config["q2_warm_start"]
    return np.array(
```

```
        [
            warm["theta_rad"],
            warm["speed_m_s"],
            warm["s0_s"],
            warm["s1_s"],
            warm["s2_s"],
            *warm["fuse_delays_s"],
        ],
        dtype=float,
    )


def _initial_population(config: dict[str, Any]) -> np.ndarray:
    bounds = _bounds(config)
    settings = config["global_search"]
    rng = np.random.default_rng(int(config["seed"]))
    count = int(settings["popsize"]) * len(bounds)
    lower = np.array([item[0] for item in bounds])
    upper = np.array([item[1] for item in bounds])
    population = rng.uniform(lower, upper, size=(count, len(bounds)))
    warm = _warm_vector(config)
    structured = []
    fuse_families = (
        (2.4968, 2.7, 0.5),
        (2.4968, 1.5, 0.5),
        (2.4968, 3.5, 4.5),
        (2.4968, 2.7, 2.0),
    )
    for speed in (70.0, 90.0, 110.0, 140.0):
        for theta_offset, fuses in zip((-0.03, -0.01, 0.01, 0.03), fuse_families, strict=True):
            structured.append(
                np.array([warm[0] + theta_offset, speed, 0.0, 0.0, 0.0, *fuses], dtype=float)
            )
    structured.insert(0, warm)
    for index, candidate in enumerate(structured[: len(population)]):
        population[index] = np.clip(candidate, lower, upper)
    return population


def _deduplicate(values: Sequence[np.ndarray]) -> list[np.ndarray]:
    unique: list[np.ndarray] = []
    for value in values:
        if not any(np.linalg.norm(value - existing) < 1e-7 for existing in unique):
            unique.append(np.asarray(value, dtype=float))
    return unique


def _individual_strict_intervals(
    strategy: Q3Strategy, gravity: float, config: dict[str, Any]
) -> tuple[tuple[OcclusionInterval, ...], ...]:
    q2_config = {
        "formal_evaluation": {
            "time_scan_step_s": config["formal_evaluation"]["time_scan_step_s"],
            "root_tolerance_s": config["formal_evaluation"]["root_tolerance_s"],
            "rim_samples": 2048,
        }
    }
    return tuple(
        q2_formal_evaluation(bomb, gravity, q2_config).strict_intervals
        for bomb in _bomb_strategies(strategy)
    )

```

```python

def _formal_evaluation(strategy: Q3Strategy, config: dict[str, Any]) -> Q3Evaluation:
    gravity = float(config["gravity"])
    bombs = _bomb_geometries(strategy, gravity)
    settings = config["formal_evaluation"]
    margin = _settings_margin(bombs, settings, refine=True)
    joint = _find_joint_intervals(
        margin,
        bombs,
        scan_step=float(settings["time_scan_step_s"]),
        root_tolerance=float(settings["root_tolerance_s"]),
    )
    individual = _individual_strict_intervals(strategy, gravity, config)
    conservative = _merge_intervals([interval for group in individual for interval in group])
    deletion = []
    for removed in range(3):
        remaining = tuple(bomb for index, bomb in enumerate(bombs) if index != removed)
        remaining_margin = _settings_margin(remaining, settings, refine=False)
        intervals = _find_joint_intervals(
            remaining_margin,
            remaining,
            scan_step=float(settings["time_scan_step_s"]),
            root_tolerance=float(settings["root_tolerance_s"]),
        )
        deletion.append(min(_duration(intervals), _duration(joint)))
    first_duration = _duration(individual[0])
    first_two = bombs[:2]
    first_two_intervals = _find_joint_intervals(
        _settings_margin(first_two, settings, refine=True),
        first_two,
        scan_step=float(settings["time_scan_step_s"]),
        root_tolerance=float(settings["root_tolerance_s"]),
    )
    first_two_duration = _duration(first_two_intervals)
    incremental = (
        first_duration,
        first_two_duration - first_duration,
        _duration(joint) - first_two_duration,
    )
    return Q3Evaluation(
        strategy=strategy,
        bombs=bombs,
        joint_intervals=joint,
        individual_strict_intervals=individual,
        conservative_union_intervals=conservative,
        deletion_durations_s=tuple(deletion),
        ordered_incremental_durations_s=incremental,
    )


def optimize_question_3(
    config: dict[str, Any], *, stage_log_path: Path | None = None
) -> tuple[list[Q3Evaluation], dict[str, Any]]:
    """Run seeded eight-dimensional global search and multi-start local refinement."""
    gravity = float(config["gravity"])
    global_settings = config["global_search"]
    result = differential_evolution(
        lambda values: _objective(values, gravity, global_settings, roots=False),
        _bounds(config),
        seed=int(config["seed"]),
        init=_initial_population(config),
        maxiter=int(global_settings["maxiter"]),
```

```
        popsize=int(global_settings["popsize"]),
        tol=float(global_settings["tol"]),
        polish=bool(global_settings["polish"]),
        workers=1,
    )
    print(
        "Q3 global search complete; "
        f"evaluations={result.nfev}; sampled_duration={max(0.0, -float(result.fun)):.6f}s"
    )
    order = np.argsort(result.population_energies)
    persistent_incumbent = np.asarray(config["persistent_incumbent"]["vector"], dtype=float)
    candidates = [result.x, _warm_vector(config), persistent_incumbent]
    candidates.extend(np.asarray(result.population)[order[:10]])
    if stage_log_path is not None:
        stage_log_path.write_text(
            json.dumps(
                {
                    "stage": "global-search-complete",
                    "seed": int(config["seed"]),
                    "success": bool(result.success),
                    "message": str(result.message),
                    "evaluations": int(result.nfev),
                    "sampled_duration_s": max(0.0, -float(result.fun)),
                    "ranked_candidate_vectors": [
                        np.asarray(item, dtype=float).tolist() for item in candidates
                    ],
                },
                ensure_ascii=False,
                indent=2,
            )
            + "\n",
            encoding="utf-8",
        )
    local_results = []
    local_settings = config["local_refinement"]
    for start in _deduplicate(candidates)[: int(local_settings["starts"])]:
        local = minimize(
            lambda values: _objective(values, gravity, local_settings, roots=True),
            start,
            method="Nelder-Mead",
            bounds=_bounds(config),
            options={
                "maxfev": int(local_settings["maxfev"]),
                "xatol": float(local_settings["xatol"]),
                "fatol": float(local_settings["fatol"]),
            },
        )
        local_results.append(local)
        candidates.append(local.x)
        print(
            "Q3 local refinement complete; "
            f"start={len(local_results)}; evaluations={local.nfev}; "
            f"duration={max(0.0, -float(local.fun)):.6f}s"
        )
        if stage_log_path is not None:
            stage_log_path.write_text(
                json.dumps(
                    {
                        "stage": "local-refinement-in-progress",
                        "completed_starts": len(local_results),
                        "planned_starts": int(local_settings["starts"]),
                        "candidate_vectors": [
```

```
                        np.asarray(item, dtype=float).tolist() for item in candidates
                    ],
                    "local_results": [
                        {
                            "success": bool(item.success),
                            "message": str(item.message),
                            "evaluations": int(item.nfev),
                            "duration_s": max(0.0, -float(item.fun)),
                            "vector": np.asarray(item.x, dtype=float).tolist(),
                        }
                        for item in local_results
                    ],
                },
                ensure_ascii=False,
                indent=2,
            )
            + "\n",
            encoding="utf-8",
        )
    feasible = [
        item
        for item in _deduplicate(candidates)
        if _feasibility_violation(_strategy_from_array(item), gravity) <= 1e-12
    ]
    ranked = sorted(
        feasible,
        key=lambda values: _objective(values, gravity, local_settings, roots=True),
    )
    count = min(int(config["formal_evaluation"]["candidate_count"]), len(ranked))
    formal = []
    for index, item in enumerate(ranked[:count], 1):
        formal.append(_formal_evaluation(_strategy_from_array(item), config))
        print(f"Q3 formal candidate complete; candidate={index}/{count}")
    formal.sort(key=lambda item: item.joint_duration_s, reverse=True)
    if stage_log_path is not None:
        stage_log_path.write_text(
            json.dumps(
                {
                    "stage": "search-complete",
                    "seed": int(config["seed"]),
                    "global_success": bool(result.success),
                    "global_message": str(result.message),
                    "global_evaluations": int(result.nfev),
                    "completed_local_starts": len(local_results),
                    "formal_candidate_count": len(formal),
                    "formal_candidates": [
                        _strategy_summary(item, rank) for rank, item in enumerate(formal, 1)
                    ],
                    "best_formal_joint_duration_s": (
                        formal[0].joint_duration_s if formal else None
                    ),
                    "sampled_time_grid": "absolute task time t=0",
                },
                ensure_ascii=False,
                indent=2,
            )
            + "\n",
            encoding="utf-8",
        )
    log = {
        "seed": int(config["seed"]),
        "variable_order": [
```

```
            "theta_rad",
            "speed_m_s",
            "s0_s",
            "s1_s",
            "s2_s",
            "fuse1_s",
            "fuse2_s",
            "fuse3_s",
        ],
        "global_search": {
            "success": bool(result.success),
            "message": str(result.message),
            "iterations": int(result.nit),
            "evaluations": int(result.nfev),
            "sampled_joint_duration_s": -float(result.fun),
            "settings": global_settings,
        },
        "candidate_initialization": {
            "mechanism": (
                "Q2 anchor plus structured partial-edge-complement seeds over four speeds, "
                "four heading offsets, and simultaneous/overlapping/staggered fuse families"
            ),
            "interpretation": (
                "later bombs target partial view-domain gaps; no single-cloud relay is assumed"
            ),
            "structured_seed_count": 17,
            "persistent_incumbent": config["persistent_incumbent"],
        },
        "sampled_time_grid": {
            "anchor": "absolute task time t=0",
            "included_points": "fixed-step grid plus window endpoints and cloud events",
            "invariance": (
                "inactive late clouds cannot shift the sampling phase of earlier coverage"
            ),
        },
        "corrected_full_rerun": True,
        "local_refinements": [
            {
                "success": bool(item.success),
                "message": str(item.message),
                "evaluations": int(item.nfev),
                "root_refined_duration_s": -float(item.fun) if item.fun <= 0 else None,
            }
            for item in local_results
        ],
        "formal_settings": config["formal_evaluation"],
        "status": "converged" if result.success else "budget-exhausted-with-feasible-candidates",
        "convergence_claimed": bool(result.success),
        "feasible_candidate_count": len(feasible),
    }
    return formal, log


def _strategy_summary(evaluation: Q3Evaluation, rank: int) -> dict[str, Any]:
    strategy = evaluation.strategy
    individual_total = sum(
        _duration(intervals) for intervals in evaluation.individual_strict_intervals
    )
    return {
        "rank": rank,
        "theta_rad": strategy.theta_rad,
        "theta_deg": float(np.degrees(strategy.theta_rad) % 360.0),
```

```
        "speed_m_s": strategy.speed_m_s,
        "release_times_s": json.dumps(strategy.release_times_s),
        "fuse_delays_s": json.dumps(strategy.fuse_delays_s),
        "explosion_times_s": json.dumps(strategy.explosion_times_s),
        "joint_duration_s": evaluation.joint_duration_s,
        "conservative_duration_s": evaluation.conservative_duration_s,
        "joint_synergy_s": evaluation.joint_duration_s - evaluation.conservative_duration_s,
        "sum_individual_strict_duration_s": individual_total,
        "individual_strict_overlap_s": individual_total - evaluation.conservative_duration_s,
    }


def _bomb_rows(evaluation: Q3Evaluation) -> list[dict[str, Any]]:
    rows = []
    individual_durations = [_duration(group) for group in evaluation.individual_strict_intervals]
    for bomb, individual, deleted, incremental in zip(
        evaluation.bombs,
        individual_durations,
        evaluation.deletion_durations_s,
        evaluation.ordered_incremental_durations_s,
        strict=True,
    ):
        rows.append(
            {
                "schema_status": "provisional-schema; official result1.xlsx unavailable",
                "drone": "FY1",
                "bomb": bomb.bomb,
                "theta_rad": evaluation.strategy.theta_rad,
                "theta_deg": float(np.degrees(evaluation.strategy.theta_rad) % 360.0),
                "speed_m_s": evaluation.strategy.speed_m_s,
                "release_time_s": bomb.release_time_s,
                "release_x_m": bomb.release_point[0],
                "release_y_m": bomb.release_point[1],
                "release_z_m": bomb.release_point[2],
                "fuse_delay_s": bomb.fuse_delay_s,
                "explosion_time_s": bomb.explosion_time_s,
                "explosion_x_m": bomb.explosion_point[0],
                "explosion_y_m": bomb.explosion_point[1],
                "explosion_z_m": bomb.explosion_point[2],
                "individual_strict_duration_s": individual,
                "joint_deletion_loss_s": evaluation.joint_duration_s - deleted,
                "ordered_incremental_contribution_s": incremental,
                "minimum_cloud_center_height_m": bomb.explosion_point[2]
                - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME,
            }
        )
    return rows


def _independent_ray_intersections(
    points: np.ndarray, missile: np.ndarray, clouds: Sequence[np.ndarray]
) -> np.ndarray:
    """Independently test segment-sphere intersections through quadratic roots."""
    directions = points - missile
    lengths = np.linalg.norm(directions, axis=1)
    unit_directions = directions / lengths[:, None]
    covered = np.zeros(len(points), dtype=bool)
    for cloud in clouds:
        offset = missile - cloud
        b = 2.0 * (unit_directions @ offset)
        c = float(offset @ offset) - CLOUD_RADIUS**2
        discriminant = b**2 - 4.0 * c
```

```
        real = discriminant >= 0.0
        root = np.sqrt(np.maximum(discriminant, 0.0))
        lower = (-b - root) / 2.0
        upper = (-b + root) / 2.0
        covered |= real & (upper >= 0.0) & (lower <= lengths)
    return covered


def _validation_record(evaluation: Q3Evaluation, config: dict[str, Any]) -> dict[str, Any]:
    gravity = float(config["gravity"])
    validation = config["validation"]
    convergence = []
    convergence_settings = validation["convergence_settings"]
    for index, settings in enumerate(convergence_settings):
        use_refinement = index == len(convergence_settings) - 1
        merged = {
            **settings,
            "continuous_refinement_starts": 1,
            "continuous_refinement_maxfev": 50,
            "adaptive_max_surface_angles": (
                int(config["formal_evaluation"]["adaptive_max_surface_angles"])
                if use_refinement
                else int(settings["surface_angles"])
            ),
            "adaptive_max_surface_levels": (
                int(config["formal_evaluation"]["adaptive_max_surface_levels"])
                if use_refinement
                else int(settings["surface_levels"])
            ),
        }
        margin = _settings_margin(evaluation.bombs, merged, refine=use_refinement)
        intervals = _find_joint_intervals(
            margin,
            evaluation.bombs,
            scan_step=float(settings["time_scan_step_s"]),
            root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
        )
        convergence.append(
            {
                **settings,
                "continuous_surface_refinement": use_refinement,
                "joint_duration_s": _duration(intervals),
            }
        )

    boundary = []
    direct = []
    probe = float(validation["boundary_probe_s"])
    points = _surface_points(
        int(validation["direct_surface_angles"]), int(validation["direct_surface_levels"])
    )
    for interval in evaluation.joint_intervals:
        for location, time in (
            ("before-start", interval.start - probe),
            ("start", interval.start),
            ("midpoint", 0.5 * (interval.start + interval.end)),
            ("end", interval.end),
            ("after-end", interval.end + probe),
        ):
            assessment = joint_coverage_assessment(
                time,
                evaluation.bombs,
```

```
                surface_angles=int(config["formal_evaluation"]["surface_angles"]),
                surface_levels=int(config["formal_evaluation"]["surface_levels"]),
                continuous_refinement=True,
                refinement_starts=int(config["formal_evaluation"]["continuous_refinement_starts"]),
                refinement_maxfev=int(config["formal_evaluation"]["continuous_refinement_maxfev"]),
                adaptive_max_surface_angles=int(
                    config["formal_evaluation"]["adaptive_max_surface_angles"]
                ),
                adaptive_max_surface_levels=int(
                    config["formal_evaluation"]["adaptive_max_surface_levels"]
                ),
            )
            active = _active_bombs(time, evaluation.bombs)
            covered = _independent_ray_intersections(
                points,
                missile_position(time),
                [_cloud_center(time, bomb) for bomb in active],
            )
            boundary.append(
                {
                    "location": location,
                    "time_s": time,
                    "coverage_status": assessment.status,
                    "worst_margin_m": assessment.signed_margin_m,
                    "sampled_lower_bound_m": assessment.sampled_lower_bound_m,
                    "lipschitz_upper_bound_m": assessment.lipschitz_upper_bound_m,
                    "surface_angles": assessment.surface_angles,
                    "surface_levels": assessment.surface_levels,
                    "adaptive_trace": assessment.refinement_trace,
                }
            )
            direct.append(
                {
                    "location": location,
                    "time_s": time,
                    "sampled_rays": len(points),
                    "uncovered_rays": int(np.count_nonzero(~covered)),
                    "all_covered": bool(np.all(covered)),
                }
            )

    q2 = config["q2_warm_start"]
    q2_strategy = Q2Strategy(q2["theta_rad"], q2["speed_m_s"], 0.0, q2["fuse_delays_s"][0])
    q2_config = {
        "formal_evaluation": {
            "time_scan_step_s": config["formal_evaluation"]["time_scan_step_s"],
            "root_tolerance_s": config["formal_evaluation"]["root_tolerance_s"],
            "rim_samples": 2048,
        }
    }

    q2_reference = q2_formal_evaluation(q2_strategy, gravity, q2_config)
    single_bomb = _bomb_geometries(
        Q3Strategy(
            q2_strategy.theta_rad, q2_strategy.speed_m_s, 0, 0, 0, q2_strategy.fuse_delay_s, 19, 19
        ),
        gravity,
    )[0]
    single_settings = config["formal_evaluation"]
    single_intervals = _find_joint_intervals(
        _settings_margin((single_bomb,), single_settings, refine=True),
        (single_bomb,),
```

```
        scan_step=float(single_settings["time_scan_step_s"]),
        root_tolerance=float(single_settings["root_tolerance_s"]),
    )
    permutation_durations = []
    permutation_settings = {
        **validation["convergence_settings"][1],
        "continuous_refinement_starts": 0,
        "continuous_refinement_maxfev": 0,
    }
    for order in permutations(evaluation.bombs):
        intervals = _find_joint_intervals(
            _settings_margin(order, permutation_settings, refine=False),
            order,
            scan_step=float(permutation_settings["time_scan_step_s"]),
            root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
        )
        permutation_durations.append(_duration(intervals))

    complement_time = 4.768
    reference_cloud = np.array([17625.525268056128, 10.240444866222791, 1762.640057120904])
    complement_bombs = tuple(
        BombGeometry(
            bomb=index,
            release_time_s=complement_time,
            fuse_delay_s=0.0,
            explosion_time_s=complement_time,
            release_point=tuple(reference_cloud + np.array([0.0, sign * 9.65, 0.0])),
            explosion_point=tuple(reference_cloud + np.array([0.0, sign * 9.65, 0.0])),
        )
        for index, sign in enumerate((-1.0, 1.0), 1)
    )
    complement_settings = {
        "surface_angles": 360,
        "surface_levels": 41,
        "continuous_refinement_starts": 8,
        "continuous_refinement_maxfev": 100,
    }
    complement_individual = [
        joint_coverage_margin(
            complement_time,
            (bomb,),
            surface_angles=360,
            surface_levels=41,
            continuous_refinement=True,
            refinement_starts=8,
            refinement_maxfev=100,
        )
        for bomb in complement_bombs
    ]
    complement_joint = _settings_margin(complement_bombs, complement_settings, refine=True)(
        complement_time
    )
    return {
        "feasibility": {
            "violation": _feasibility_violation(evaluation.strategy, gravity),
            "release_gaps_s": np.diff(evaluation.strategy.release_times_s).tolist(),
            "explosion_times_s": list(evaluation.strategy.explosion_times_s),
            "explosion_heights_m": [bomb.explosion_point[2] for bomb in evaluation.bombs],
            "cloud_minimum_heights_m": [
                bomb.explosion_point[2] - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME
                for bomb in evaluation.bombs
            ],
```

```
        },
        "surface_time_convergence": convergence,
        "boundary_checks": boundary,
        "independent_quadratic_ray_checks": direct,
        "label_permutation_durations_s": permutation_durations,
        "complementarity_construct": {
            "time_s": complement_time,
            "cloud_y_offsets_m": [-9.65, 9.65],
            "individual_worst_margins_m": complement_individual,
            "joint_worst_margin_m": complement_joint,
            "each_individual_incomplete": all(value > 0.0 for value in complement_individual),
            "joint_complete": complement_joint <= 0.0,
        },
        "single_cloud_q2_regression": {
            "q3_joint_duration_s": _duration(single_intervals),
            "q2_strict_duration_s": q2_reference.strict_duration_s,
            "absolute_difference_s": abs(
                _duration(single_intervals) - q2_reference.strict_duration_s
            ),
            "tolerance_s": float(validation["single_cloud_tolerance_s"]),
        },
        "joint_vs_conservative": {
            "joint_duration_s": evaluation.joint_duration_s,
            "conservative_duration_s": evaluation.conservative_duration_s,
            "joint_not_less": evaluation.joint_duration_s + 1e-9
            >= evaluation.conservative_duration_s,
        },
        "deletion": [
            {
                "bomb": index,
                "remaining_duration_s": duration,
                "marginal_loss_s": evaluation.joint_duration_s - duration,
            }
            for index, duration in enumerate(evaluation.deletion_durations_s, 1)
        ],
    }


def _conditional_third_bomb_search(
    baseline: Q3Evaluation, config: dict[str, Any]
) -> tuple[dict[str, Any], Q3Evaluation | None]:
    """Optimize only the third bomb conditional on the first two formal controls."""
    settings = config["conditional_third_bomb_search"]
    gravity = float(config["gravity"])
    baseline_vector = np.array(list(asdict(baseline.strategy).values()), dtype=float)

    def full_vector(values: np.ndarray) -> np.ndarray:
        candidate = baseline_vector.copy()
        candidate[4] = values[0]
        candidate[7] = values[1]
        return candidate

    search_settings = {
        "time_step_s": settings["time_step_s"],
        "surface_angles": settings["surface_angles"],
        "surface_levels": settings["surface_levels"],
    }
    result = differential_evolution(
        lambda values: _objective(full_vector(values), gravity, search_settings, roots=False),
        [tuple(config["bounds"]["s2_s"]), tuple(config["bounds"]["fuse_delay_s"])],
        seed=int(settings["seed"]),
        maxiter=int(settings["maxiter"]),
```

```
        popsize=int(settings["popsize"]),
        tol=float(settings["tol"]),
        polish=False,
        workers=1,
    )
    population = np.asarray(result.population)
    order = np.argsort(result.population_energies)
    candidate_values = [result.x]
    candidate_values.extend(population[order[: int(settings["root_candidate_count"])]])
    candidate_values.append(np.array([baseline.strategy.s2_s, baseline.strategy.fuse3_s]))
    root_settings = {
        "time_step_s": config["formal_evaluation"]["time_scan_step_s"],
        "surface_angles": config["formal_evaluation"]["surface_angles"],
        "surface_levels": config["formal_evaluation"]["surface_levels"],
    }
    root_ranked = []
    for values in _deduplicate(candidate_values):
        objective = _objective(full_vector(values), gravity, root_settings, roots=True)
        root_ranked.append(
            {
                "s2_s": float(values[0]),
                "fuse3_s": float(values[1]),
                "root_refined_duration_s": max(0.0, -objective),
                "vector": full_vector(values),
            }
        )
    root_ranked.sort(key=lambda item: item["root_refined_duration_s"], reverse=True)
    formal_candidate = _formal_evaluation(
        _strategy_from_array(np.asarray(root_ranked[0]["vector"])), config
    )
    improvement = formal_candidate.joint_duration_s - baseline.joint_duration_s
    threshold = float(settings["improvement_threshold_s"])
    record = {
        "scope": "theta, speed, and first two bombs fixed at the formal baseline",
        "variable_order": ["s2_s", "fuse3_s"],
        "seed": int(settings["seed"]),
        "search_success": bool(result.success),
        "search_message": str(result.message),
        "search_evaluations": int(result.nfev),
        "settings": settings,
        "root_refined_candidates": [
            {key: value for key, value in item.items() if key != "vector"} for item in root_ranked
        ],
        "formal_candidate": _strategy_summary(formal_candidate, 1),
        "baseline_joint_duration_s": baseline.joint_duration_s,
        "formal_improvement_s": improvement,
        "improvement_threshold_s": threshold,
        "material_improvement_found": improvement > threshold,
        "conclusion": (
            "conditional search found a material third-bomb contribution"
            if improvement > threshold
            else "conditional on the first two bombs, no material third-bomb contribution was found"
        ),
    }
    return record, formal_candidate if improvement > threshold else None


def run_question_3(project_root: Path, config: dict[str, Any]) -> list[Path]:
    """Optimize Question 3 and write tables, provisional workbook, and audit logs."""
    paths = prepare_output_paths(project_root / "outputs")
    stage_log_path = paths.logs / "q3_search_stage.json"
    evaluations, solver = optimize_question_3(config, stage_log_path=stage_log_path)
```

```
    if not evaluations:
        raise RuntimeError("Question 3 optimization produced no feasible formal candidate")
    best = evaluations[0]
    conditional_record, conditional_improvement = _conditional_third_bomb_search(best, config)
    if conditional_improvement is not None:
        evaluations = [conditional_improvement, *evaluations]
        evaluations.sort(key=lambda item: item.joint_duration_s, reverse=True)
        best = evaluations[0]
    near_count = min(int(config["near_optimal_count"]), len(evaluations))
    summaries = [
        _strategy_summary(item, rank) for rank, item in enumerate(evaluations[:near_count], 1)
    ]
    summary_path = save_table(
        pd.DataFrame(summaries[:1]), paths.tables / "q3_summary.csv", overwrite=True
    )
    candidates_path = save_table(
        pd.DataFrame(summaries), paths.tables / "q3_candidates.csv", overwrite=True
    )
    bombs = pd.DataFrame(_bomb_rows(best))
    bombs_path = save_table(bombs, paths.tables / "q3_bombs.csv", overwrite=True)
    interval_rows = []
    for criterion, groups in (
        ("joint-union-cover", (best.joint_intervals,)),
        ("conservative-single-cloud-union", (best.conservative_union_intervals,)),
        ("bomb-1-strict", (best.individual_strict_intervals[0],)),
        ("bomb-2-strict", (best.individual_strict_intervals[1],)),
        ("bomb-3-strict", (best.individual_strict_intervals[2],)),
    ):
        for group in groups:
            for number, interval in enumerate(group, 1):
                interval_rows.append(
                    {
                        "criterion": criterion,
                        "interval": number,
                        "start_s": interval.start,
                        "end_s": interval.end,
                        "duration_s": interval.duration,
                    }
                )
    intervals_path = save_table(
        pd.DataFrame(interval_rows), paths.tables / "q3_intervals.csv", overwrite=True
    )
    workbook_path = paths.tables / "result1.xlsx"
    with pd.ExcelWriter(workbook_path, engine="openpyxl") as writer:
        bombs.to_excel(writer, sheet_name="provisional_result1", index=False)
        pd.DataFrame(
            [
                {
                    "schema_status": "provisional",
                    "reason": "official result1.xlsx template is missing",
                    "mapping_action": (
                        "map these explicit fields when the official template is supplied"
                    ),
                }
            ]
        ).to_excel(writer, sheet_name="schema_note", index=False)

    optimization_path = paths.logs / "q3_optimization.json"
    optimization_path.write_text(
        json.dumps(
            {
                "model": "q3-joint-ray-cover-v1",
```

```
                "quantifiers": (
                    "for every target ray, at least one active cloud intersects before target"
                ),
                "best": {**_strategy_summary(best, 1), "bombs": _bomb_rows(best)},
                "near_optimal_candidates": summaries,
                "solver": solver,
            },
            ensure_ascii=False,
            indent=2,
        )
        + "\n",
        encoding="utf-8",
    )
    validation_path = paths.logs / "q3_validation.json"
    validation_path.write_text(
        json.dumps(_validation_record(best, config), ensure_ascii=False, indent=2) + "\n",
        encoding="utf-8",
    )
    conditional_path = paths.logs / "q3_third_bomb_incremental.json"
    conditional_path.write_text(
        json.dumps(conditional_record, ensure_ascii=False, indent=2) + "\n",
        encoding="utf-8",
    )
    return [
        summary_path,
        candidates_path,
        bombs_path,
        intervals_path,
        workbook_path,
        stage_log_path,
        optimization_path,
        validation_path,
        conditional_path,
    ]
```

B.5 问题四程序

```
"""Question 4 optimization for one smoke bomb from each of FY1, FY2, and FY3."""

from __future__ import annotations

import json
from collections.abc import Sequence
from dataclasses import asdict, dataclass
from itertools import permutations
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd
from scipy.optimize import differential_evolution, minimize

from mmkit.artifacts import prepare_output_paths, save_table

from .q1 import CLOUD_DESCENT_SPEED, CLOUD_LIFETIME, OcclusionInterval, missile_position
from .q2 import missile_hit_time
from .q3 import (
    BombGeometry,
    Q3Strategy,
    _active_bombs,
    _cloud_center,
```

```
    _duration,
    _find_joint_intervals,
    _independent_ray_intersections,
    _merge_intervals,
    _sampled_metrics,
    _settings_margin,
    _surface_points,
    joint_coverage_assessment,
    joint_coverage_margin,
)
from .q3 import (
    _bomb_geometries as q3_bomb_geometries,
)

UAV_NAMES = ("FY1", "FY2", "FY3")


@dataclass(frozen=True, slots=True)
class UAVControl:
    """Flight and bomb controls for one UAV."""

    theta_rad: float
    speed_m_s: float
    release_time_s: float
    fuse_delay_s: float

    @property
    def explosion_time_s(self) -> float:
        return self.release_time_s + self.fuse_delay_s


@dataclass(frozen=True, slots=True)
class Q4Strategy:
    """Independent four-variable controls for FY1, FY2, and FY3."""

    controls: tuple[UAVControl, UAVControl, UAVControl]


@dataclass(frozen=True, slots=True)
class Q4Evaluation:
    """Formal joint evaluation for one Question 4 strategy."""

    strategy: Q4Strategy
    bombs: tuple[BombGeometry, BombGeometry, BombGeometry]
    joint_intervals: tuple[OcclusionInterval, ...]
    individual_intervals: tuple[tuple[OcclusionInterval, ...], ...]
    conservative_intervals: tuple[OcclusionInterval, ...]
    deletion_durations_s: tuple[float, float, float]

    @property
    def joint_duration_s(self) -> float:
        return _duration(self.joint_intervals)

    @property
    def conservative_duration_s(self) -> float:
        return _duration(self.conservative_intervals)


def _strategy_from_array(values: np.ndarray) -> Q4Strategy:
    array = np.asarray(values, dtype=float).reshape(3, 4)
    return Q4Strategy(tuple(UAVControl(*(float(value) for value in row)) for row in array))

```

```
87
88  def _strategy_vector(strategy: Q4Strategy) -> np.ndarray:
89      return np.array(
90          [value for control in strategy.controls for value in asdict(control).values()], dtype=float
91      )
92
93
94  def _uav_initials(config: dict[str, Any]) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
95      return tuple(np.asarray(config["uavs"][name], dtype=float) for name in UAV_NAMES)
96
97
98  def _bomb_geometries(
99      strategy: Q4Strategy, gravity: float, config: dict[str, Any]
100 ) -> tuple[BombGeometry, BombGeometry, BombGeometry]:
101     bombs = []
102     for index, (control, initial) in enumerate(
103         zip(strategy.controls, _uav_initials(config), strict=True), 1
104     ):
105         heading = np.array([np.cos(control.theta_rad), np.sin(control.theta_rad), 0.0])
106         release = initial + control.speed_m_s * control.release_time_s * heading
107         explosion = initial + control.speed_m_s * control.explosion_time_s * heading
108         explosion = explosion + np.array([0.0, 0.0, -0.5 * gravity * control.fuse_delay_s**2])
109         bombs.append(
110             BombGeometry(
111                 bomb=index,
112                 release_time_s=control.release_time_s,
113                 fuse_delay_s=control.fuse_delay_s,
114                 explosion_time_s=control.explosion_time_s,
115                 release_point=tuple(float(value) for value in release),
116                 explosion_point=tuple(float(value) for value in explosion),
117             )
118         )
119     return tuple(bombs)
120
121
122 def _fuse_maxima(config: dict[str, Any]) -> tuple[float, float, float]:
123     gravity = float(config["gravity"])
124     return tuple(float(np.sqrt(2.0 * initial[2] / gravity)) for initial in _uav_initials(config))
125
126
127 def _bounds(config: dict[str, Any]) -> list[tuple[float, float]]:
128     common = config["bounds"]
129     bounds = []
130     for fuse_maximum in _fuse_maxima(config):
131         bounds.extend(
132             (
133                 tuple(common["theta_rad"]),
134                 tuple(common["speed_m_s"]),
135                 tuple(common["release_time_s"]),
136                 (0.0, fuse_maximum),
137             )
138         )
139     return bounds
140
141
142 def _feasibility_violation(strategy: Q4Strategy, gravity: float, config: dict[str, Any]) -> float:
143     violation = 0.0
144     hit_time = missile_hit_time()
145     for control, bomb, fuse_maximum, initial in zip(
146         strategy.controls,
147         _bomb_geometries(strategy, gravity, config),
148         _fuse_maxima(config),
```

```
        _uav_initials(config),
        strict=True,
    ):
        violation += max(0.0, 70.0 - control.speed_m_s) / 70.0
        violation += max(0.0, control.speed_m_s - 140.0) / 70.0
        violation += max(0.0, -control.release_time_s) / hit_time
        violation += max(0.0, -control.fuse_delay_s) / fuse_maximum
        violation += max(0.0, control.fuse_delay_s - fuse_maximum) / fuse_maximum
        violation += max(0.0, control.explosion_time_s - hit_time) / hit_time
        violation += max(0.0, -bomb.explosion_point[2]) / float(initial[2])
    return violation


def _objective(
    values: np.ndarray,
    gravity: float,
    config: dict[str, Any],
    settings: dict[str, Any],
    *,
    roots: bool,
) -> float:
    strategy = _strategy_from_array(values)
    violation = _feasibility_violation(strategy, gravity, config)
    if violation > 0.0:
        return 100.0 + 100.0 * violation
    bombs = _bomb_geometries(strategy, gravity, config)
    margin = _settings_margin(bombs, settings, refine=False)
    duration, minimum = _sampled_metrics(bombs, margin, float(settings["time_step_s"]))
    if duration <= 0.0:
        return max(minimum, 0.0) / 100.0
    if not roots:
        return -duration
    intervals = _find_joint_intervals(
        margin,
        bombs,
        scan_step=float(settings["time_step_s"]),
        root_tolerance=1e-8,
    )
    return -_duration(intervals)


def _single_initial_population(
    initial: np.ndarray, fuse_maximum: float, config: dict[str, Any], seed: int
) -> np.ndarray:
    settings = config["single_search"]
    count = 4 * int(settings["popsize"])
    rng = np.random.default_rng(seed)
    lower = np.array([0.0, 70.0, 0.0, 0.0])
    upper = np.array([2.0 * np.pi, 140.0, 30.0, fuse_maximum])
    population = rng.uniform(lower, upper, size=(count, 4))
    toward_origin = float(np.arctan2(-initial[1], -initial[0]) % (2.0 * np.pi))
    structured = []
    for speed in (70.0, 100.0, 140.0):
        for release, fraction in ((0.0, 0.15), (0.0, 0.35), (2.0, 0.25)):
            structured.append(np.array([toward_origin, speed, release, fraction * fuse_maximum]))
    for index, candidate in enumerate(structured):
        population[index] = candidate
    return population


def _single_searches(config: dict[str, Any]) -> tuple[list[np.ndarray], list[dict[str, Any]]]:
    gravity = float(config["gravity"])
```

```
    settings = config["single_search"]
    best_vectors = []
    records = []
    for index, (name, initial, fuse_maximum) in enumerate(
        zip(UAV_NAMES, _uav_initials(config), _fuse_maxima(config), strict=True)
    ):

        def objective(values: np.ndarray, fixed_index: int = index) -> float:
            controls = [UAVControl(0.0, 70.0, 0.0, 0.0) for _ in UAV_NAMES]
            controls[fixed_index] = UAVControl(*(float(value) for value in values))
            strategy = Q4Strategy(tuple(controls))
            bomb = (_bomb_geometries(strategy, gravity, config)[fixed_index],)
            control = strategy.controls[fixed_index]
            if control.explosion_time_s > missile_hit_time():
                return 100.0 + control.explosion_time_s - missile_hit_time()
            margin = _settings_margin(bomb, settings, refine=False)
            duration, minimum = _sampled_metrics(bomb, margin, float(settings["time_step_s"]))
            return -duration if duration > 0.0 else max(minimum, 0.0) / 100.0

        result = differential_evolution(
            objective,
            [
                tuple(config["bounds"]["theta_rad"]),
                tuple(config["bounds"]["speed_m_s"]),
                tuple(config["bounds"]["release_time_s"]),
                (0.0, fuse_maximum),
            ],
            seed=int(config["seed"]) + index,
            init=_single_initial_population(
                initial, fuse_maximum, config, int(config["seed"]) + index
            ),
            maxiter=int(settings["maxiter"]),
            popsize=int(settings["popsize"]),
            tol=float(settings["tol"]),
            polish=False,
            workers=1,
        )
        best_vectors.append(np.asarray(result.x, dtype=float))
        records.append(
            {
                "uav": name,
                "success": bool(result.success),
                "message": str(result.message),
                "evaluations": int(result.nfev),
                "sampled_duration_s": max(0.0, -float(result.fun)),
                "vector": np.asarray(result.x, dtype=float).tolist(),
            }
        )
    return best_vectors, records


def _joint_initial_population(
    single_vectors: Sequence[np.ndarray], config: dict[str, Any]
) -> np.ndarray:
    bounds = _bounds(config)
    count = len(bounds) * int(config["joint_search"]["popsize"])
    rng = np.random.default_rng(int(config["seed"]) + 100)
    lower = np.array([bound[0] for bound in bounds])
    upper = np.array([bound[1] for bound in bounds])
    population = rng.uniform(lower, upper, size=(count, len(bounds)))
    base = np.concatenate(single_vectors)
    population[0] = base
```

```
    variants = (
        np.zeros(12),
        np.array([0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 4, 0], dtype=float),
        np.array([0, 0, 0, 0, 0.03, 10, 0, 0, -0.03, -5, 0, 0], dtype=float),
        np.array([0, 0, 0, 0.3, 0, 0, 0, -0.3, 0, 0, 0, 0.2], dtype=float),
    )
    for index, offset in enumerate(variants, 1):
        population[index] = np.clip(base + offset, lower, upper)
    for index in range(5, min(16, count)):
        population[index] = np.clip(base + rng.normal(0.0, 0.2, 12), lower, upper)
    return population


def _deduplicate(candidates: Sequence[np.ndarray]) -> list[np.ndarray]:
    unique = []
    for candidate in candidates:
        if not any(np.linalg.norm(candidate - existing) < 1e-7 for existing in unique):
            unique.append(np.asarray(candidate, dtype=float))
    return unique


def _formal_evaluation(strategy: Q4Strategy, config: dict[str, Any]) -> Q4Evaluation:
    gravity = float(config["gravity"])
    bombs = _bomb_geometries(strategy, gravity, config)
    settings = config["formal_evaluation"]
    joint = _find_joint_intervals(
        _settings_margin(bombs, settings, refine=True),
        bombs,
        scan_step=float(settings["time_scan_step_s"]),
        root_tolerance=float(settings["root_tolerance_s"]),
    )
    individual = []
    for bomb in bombs:
        intervals = _find_joint_intervals(
            _settings_margin((bomb,), settings, refine=True),
            (bomb,),
            scan_step=float(settings["time_scan_step_s"]),
            root_tolerance=float(settings["root_tolerance_s"]),
        )
        individual.append(intervals)
    conservative = _merge_intervals([interval for group in individual for interval in group])
    deletion = []
    for removed in range(3):
        remaining = tuple(bomb for index, bomb in enumerate(bombs) if index != removed)
        intervals = _find_joint_intervals(
            _settings_margin(remaining, settings, refine=False),
            remaining,
            scan_step=float(settings["time_scan_step_s"]),
            root_tolerance=float(settings["root_tolerance_s"]),
        )
        deletion.append(min(_duration(intervals), _duration(joint)))
    return Q4Evaluation(
        strategy=strategy,
        bombs=bombs,
        joint_intervals=joint,
        individual_intervals=tuple(individual),
        conservative_intervals=conservative,
        deletion_durations_s=tuple(deletion),
    )


def optimize_question_4(
```

```
    config: dict[str, Any], stage_log_path: Path
) -> tuple[list[Q4Evaluation], dict[str, Any]]:
    """Run single-UAV searches, joint global search, local refinement, and formal ranking."""
    gravity = float(config["gravity"])
    single_vectors, single_records = _single_searches(config)
    stage_log_path.write_text(
        json.dumps(
            {"stage": "single-search-complete", "single_candidates": single_records},
            ensure_ascii=False,
            indent=2,
        )
        + "\n",
        encoding="utf-8",
    )
    print("Q4 single-UAV searches complete", flush=True)
    settings = config["joint_search"]
    result = differential_evolution(
        lambda values: _objective(values, gravity, config, settings, roots=False),
        _bounds(config),
        seed=int(config["seed"]) + 10,
        init=_joint_initial_population(single_vectors, config),
        maxiter=int(settings["maxiter"]),
        popsize=int(settings["popsize"]),
        tol=float(settings["tol"]),
        polish=False,
        workers=1,
    )
    order = np.argsort(result.population_energies)
    candidates = [result.x, np.concatenate(single_vectors)]
    candidates.extend(np.asarray(result.population)[order[:10]])
    stage_log_path.write_text(
        json.dumps(
            {
                "stage": "joint-global-complete",
                "single_candidates": single_records,
                "success": bool(result.success),
                "message": str(result.message),
                "evaluations": int(result.nfev),
                "sampled_duration_s": max(0.0, -float(result.fun)),
                "candidate_vectors": [np.asarray(item).tolist() for item in candidates],
                "sampled_time_grid": "absolute task time t=0",
            },
            ensure_ascii=False,
            indent=2,
        )
        + "\n",
        encoding="utf-8",
    )
    print("Q4 joint global search complete", flush=True)
    local_results = []
    local_settings = config["local_refinement"]
    for start in _deduplicate(candidates)[: int(local_settings["starts"])]:
        local = minimize(
            lambda values: _objective(values, gravity, config, local_settings, roots=True),
            start,
            method="Nelder-Mead",
            bounds=_bounds(config),
            options={
                "maxfev": int(local_settings["maxfev"]),
                "xatol": float(local_settings["xatol"]),
                "fatol": float(local_settings["fatol"]),
            },
```

```
        )
        local_results.append(local)
        candidates.append(local.x)
        stage_log_path.write_text(
            json.dumps(
                {
                    "stage": "local-refinement-in-progress",
                    "completed_starts": len(local_results),
                    "planned_starts": int(local_settings["starts"]),
                    "local_results": [
                        {
                            "success": bool(item.success),
                            "message": str(item.message),
                            "evaluations": int(item.nfev),
                            "duration_s": max(0.0, -float(item.fun)),
                            "vector": np.asarray(item.x).tolist(),
                        }
                        for item in local_results
                    ],
                },
                ensure_ascii=False,
                indent=2,
            )
            + "\n",
            encoding="utf-8",
        )
        print(f"Q4 local refinement {len(local_results)} complete", flush=True)
    feasible = [
        item
        for item in _deduplicate(candidates)
        if _feasibility_violation(_strategy_from_array(item), gravity, config) <= 1e-12
    ]
    ranked = sorted(
        feasible,
        key=lambda values: _objective(values, gravity, config, local_settings, roots=True),
    )
    count = min(int(config["formal_evaluation"]["candidate_count"]), len(ranked))
    incumbent = np.concatenate(single_vectors)
    formal_vectors = _deduplicate([*ranked[: max(0, count - 1)], incumbent])[:count]
    formal = []
    for index, candidate in enumerate(formal_vectors, 1):
        formal.append(_formal_evaluation(_strategy_from_array(candidate), config))
        print(f"Q4 formal candidate {index}/{count} complete", flush=True)
    formal.sort(key=lambda item: item.joint_duration_s, reverse=True)
    stage_log_path.write_text(
        json.dumps(
            {
                "stage": "search-complete",
                "single_candidates": single_records,
                "global_success": bool(result.success),
                "global_message": str(result.message),
                "global_evaluations": int(result.nfev),
                "completed_local_starts": len(local_results),
                "formal_candidate_count": len(formal),
                "formal_durations_s": [item.joint_duration_s for item in formal],
                "single_concat_incumbent_retained": True,
                "sampled_time_grid": "absolute task time t=0",
            },
            ensure_ascii=False,
            indent=2,
        )
        + "\n",
```

```
        encoding="utf-8",
    )
    solver = {
        "seed": int(config["seed"]),
        "single_searches": single_records,
        "joint_global": {
            "success": bool(result.success),
            "message": str(result.message),
            "evaluations": int(result.nfev),
            "sampled_duration_s": max(0.0, -float(result.fun)),
            "settings": settings,
        },
        "local_refinements": [
            {
                "success": bool(item.success),
                "message": str(item.message),
                "evaluations": int(item.nfev),
                "duration_s": max(0.0, -float(item.fun)),
            }
            for item in local_results
        ],
        "candidate_initialization": (
            "per-UAV single candidates plus relay, overlap, heading/speed, "
            "and edge-complement variants"
        ),
        "sampled_time_grid": "absolute task time t=0 plus cloud events",
        "status": "converged" if result.success else "budget-exhausted-with-feasible-candidates",
        "convergence_claimed": bool(result.success),
        "formal_settings": config["formal_evaluation"],
    }
    return formal, solver


def _summary_row(evaluation: Q4Evaluation, rank: int) -> dict[str, Any]:
    return {
        "rank": rank,
        "joint_duration_s": evaluation.joint_duration_s,
        "conservative_duration_s": evaluation.conservative_duration_s,
        "joint_synergy_s": evaluation.joint_duration_s - evaluation.conservative_duration_s,
        "individual_durations_s": json.dumps(
            [_duration(intervals) for intervals in evaluation.individual_intervals]
        ),
    }


def _uav_rows(evaluation: Q4Evaluation) -> list[dict[str, Any]]:
    rows = []
    for name, control, bomb, individual, deletion in zip(
        UAV_NAMES,
        evaluation.strategy.controls,
        evaluation.bombs,
        evaluation.individual_intervals,
        evaluation.deletion_durations_s,
        strict=True,
    ):
        rows.append(
            {
                "schema_status": "provisional-schema; official result2.xlsx unavailable",
                "uav": name,
                "theta_rad": control.theta_rad,
                "theta_deg": float(np.degrees(control.theta_rad) % 360.0),
                "speed_m_s": control.speed_m_s,
```

```
                "release_time_s": control.release_time_s,
                "release_x_m": bomb.release_point[0],
                "release_y_m": bomb.release_point[1],
                "release_z_m": bomb.release_point[2],
                "fuse_delay_s": control.fuse_delay_s,
                "explosion_time_s": control.explosion_time_s,
                "explosion_x_m": bomb.explosion_point[0],
                "explosion_y_m": bomb.explosion_point[1],
                "explosion_z_m": bomb.explosion_point[2],
                "individual_strict_duration_s": _duration(individual),
                "joint_deletion_loss_s": evaluation.joint_duration_s - deletion,
                "minimum_cloud_center_height_m": bomb.explosion_point[2]
                - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME,
            }
        )
    return rows


def _validation_record(evaluation: Q4Evaluation, config: dict[str, Any]) -> dict[str, Any]:
    validation = config["validation"]
    hit_time = missile_hit_time()
    convergence = []
    convergence_settings = validation["convergence_settings"]
    for index, settings in enumerate(convergence_settings):
        refine = index == len(convergence_settings) - 1
        merged = {
            **settings,
            "continuous_refinement_starts": 1,
            "continuous_refinement_maxfev": 50,
            "adaptive_max_surface_angles": (
                config["formal_evaluation"]["adaptive_max_surface_angles"]
                if refine
                else settings["surface_angles"]
            ),
            "adaptive_max_surface_levels": (
                config["formal_evaluation"]["adaptive_max_surface_levels"]
                if refine
                else settings["surface_levels"]
            ),
        }
        intervals = _find_joint_intervals(
            _settings_margin(evaluation.bombs, merged, refine=refine),
            evaluation.bombs,
            scan_step=float(settings["time_scan_step_s"]),
            root_tolerance=float(config["formal_evaluation"]["root_tolerance_s"]),
        )
        convergence.append(
            {**settings, "continuous_refinement": refine, "joint_duration_s": _duration(intervals)}
        )
    boundary = []
    direct = []
    points = _surface_points(
        int(validation["direct_surface_angles"]), int(validation["direct_surface_levels"])
    )
    probe = float(validation["boundary_probe_s"])
    formal = config["formal_evaluation"]
    for interval in evaluation.joint_intervals:
        for location, time in (
            ("before-start", interval.start - probe),
            ("start", interval.start),
            ("midpoint", 0.5 * (interval.start + interval.end)),
            ("end", interval.end),
```

```
            ("after-end", interval.end + probe),
        ):
            assessment = joint_coverage_assessment(
                time,
                evaluation.bombs,
                surface_angles=int(formal["surface_angles"]),
                surface_levels=int(formal["surface_levels"]),
                continuous_refinement=True,
                refinement_starts=int(formal["continuous_refinement_starts"]),
                refinement_maxfev=int(formal["continuous_refinement_maxfev"]),
                adaptive_max_surface_angles=int(formal["adaptive_max_surface_angles"]),
                adaptive_max_surface_levels=int(formal["adaptive_max_surface_levels"]),
            )
            active = _active_bombs(time, evaluation.bombs)
            covered = _independent_ray_intersections(
                points,
                missile_position(time),
                [_cloud_center(time, bomb) for bomb in active],
            )
            boundary.append(
                {
                    "location": location,
                    "time_s": time,
                    "status": assessment.status,
                    "margin_m": assessment.signed_margin_m,
                    "lower_bound_m": assessment.sampled_lower_bound_m,
                    "upper_bound_m": assessment.lipschitz_upper_bound_m,
                    "surface_angles": assessment.surface_angles,
                    "surface_levels": assessment.surface_levels,
                    "trace": assessment.refinement_trace,
                }
            )
            direct.append(
                {
                    "location": location,
                    "time_s": time,
                    "sampled_rays": len(points),
                    "uncovered_rays": int(np.count_nonzero(~covered)),
                    "all_covered": bool(np.all(covered)),
                }
            )
    permutation_durations = []
    medium = validation["convergence_settings"][1]
    for order in permutations(evaluation.bombs):
        intervals = _find_joint_intervals(
            _settings_margin(order, medium, refine=False),
            order,
            scan_step=float(medium["time_scan_step_s"]),
            root_tolerance=float(formal["root_tolerance_s"]),
        )
        permutation_durations.append(_duration(intervals))
    q3_vector = np.array(
        [
            3.1047922065753095,
            75.53740845287045,
            0.00008659727834377369,
            0.0001070796609000733,
            0.0000817521129715,
            2.7648584468708908,
            2.58163903013058,
            9.334353521045797,
        ]
```

```
    )
    q3_bombs = q3_bomb_geometries(Q3Strategy(*q3_vector), float(config["gravity"]))
    q3_intervals = _find_joint_intervals(
        _settings_margin(q3_bombs, formal, refine=True),
        q3_bombs,
        scan_step=float(formal["time_scan_step_s"]),
        root_tolerance=float(formal["root_tolerance_s"]),
    )
    complement_time = 4.768
    center = np.array([17625.525268056128, 10.240444866222791, 1762.640057120904])
    complement_bombs = tuple(
        BombGeometry(
            index,
            complement_time,
            0.0,
            complement_time,
            tuple(center + np.array([0.0, sign * 9.65, 0.0])),
            tuple(center + np.array([0.0, sign * 9.65, 0.0])),
        )
        for index, sign in enumerate((-1.0, 1.0), 1)
    )
    individual_complement = [
        joint_coverage_margin(
            complement_time,
            (bomb,),
            surface_angles=360,
            surface_levels=41,
            continuous_refinement=True,
            refinement_starts=1,
            refinement_maxfev=50,
        )
        for bomb in complement_bombs
    ]
    joint_complement = joint_coverage_margin(
        complement_time,
        complement_bombs,
        surface_angles=360,
        surface_levels=41,
        continuous_refinement=True,
        refinement_starts=1,
        refinement_maxfev=50,
    )
    return {
        "feasibility": {
            "violation": _feasibility_violation(
                evaluation.strategy, float(config["gravity"]), config
            ),
            "speeds_m_s": [control.speed_m_s for control in evaluation.strategy.controls],
            "release_times_s": [control.release_time_s for control in evaluation.strategy.controls],
            "fuse_delays_s": [control.fuse_delay_s for control in evaluation.strategy.controls],
            "explosion_times_s": [
                control.explosion_time_s for control in evaluation.strategy.controls
            ],
            "explosion_heights_m": [bomb.explosion_point[2] for bomb in evaluation.bombs],
            "cloud_minimum_heights_m": [
                bomb.explosion_point[2] - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME
                for bomb in evaluation.bombs
            ],
            "all_speeds_within_bounds": all(
                70.0 <= control.speed_m_s <= 140.0 for control in evaluation.strategy.controls
            ),
            "all_release_and_fuse_times_nonnegative": all(
```

```
                control.release_time_s >= 0.0 and control.fuse_delay_s >= 0.0
                for control in evaluation.strategy.controls
            ),
            "all_explosions_before_missile_hit": all(
                control.explosion_time_s <= hit_time for control in evaluation.strategy.controls
            ),
            "all_explosions_before_ground": all(
                bomb.explosion_point[2] >= 0.0 for bomb in evaluation.bombs
            ),
            "cloud_center_below_ground_triggered": any(
                bomb.explosion_point[2] - CLOUD_DESCENT_SPEED * CLOUD_LIFETIME < 0.0
                for bomb in evaluation.bombs
            ),
        },
        "surface_time_convergence": convergence,
        "boundary_checks": boundary,
        "independent_unit_direction_ray_checks": direct,
        "label_permutation_durations_s": permutation_durations,
        "q3_geometry_regression": {
            "computed_duration_s": _duration(q3_intervals),
            "expected_duration_s": float(validation["q3_regression_expected_s"]),
            "absolute_difference_s": abs(
                _duration(q3_intervals) - float(validation["q3_regression_expected_s"])
            ),
            "tolerance_s": float(validation["q3_regression_tolerance_s"]),
        },
        "complementarity_construct": {
            "individual_margins_m": individual_complement,
            "joint_margin_m": joint_complement,
            "each_individual_incomplete": all(value > 0.0 for value in individual_complement),
            "joint_complete": joint_complement <= 0.0,
        },
        "joint_vs_conservative": {
            "joint_duration_s": evaluation.joint_duration_s,
            "conservative_duration_s": evaluation.conservative_duration_s,
            "joint_not_less": evaluation.joint_duration_s + 1e-9
            >= evaluation.conservative_duration_s,
            "joint_not_less_than_each_individual": evaluation.joint_duration_s + 1e-9
            >= max(_duration(intervals) for intervals in evaluation.individual_intervals),
        },
        "deletion": [
            {
                "uav": name,
                "remaining_duration_s": duration,
                "marginal_loss_s": evaluation.joint_duration_s - duration,
            }
            for name, duration in zip(UAV_NAMES, evaluation.deletion_durations_s, strict=True)
        ],
    }


def run_question_4(project_root: Path, config: dict[str, Any]) -> list[Path]:
    """Run Question 4 and write strategy, workbook, solver, and validation artifacts."""
    paths = prepare_output_paths(project_root / "outputs")
    stage_path = paths.logs / "q4_search_stage.json"
    evaluations, solver = optimize_question_4(config, stage_path)
    if not evaluations:
        raise RuntimeError("Question 4 optimization produced no formal candidate")
    best = evaluations[0]
    near_count = min(int(config["near_optimal_count"]), len(evaluations))
    summaries = [_summary_row(item, rank) for rank, item in enumerate(evaluations[:near_count], 1)]
    summary_path = save_table(
```

```
        pd.DataFrame(summaries[:1]), paths.tables / "q4_summary.csv", overwrite=True
    )
    candidates_path = save_table(
        pd.DataFrame(summaries), paths.tables / "q4_candidates.csv", overwrite=True
    )
    rows = pd.DataFrame(_uav_rows(best))
    uavs_path = save_table(rows, paths.tables / "q4_uavs.csv", overwrite=True)
    interval_rows = []
    groups = [
        ("joint-union-cover", best.joint_intervals),
        ("conservative-single-cloud-union", best.conservative_intervals),
        *(
            (f"{name}-strict", intervals)
            for name, intervals in zip(UAV_NAMES, best.individual_intervals, strict=True)
        ),
    ]
    for criterion, intervals in groups:
        for index, interval in enumerate(intervals, 1):
            interval_rows.append(
                {
                    "criterion": criterion,
                    "interval": index,
                    "start_s": interval.start,
                    "end_s": interval.end,
                    "duration_s": interval.duration,
                }
            )
    intervals_path = save_table(
        pd.DataFrame(interval_rows), paths.tables / "q4_intervals.csv", overwrite=True
    )
    workbook_path = paths.tables / "result2.xlsx"
    with pd.ExcelWriter(workbook_path, engine="openpyxl") as writer:
        rows.to_excel(writer, sheet_name="provisional_result2", index=False)
        pd.DataFrame(
            [
                {
                    "schema_status": "provisional",
                    "reason": "official result2.xlsx template is missing",
                    "mapping_action": "map explicit fields when the official template is supplied",
                }
            ]
        ).to_excel(writer, sheet_name="schema_note", index=False)
    optimization_path = paths.logs / "q4_optimization.json"
    optimization_path.write_text(
        json.dumps(
            {
                "model": "q4-three-uav-joint-ray-cover-v1",
                "best": {**_summary_row(best, 1), "uavs": _uav_rows(best)},
                "near_optimal_candidates": summaries,
                "solver": solver,
            },
            ensure_ascii=False,
            indent=2,
        )
        + "\n",
        encoding="utf-8",
    )
    validation_path = paths.logs / "q4_validation.json"
    validation_path.write_text(
        json.dumps(_validation_record(best, config), ensure_ascii=False, indent=2) + "\n",
        encoding="utf-8",
    )
```

```
    return [
        summary_path,
        candidates_path,
        uavs_path,
        intervals_path,
        workbook_path,
        stage_path,
        optimization_path,
        validation_path,
    ]
```

B.6 问题五程序

```
"""Question 5 hierarchical optimization for five UAVs and three missiles."""

from __future__ import annotations

import json
from collections.abc import Sequence
from dataclasses import asdict, dataclass
from itertools import product
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd
from scipy.optimize import brentq, differential_evolution, minimize

from mmkit.artifacts import prepare_output_paths, save_table

from .q1 import CLOUD_LIFETIME, MISSILE_SPEED, OcclusionInterval
from .q3 import (
    BombGeometry,
    _cloud_center,
    _continuous_worst_margin,
    _coverage_margins,
    _duration,
    _independent_ray_intersections,
    _merge_intervals,
    _surface_covering_radius,
    _surface_points,
)
from .q4 import Q4Strategy, UAVControl
from .q4 import _bomb_geometries as q4_bomb_geometries

UAV_NAMES = ("FY1", "FY2", "FY3", "FY4", "FY5")
MISSILE_NAMES = ("M1", "M2", "M3")
TARGET_CENTER = np.array([0.0, 200.0, 5.0])


@dataclass(frozen=True, slots=True)
class UAVPlan:
    """Shared flight controls and three automatically spaced bomb slots."""

    theta_rad: float
    speed_m_s: float
    s0_s: float
    s1_s: float
    s2_s: float
    fuse1_s: float
    fuse2_s: float
```

```
    fuse3_s: float

    @property
    def release_times_s(self) -> tuple[float, float, float]:
        return (
            self.s0_s,
            self.s0_s + 1.0 + self.s1_s,
            self.s0_s + 2.0 + self.s1_s + self.s2_s,
        )

    @property
    def fuse_delays_s(self) -> tuple[float, float, float]:
        return (self.fuse1_s, self.fuse2_s, self.fuse3_s)


@dataclass(frozen=True, slots=True)
class Q5Strategy:
    """Forty-variable strategy represented as five eight-variable UAV blocks."""

    plans: tuple[UAVPlan, UAVPlan, UAVPlan, UAVPlan, UAVPlan]


@dataclass(frozen=True, slots=True)
class MissileEvaluation:
    """Formal joint coverage result for one missile."""

    missile: str
    intervals: tuple[OcclusionInterval, ...]

    @property
    def duration_s(self) -> float:
        return _duration(self.intervals)


@dataclass(frozen=True, slots=True)
class Q5Evaluation:
    """Formal three-missile evaluation and per-bomb deletion contributions."""

    strategy: Q5Strategy
    bombs: tuple[BombGeometry, ...]
    missiles: tuple[MissileEvaluation, MissileEvaluation, MissileEvaluation]
    deletion_losses_s: tuple[tuple[float, float, float], ...]

    @property
    def durations_s(self) -> tuple[float, float, float]:
        return tuple(item.duration_s for item in self.missiles)

    @property
    def j_sum_s(self) -> float:
        return sum(self.durations_s)

    @property
    def j_min_s(self) -> float:
        return min(self.durations_s)


def missile_hit_time(initial: np.ndarray) -> float:
    """Return time for an arbitrary missile to reach the false target at the origin."""
    return float(np.linalg.norm(initial) / MISSILE_SPEED)


def missile_position(initial: np.ndarray, time: float) -> np.ndarray:
```

```
    """Return an arbitrary missile position at absolute task time."""
    return initial * (1.0 - time / missile_hit_time(initial))


def _strategy_from_array(values: np.ndarray) -> Q5Strategy:
    array = np.asarray(values, dtype=float).reshape(5, 8)
    return Q5Strategy(tuple(UAVPlan(*(float(value) for value in row)) for row in array))


def _strategy_vector(strategy: Q5Strategy) -> np.ndarray:
    return np.array([value for plan in strategy.plans for value in asdict(plan).values()])


def _uav_initials(config: dict[str, Any]) -> tuple[np.ndarray, ...]:
    return tuple(np.asarray(config["uavs"][name], dtype=float) for name in UAV_NAMES)


def _missile_initials(config: dict[str, Any]) -> tuple[np.ndarray, ...]:
    return tuple(np.asarray(config["missiles"][name], dtype=float) for name in MISSILE_NAMES)


def _bomb_geometries(strategy: Q5Strategy, config: dict[str, Any]) -> tuple[BombGeometry, ...]:
    gravity = float(config["gravity"])
    bombs = []
    number = 1
    for plan, initial in zip(strategy.plans, _uav_initials(config), strict=True):
        heading = np.array([np.cos(plan.theta_rad), np.sin(plan.theta_rad), 0.0])
        for release, fuse in zip(plan.release_times_s, plan.fuse_delays_s, strict=True):
            explosion_time = release + fuse
            release_point = initial + plan.speed_m_s * release * heading
            explosion_point = initial + plan.speed_m_s * explosion_time * heading
            explosion_point += np.array([0.0, 0.0, -0.5 * gravity * fuse**2])
            bombs.append(
                BombGeometry(
                    number,
                    release,
                    fuse,
                    explosion_time,
                    tuple(float(value) for value in release_point),
                    tuple(float(value) for value in explosion_point),
                )
            )
            number += 1
    return tuple(bombs)


def _fuse_maximum(initial: np.ndarray, gravity: float) -> float:
    return float(np.sqrt(2.0 * initial[2] / gravity))


def _block_bounds(initial: np.ndarray, config: dict[str, Any]) -> list[tuple[float, float]]:
    bounds = config["bounds"]
    fuse = (0.0, _fuse_maximum(initial, float(config["gravity"])))
    return [
        tuple(bounds["theta_rad"]),
        tuple(bounds["speed_m_s"]),
        tuple(bounds["s0_s"]),
        tuple(bounds["s1_s"]),
        tuple(bounds["s2_s"]),
        fuse,
        fuse,
        fuse,
```

```
    ]


def _feasibility_violation(strategy: Q5Strategy, config: dict[str, Any]) -> float:
    violation = 0.0
    latest_hit = max(missile_hit_time(item) for item in _missile_initials(config))
    bombs = _bomb_geometries(strategy, config)
    for uav_index, (plan, initial) in enumerate(
        zip(strategy.plans, _uav_initials(config), strict=True)
    ):
        fuse_max = _fuse_maximum(initial, float(config["gravity"]))
        violation += max(0.0, 70.0 - plan.speed_m_s) / 70.0
        violation += max(0.0, plan.speed_m_s - 140.0) / 70.0
        for value in asdict(plan).values():
            violation += max(0.0, -value - 1e-12) / max(1.0, fuse_max)
        for bomb in bombs[3 * uav_index : 3 * uav_index + 3]:
            violation += max(0.0, bomb.explosion_time_s - latest_hit) / latest_hit
            violation += max(0.0, -bomb.explosion_point[2]) / float(initial[2])
    return violation


def _active_bombs(time: float, bombs: Sequence[BombGeometry]) -> tuple[BombGeometry, ...]:
    return tuple(
        bomb
        for bomb in bombs
        if bomb.explosion_time_s <= time <= bomb.explosion_time_s + CLOUD_LIFETIME
    )


def _margin(
    time: float,
    bombs: Sequence[BombGeometry],
    missile_initial: np.ndarray,
    settings: dict[str, Any],
    *,
    refine: bool,
) -> float:
    active = _active_bombs(time, bombs)
    if not active or time > missile_hit_time(missile_initial):
        return float("inf")
    missile = missile_position(missile_initial, time)
    angles = int(settings["surface_angles"])
    levels = int(settings["surface_levels"])
    points = _surface_points(angles, levels)
    margins = _coverage_margins(points, missile, active, time)
    sampled = float(np.max(margins))
    if not refine or sampled > 0.0:
        return sampled
    maximum_angles = int(settings.get("adaptive_max_surface_angles", angles))
    maximum_levels = int(settings.get("adaptive_max_surface_levels", levels))
    while sampled + _surface_covering_radius(angles, levels) > 0.0 and (
        angles < maximum_angles or levels < maximum_levels
    ):
        angles = min(maximum_angles, 2 * angles)
        levels = min(maximum_levels, 2 * (levels - 1) + 1)
        points = _surface_points(angles, levels)
        margins = _coverage_margins(points, missile, active, time)
        sampled = float(np.max(margins))
        if sampled > 0.0:
            return sampled
    upper = sampled + _surface_covering_radius(angles, levels)
    if upper <= 0.0:
```

```
        return upper
    return _continuous_worst_margin(
        missile,
        active,
        time,
        points,
        margins,
        starts=int(settings.get("continuous_refinement_starts", 1)),
        maxfev=int(settings.get("continuous_refinement_maxfev", 40)),
    )


def _find_intervals(
    bombs: Sequence[BombGeometry],
    missile_initial: np.ndarray,
    settings: dict[str, Any],
    *,
    refine: bool,
) -> tuple[OcclusionInterval, ...]:
    hit = missile_hit_time(missile_initial)
    start = min(bomb.explosion_time_s for bomb in bombs)
    end = min(hit, max(bomb.explosion_time_s + CLOUD_LIFETIME for bomb in bombs))
    if start > end:
        return ()
    step = float(settings.get("time_scan_step_s", settings.get("time_step_s", 0.2)))
    tolerance = float(settings.get("root_tolerance_s", 1e-8))
    regular = np.arange(0.0, hit + 0.5 * step, step)
    regular = regular[(regular >= start) & (regular <= end)]
    events = np.array(
        [
            event
            for bomb in bombs
            for event in (bomb.explosion_time_s, bomb.explosion_time_s + CLOUD_LIFETIME)
            if start <= event <= end
        ]
    )
    probe = max(tolerance * 10.0, step * 1e-5)
    probes = np.array(
        [
            value
            for event in events
            for value in (max(start, event - probe), min(end, event + probe))
        ]
    )
    times = np.unique(np.concatenate(([start, end], regular, events, probes)))

    def function(time: float) -> float:
        return _margin(time, bombs, missile_initial, settings, refine=refine)

    values = np.array([function(float(time)) for time in times])
    inside = values <= 0.0
    result = []
    current = float(times[0]) if inside[0] else None
    for index in np.flatnonzero(inside[1:] != inside[:-1]):
        left, right = float(times[index]), float(times[index + 1])
        midpoint = 0.5 * (left + right)
        active = tuple(bomb.bomb for bomb in _active_bombs(left, bombs))
        middle = tuple(bomb.bomb for bomb in _active_bombs(midpoint, bombs))
        ending = tuple(bomb.bomb for bomb in _active_bombs(right, bombs))
        if active != middle and middle == ending:
            boundary = left
        elif active == middle and middle != ending:
```

```
            boundary = right
        else:
            boundary = float(brentq(function, left, right, xtol=tolerance, rtol=1e-14))
        if inside[index + 1]:
            current = boundary
        elif current is not None:
            result.append(OcclusionInterval(current, boundary))
            current = None
    if current is not None:
        result.append(OcclusionInterval(current, float(times[-1])))
    return _merge_intervals(result)


def _sampled_duration(
    bombs: Sequence[BombGeometry], missile_initial: np.ndarray, settings: dict[str, Any]
) -> tuple[float, float]:
    hit = missile_hit_time(missile_initial)
    step = float(settings["time_step_s"])
    start = min(bomb.explosion_time_s for bomb in bombs)
    end = min(hit, max(bomb.explosion_time_s + CLOUD_LIFETIME for bomb in bombs))
    if start > end:
        return 0.0, float("inf")
    anchored = np.arange(0.0, hit + 0.5 * step, step)
    anchored = anchored[(anchored >= start) & (anchored <= end)]
    events = np.array(
        [
            event
            for bomb in bombs
            for event in (bomb.explosion_time_s, bomb.explosion_time_s + CLOUD_LIFETIME)
            if start <= event <= end
        ]
    )
    times = np.unique(np.concatenate(([start, end], anchored, events)))
    values = np.array(
        [_margin(float(time), bombs, missile_initial, settings, refine=False) for time in times]
    )
    return float(np.trapezoid((values <= 0.0).astype(float), times)), float(np.min(values))


def _single_bomb(control: np.ndarray, initial: np.ndarray, gravity: float) -> BombGeometry:
    theta, speed, release, fuse = control
    heading = np.array([np.cos(theta), np.sin(theta), 0.0])
    explosion_time = release + fuse
    release_point = initial + speed * release * heading
    explosion_point = initial + speed * explosion_time * heading
    explosion_point += np.array([0.0, 0.0, -0.5 * gravity * fuse**2])
    return BombGeometry(
        1,
        float(release),
        float(fuse),
        float(explosion_time),
        tuple(float(value) for value in release_point),
        tuple(float(value) for value in explosion_point),
    )


def _single_population(
    uav: np.ndarray, missile: np.ndarray, fuse_max: float, settings: dict[str, Any], seed: int
) -> np.ndarray:
    count = 4 * int(settings["popsize"])
    rng = np.random.default_rng(seed)
    lower = np.array([0.0, 70.0, 0.0, 0.0])
```

```
    upper = np.array([2.0 * np.pi, 140.0, 45.0, fuse_max])
    population = rng.uniform(lower, upper, size=(count, 4))
    structured = []
    hit = missile_hit_time(missile)
    for time in np.linspace(3.0, 0.85 * hit, 40):
        point = missile_position(missile, float(time))
        for fraction in (0.05, 0.15, 0.3, 0.5, 0.7):
            desired = (1.0 - fraction) * point + fraction * TARGET_CENTER
            displacement = desired[:2] - uav[:2]
            speed = float(np.linalg.norm(displacement) / time)
            height_drop = uav[2] - desired[2]
            if 70.0 <= speed <= 140.0 and height_drop >= 0.0:
                fuse = float(np.sqrt(2.0 * height_drop / 9.8))
                release = float(time - fuse)
                if 0.0 <= release <= 45.0 and fuse <= fuse_max:
                    structured.append(
                        np.array(
                            [
                                np.arctan2(displacement[1], displacement[0]) % (2 * np.pi),
                                speed,
                                release,
                                fuse,
                            ]
                        )
                    )
    for index, item in enumerate(structured[:count]):
        population[index] = item
    return population


def _candidate_library(
    config: dict[str, Any],
) -> tuple[list[dict[str, Any]], dict[tuple[int, int], np.ndarray]]:
    settings = config["single_library"]
    gravity = float(config["gravity"])
    records = []
    best = {}
    for uav_index, uav in enumerate(_uav_initials(config)):
        fuse_max = _fuse_maximum(uav, gravity)
        for missile_index, missile in enumerate(_missile_initials(config)):
            hit = missile_hit_time(missile)

            def objective(
                values: np.ndarray,
                fixed_uav: np.ndarray = uav,
                fixed_missile: np.ndarray = missile,
                fixed_hit: float = hit,
            ) -> float:
                bomb = _single_bomb(values, fixed_uav, gravity)
                if bomb.explosion_time_s > fixed_hit or bomb.explosion_point[2] < 0.0:
                    return 100.0 + max(0.0, bomb.explosion_time_s - fixed_hit)
                duration, minimum = _sampled_duration((bomb,), fixed_missile, settings)
                return -duration if duration > 0.0 else max(0.0, minimum) / 100.0

            seed = int(config["seed"]) + 10 * uav_index + missile_index
            result = differential_evolution(
                objective,
                [(0.0, 2 * np.pi), (70.0, 140.0), (0.0, 45.0), (0.0, fuse_max)],
                init=_single_population(uav, missile, fuse_max, settings, seed),
                seed=seed,
                maxiter=int(settings["maxiter"]),
                popsize=int(settings["popsize"]),
```

```
                tol=float(settings["tol"]),
                polish=False,
                workers=1,
            )
            vector = np.asarray(result.x, dtype=float)
            best[(uav_index, missile_index)] = vector
            records.append(
                {
                    "uav": UAV_NAMES[uav_index],
                    "missile": MISSILE_NAMES[missile_index],
                    "sampled_duration_s": max(0.0, -float(result.fun)),
                    "physical_feasible": bool(
                        _single_bomb(vector, uav, gravity).explosion_time_s <= hit
                        and _single_bomb(vector, uav, gravity).explosion_point[2] >= 0.0
                    ),
                    "positive_coverage_found": bool(float(result.fun) < 0.0),
                    "evaluations": int(result.nfev),
                    "success": bool(result.success),
                    "message": str(result.message),
                    "vector": vector.tolist(),
                }
            )
    return records, best


def _assembled_strategy(
    assignment: Sequence[int], library: dict[tuple[int, int], np.ndarray]
) -> Q5Strategy:
    plans = []
    for uav_index, missile_index in enumerate(assignment):
        theta, speed, release, fuse = library[(uav_index, missile_index)]
        base = max(0.0, float(release) - 1.0)
        plans.append(
            UAVPlan(
                float(theta), float(speed), base, 0.0, 0.0, float(fuse), float(fuse), float(fuse)
            )
        )
    return Q5Strategy(tuple(plans))


def _q4_anchor_strategy(
    library: dict[tuple[int, int], np.ndarray], config: dict[str, Any]
) -> Q5Strategy:
    """Embed the three formally verified Q4 bombs as a monotone M1 incumbent."""
    plans = []
    for row in config["validation"]["q4_strategy"]:
        theta, speed, release, fuse = (float(value) for value in row)
        plans.append(UAVPlan(theta, speed, release, 0.0, 0.0, fuse, fuse, fuse))
    for uav_index, missile_index in ((3, 1), (4, 2)):
        theta, speed, release, fuse = library[(uav_index, missile_index)]
        plans.append(
            UAVPlan(
                float(theta),
                float(speed),
                float(release),
                0.0,
                0.0,
                float(fuse),
                float(fuse),
                float(fuse),
            )
        )
```

```
    return Q5Strategy(tuple(plans))


def _proxy_scores(
    strategy: Q5Strategy, config: dict[str, Any], settings: dict[str, Any]
) -> tuple[float, tuple[float, float, float]]:
    violation = _feasibility_violation(strategy, config)
    if violation > 0.0:
        return -100.0 - 100.0 * violation, (0.0, 0.0, 0.0)
    bombs = _bomb_geometries(strategy, config)
    durations = tuple(
        _sampled_duration(bombs, missile, settings)[0] for missile in _missile_initials(config)
    )
    score = sum(durations) - 2.0 * sum(max(0.0, 0.05 - value) for value in durations)
    return score, durations


def _block_refine(
    strategy: Q5Strategy, config: dict[str, Any], stage_path: Path
) -> tuple[Q5Strategy, list[dict[str, Any]]]:
    settings = config["block_refinement"]
    current = _strategy_vector(strategy)
    records = []
    for round_index in range(int(settings["rounds"])):
        for uav_index, initial in enumerate(_uav_initials(config)):
            offset = 8 * uav_index

            def objective(block: np.ndarray, fixed_offset: int = offset) -> float:
                candidate = current.copy()
                candidate[fixed_offset : fixed_offset + 8] = block
                return -_proxy_scores(_strategy_from_array(candidate), config, settings)[0]

            result = minimize(
                objective,
                current[offset : offset + 8],
                method="Nelder-Mead",
                bounds=_block_bounds(initial, config),
                options={
                    "maxfev": int(settings["maxfev_per_block"]),
                    "xatol": float(settings["xatol"]),
                    "fatol": float(settings["fatol"]),
                },
            )
            if float(result.fun) < objective(current[offset : offset + 8]):
                current[offset : offset + 8] = result.x
            score, durations = _proxy_scores(_strategy_from_array(current), config, settings)
            records.append(
                {
                    "round": round_index + 1,
                    "uav": UAV_NAMES[uav_index],
                    "evaluations": int(result.nfev),
                    "success": bool(result.success),
                    "message": str(result.message),
                    "proxy_score_s": score,
                    "durations_s": durations,
                }
            )
            stage_path.write_text(
                json.dumps({"stage": "block-refinement-in-progress", "records": records}, indent=2)
                + "\n",
                encoding="utf-8",
            )
```

```
    return _strategy_from_array(current), records


def _marginal_fill(
    strategy: Q5Strategy, config: dict[str, Any], stage_path: Path
) -> tuple[Q5Strategy, list[dict[str, Any]]]:
    """Search otherwise unused second/third slots while preserving each anchor bomb."""
    settings = config["marginal_fill"]
    current = _strategy_vector(strategy)
    records = []
    rng = np.random.default_rng(int(config["seed"]) + 900)
    for name in settings["uav_order"]:
        uav_index = UAV_NAMES.index(name)
        offset = 8 * uav_index
        initial = _uav_initials(config)[uav_index]
        fuse_max = _fuse_maximum(initial, float(config["gravity"]))
        lower = np.array([0.0, 0.0, 0.0, 0.0])
        upper = np.array([20.0, 20.0, fuse_max, fuse_max])
        population = rng.uniform(lower, upper, size=(int(settings["population_size"]), 4))
        plan = _strategy_from_array(current).plans[uav_index]
        anchor = np.array([plan.s1_s, plan.s2_s, plan.fuse2_s, plan.fuse3_s])
        population[0] = anchor
        structured = (
            (0.0, 0.0, plan.fuse1_s, plan.fuse1_s),
            (1.0, 1.0, plan.fuse1_s, plan.fuse1_s),
            (3.0, 3.0, plan.fuse1_s, plan.fuse1_s),
            (6.0, 6.0, plan.fuse1_s, plan.fuse1_s),
            (0.0, 2.0, 0.8 * plan.fuse1_s, 1.2 * plan.fuse1_s),
            (2.0, 5.0, 1.2 * plan.fuse1_s, 0.8 * plan.fuse1_s),
        )
        for index, values in enumerate(structured, 1):
            if index < len(population):
                population[index] = np.clip(values, lower, upper)

        base_vector = current.copy()

        def full_vector(
            values: np.ndarray,
            base: np.ndarray = base_vector,
            fixed_offset: int = offset,
        ) -> np.ndarray:
            candidate = base.copy()
            candidate[fixed_offset + 3 : fixed_offset + 5] = values[:2]
            candidate[fixed_offset + 6 : fixed_offset + 8] = values[2:]
            return candidate

        def objective(values: np.ndarray) -> float:
            candidate = _strategy_from_array(full_vector(values))
            return -_proxy_scores(candidate, config, settings)[0]

        before_score, before_durations = _proxy_scores(
            _strategy_from_array(current), config, settings
        )
        result = differential_evolution(
            objective,
            [(0.0, 20.0), (0.0, 20.0), (0.0, fuse_max), (0.0, fuse_max)],
            init=population,
            seed=int(config["seed"]) + 900 + uav_index,
            maxiter=int(settings["maxiter"]),
            popsize=2,
            polish=False,
            workers=1,
```

```
        )
        candidate = full_vector(np.asarray(result.x, dtype=float))
        after_score, after_durations = _proxy_scores(
            _strategy_from_array(candidate), config, settings
        )
        accepted = after_score > before_score + float(settings["improvement_threshold_s"])
        if accepted:
            current = candidate
        records.append(
            {
                "uav": name,
                "evaluations": int(result.nfev),
                "success": bool(result.success),
                "message": str(result.message),
                "before_score_s": before_score,
                "after_score_s": after_score,
                "before_durations_s": before_durations,
                "after_durations_s": after_durations,
                "accepted": accepted,
            }
        )
        stage_path.write_text(
            json.dumps({"stage": "marginal-fill-in-progress", "records": records}, indent=2) + "\n",
            encoding="utf-8",
        )
    return _strategy_from_array(current), records


def _formal_evaluation(
    strategy: Q5Strategy, config: dict[str, Any], *, compute_deletion: bool
) -> Q5Evaluation:
    bombs = _bomb_geometries(strategy, config)
    settings = config["formal_evaluation"]
    missiles = tuple(
        MissileEvaluation(name, _find_intervals(bombs, initial, settings, refine=True))
        for name, initial in zip(MISSILE_NAMES, _missile_initials(config), strict=True)
    )
    deletion = []
    if compute_deletion:
        deletion_settings = config["deletion_evaluation"]
        for removed in range(15):
            remaining = tuple(bomb for index, bomb in enumerate(bombs) if index != removed)
            losses = []
            for missile, baseline in zip(_missile_initials(config), missiles, strict=True):
                intervals = _find_intervals(remaining, missile, deletion_settings, refine=False)
                losses.append(
                    max(
                        0.0,
                        baseline.duration_s - min(_duration(intervals), baseline.duration_s),
                    )
                )
            deletion.append(tuple(losses))
    else:
        deletion = [(0.0, 0.0, 0.0)] * 15
    return Q5Evaluation(strategy, bombs, missiles, tuple(deletion))


def optimize_question_5(
    config: dict[str, Any], stage_path: Path
) -> tuple[list[Q5Evaluation], dict[str, Any], list[dict[str, Any]]]:
    """Build the 15-pair library, assemble incumbents, and refine five UAV blocks."""
    library_records, library = _candidate_library(config)
```

```
    stage_path.write_text(
        json.dumps(
            {"stage": "candidate-library-complete", "candidate_library": library_records}, indent=2
        )
        + "\n",
        encoding="utf-8",
    )
    print("Q5 15-pair candidate library complete", flush=True)
    library_scores = {
        (UAV_NAMES.index(record["uav"]), MISSILE_NAMES.index(record["missile"])): record[
            "sampled_duration_s"
        ]
        for record in library_records
    }
    assignments = []
    for assignment in product(range(3), repeat=5):
        if len(set(assignment)) < 3:
            continue
        score = sum(library_scores[(uav, missile)] for uav, missile in enumerate(assignment))
        assignments.append((score, assignment))
    assignments.sort(reverse=True)
    assembly_count = int(config["assembly"]["candidate_assignments"])
    assembled = []
    for _, assignment in assignments[:assembly_count]:
        strategy = _assembled_strategy(assignment, library)
        score, durations = _proxy_scores(strategy, config, config["assembly"])
        assembled.append((score, min(durations), strategy, assignment, durations))
    assembled.sort(key=lambda item: (item[0], item[1]), reverse=True)
    incumbent = _q4_anchor_strategy(library, config)
    incumbent_score, incumbent_durations = _proxy_scores(incumbent, config, config["assembly"])
    stage_path.write_text(
        json.dumps(
            {
                "stage": "assembly-complete",
                "q4_anchor_incumbent": {
                    "proxy_sum_s": incumbent_score,
                    "durations_s": incumbent_durations,
                    "retained_for_formal_evaluation": True,
                },
                "assignments": [
                    {
                        "assignment": [MISSILE_NAMES[index] for index in item[3]],
                        "proxy_sum_s": item[0],
                        "durations_s": item[4],
                    }
                    for item in assembled
                ],
            },
            indent=2,
        )
        + "\n",
        encoding="utf-8",
    )
    print("Q5 feasible three-missile incumbent assembled", flush=True)
    refined, block_records = _block_refine(incumbent, config, stage_path)
    filled, fill_records = _marginal_fill(refined, config, stage_path)
    candidates = [incumbent, refined, filled, *(item[2] for item in assembled[1:])]
    proxy_ranked = sorted(
        candidates,
        key=lambda item: _proxy_scores(item, config, config["block_refinement"])[0],
        reverse=True,
    )
```

```
    count = min(int(config["formal_evaluation"]["candidate_count"]), len(proxy_ranked))
    formal_strategies = [proxy_ranked[0]]
    if count > 1:
        formal_strategies.append(incumbent)
    formal = []
    for index, strategy in enumerate(formal_strategies[:count], 1):
        formal.append(_formal_evaluation(strategy, config, compute_deletion=False))
        print(f"Q5 formal candidate {index}/{count} complete", flush=True)
    formal.sort(key=lambda item: (item.j_sum_s, item.j_min_s), reverse=True)
    formal[0] = _formal_evaluation(formal[0].strategy, config, compute_deletion=True)
    stage_path.write_text(
        json.dumps(
            {
                "stage": "search-complete",
                "library_size": len(library_records),
                "block_records": block_records,
                "marginal_fill_records": fill_records,
                "formal_scores": [
                    {
                        "j_sum_s": item.j_sum_s,
                        "j_min_s": item.j_min_s,
                        "durations_s": item.durations_s,
                    }
                    for item in formal
                ],
                "q4_anchor_incumbent_retained": True,
                "sampled_time_grid": "absolute task time t=0",
            },
            indent=2,
        )
        + "\n",
        encoding="utf-8",
    )
    solver = {
        "seed": int(config["seed"]),
        "method": "15 single-pair DE searches, assignment assembly, and five bounded 8D blocks",
        "candidate_library_size": len(library_records),
        "block_refinements": block_records,
        "marginal_fill": fill_records,
        "formal_settings": config["formal_evaluation"],
        "status": "budget-limited-hierarchical-search-with-feasible-incumbent",
        "convergence_claimed": False,
        "sampled_time_grid": "absolute task time t=0 plus cloud events",
        "q4_anchor_incumbent_retained": True,
    }
    return formal, solver, library_records


def _summary(evaluation: Q5Evaluation, rank: int) -> dict[str, Any]:
    return {
        "rank": rank,
        "D1_s": evaluation.durations_s[0],
        "D2_s": evaluation.durations_s[1],
        "D3_s": evaluation.durations_s[2],
        "J_sum_s": evaluation.j_sum_s,
        "J_min_s": evaluation.j_min_s,
    }


def _bomb_rows(evaluation: Q5Evaluation) -> list[dict[str, Any]]:
    rows = []
    for index, bomb in enumerate(evaluation.bombs):
```

```
        uav_index, slot = divmod(index, 3)
        plan = evaluation.strategy.plans[uav_index]
        losses = evaluation.deletion_losses_s[index]
        rows.append(
            {
                "schema_status": "provisional-schema; official result3.xlsx unavailable",
                "uav": UAV_NAMES[uav_index],
                "bomb_slot": slot + 1,
                "used": sum(losses) > 1e-8,
                "theta_rad": plan.theta_rad,
                "theta_deg": float(np.degrees(plan.theta_rad) % 360.0),
                "speed_m_s": plan.speed_m_s,
                "release_time_s": bomb.release_time_s,
                "release_x_m": bomb.release_point[0],
                "release_y_m": bomb.release_point[1],
                "release_z_m": bomb.release_point[2],
                "fuse_delay_s": bomb.fuse_delay_s,
                "explosion_time_s": bomb.explosion_time_s,
                "explosion_x_m": bomb.explosion_point[0],
                "explosion_y_m": bomb.explosion_point[1],
                "explosion_z_m": bomb.explosion_point[2],
                "M1_deletion_loss_s": losses[0],
                "M2_deletion_loss_s": losses[1],
                "M3_deletion_loss_s": losses[2],
                "J_sum_deletion_loss_s": sum(losses),
            }
        )
    return rows


def _validation(evaluation: Q5Evaluation, config: dict[str, Any]) -> dict[str, Any]:
    bombs = evaluation.bombs
    formal = config["formal_evaluation"]
    validation = config["validation"]
    convergence = {}
    boundaries = {}
    direct = {}
    points = _surface_points(
        int(validation["direct_surface_angles"]), int(validation["direct_surface_levels"])
    )
    for name, missile, baseline in zip(
        MISSILE_NAMES, _missile_initials(config), evaluation.missiles, strict=True
    ):
        convergence[name] = []
        for index, settings in enumerate(validation["convergence_settings"]):
            merged = {
                **settings,
                **(
                    {
                        "adaptive_max_surface_angles": formal["adaptive_max_surface_angles"],
                        "adaptive_max_surface_levels": formal["adaptive_max_surface_levels"],
                        "continuous_refinement_starts": 1,
                        "continuous_refinement_maxfev": 40,
                    }
                    if index == 2
                    else {}
                ),
            }
            intervals = _find_intervals(bombs, missile, merged, refine=index == 2)
            convergence[name].append({**settings, "duration_s": _duration(intervals)})
        boundaries[name], direct[name] = [], []
        for interval in baseline.intervals:
```

```
            for location, time in (
                ("before-start", interval.start - validation["boundary_probe_s"]),
                ("start", interval.start),
                ("midpoint", 0.5 * (interval.start + interval.end)),
                ("end", interval.end),
                ("after-end", interval.end + validation["boundary_probe_s"]),
            ):
                margin = _margin(time, bombs, missile, formal, refine=True)
                active = _active_bombs(time, bombs)
                covered = _independent_ray_intersections(
                    points,
                    missile_position(missile, time),
                    [_cloud_center(time, bomb) for bomb in active],
                )
                boundaries[name].append({"location": location, "time_s": time, "margin_m": margin})
                direct[name].append(
                    {
                        "location": location,
                        "time_s": time,
                        "sampled_rays": len(points),
                        "uncovered_rays": int(np.count_nonzero(~covered)),
                        "all_covered": bool(np.all(covered)),
                    }
                )
    rng = np.random.default_rng(int(config["seed"]) + 500)
    permutations = []
    medium = validation["convergence_settings"][1]
    for _ in range(int(validation["permutation_samples"])):
        order = tuple(bombs[index] for index in rng.permutation(len(bombs)))
        permutations.append(
            [
                _duration(_find_intervals(order, missile, medium, refine=False))
                for missile in _missile_initials(config)
            ]
        )
    q4_strategy = Q4Strategy(
        tuple(UAVControl(*(float(value) for value in row)) for row in validation["q4_strategy"])
    )
    q4_config = {
        "gravity": config["gravity"],
        "uavs": {name: config["uavs"][name] for name in UAV_NAMES[:3]},
    }
    q4_bombs = q4_bomb_geometries(q4_strategy, float(config["gravity"]), q4_config)
    q4_duration = _duration(
        _find_intervals(q4_bombs, _missile_initials(config)[0], formal, refine=True)
    )
    gaps = [np.diff(plan.release_times_s).tolist() for plan in evaluation.strategy.plans]
    return {
        "feasibility": {
            "violation": _feasibility_violation(evaluation.strategy, config),
            "release_gaps_s": gaps,
            "all_release_gaps_at_least_1s": all(min(item) >= 1.0 for item in gaps),
            "all_durations_positive": all(value > 0.0 for value in evaluation.durations_s),
            "explosion_heights_m": [bomb.explosion_point[2] for bomb in bombs],
        },
        "surface_time_convergence": convergence,
        "boundary_checks": boundaries,
        "independent_unit_direction_ray_checks": direct,
        "label_permutation_duration_samples_s": permutations,
        "label_permutation_max_deviation_s": max(
            (
                abs(sample[index] - permutations[0][index])
```

```
                for sample in permutations[1:]
                for index in range(3)
            ),
            default=0.0,
        ),
        "q4_m1_geometry_regression": {
            "computed_duration_s": q4_duration,
            "expected_duration_s": float(validation["q4_regression_expected_s"]),
            "absolute_difference_s": abs(
                q4_duration - float(validation["q4_regression_expected_s"])
            ),
            "tolerance_s": float(validation["q4_regression_tolerance_s"]),
            "within_tolerance": abs(q4_duration - float(validation["q4_regression_expected_s"]))
            <= float(validation["q4_regression_tolerance_s"]),
        },
        "deletion_contributions_s": [list(item) for item in evaluation.deletion_losses_s],
        "arithmetic": {
            "durations_s": evaluation.durations_s,
            "reported_J_sum_s": evaluation.j_sum_s,
            "recomputed_J_sum_s": sum(evaluation.durations_s),
            "consistent": evaluation.j_sum_s == sum(evaluation.durations_s),
        },
        "q4_monotone_lower_bound": {
            "q5_D1_s": evaluation.durations_s[0],
            "q4_reference_s": float(validation["q4_regression_expected_s"]),
            "satisfied": evaluation.durations_s[0] + float(validation["q4_regression_tolerance_s"])
            >= float(validation["q4_regression_expected_s"]),
        },
    }


def run_question_5(project_root: Path, config: dict[str, Any]) -> list[Path]:
    """Run Question 5 and save provisional tables, workbook, logs, and stage state."""
    paths = prepare_output_paths(project_root / "outputs")
    stage_path = paths.logs / "q5_search_stage.json"
    if bool(config.get("formal_only_persistent_rerun", False)):
        previous_stage = (
            json.loads(stage_path.read_text(encoding="utf-8")) if stage_path.exists() else None
        )
        persistent = _strategy_from_array(
            np.asarray(config["persistent_incumbent"]["vector"], dtype=float)
        )
        evaluations = [_formal_evaluation(persistent, config, compute_deletion=True)]
        library_path_existing = paths.tables / "q5_candidate_library.csv"
        library = (
            pd.read_csv(library_path_existing).to_dict(orient="records")
            if library_path_existing.exists()
            else []
        )
        solver = {
            "seed": int(config["seed"]),
            "status": "persistent-incumbent-formal-rerun",
            "convergence_claimed": False,
            "persistent_incumbent": config["persistent_incumbent"],
            "marginal_fill_comparison": {
                "candidate_J_sum_s": 13.869304210239239,
                "retained_incumbent_J_sum_s": evaluations[0].j_sum_s,
                "candidate_improved_incumbent": False,
            },
            "previous_search_stage": previous_stage,
            "formal_settings": config["formal_evaluation"],
            "deletion_settings": config["deletion_evaluation"],
```

```
        }
        stage_path.write_text(
            json.dumps(
                {
                    "stage": "search-complete",
                    "mode": "persistent-incumbent-formal-rerun",
                    "persistent_J_sum_s": evaluations[0].j_sum_s,
                    "marginal_fill_candidate_J_sum_s": 13.869304210239239,
                    "retained_persistent_incumbent": True,
                    "previous_search_stage": previous_stage,
                },
                indent=2,
            )
            + "\n",
            encoding="utf-8",
        )
    else:
        evaluations, solver, library = optimize_question_5(config, stage_path)
    if not evaluations or evaluations[0].j_min_s <= 0.0:
        raise RuntimeError("Question 5 produced no strategy covering all three missiles")
    best = evaluations[0]
    summaries = [
        _summary(item, rank)
        for rank, item in enumerate(evaluations[: int(config["near_optimal_count"])], 1)
    ]
    summary_path = save_table(
        pd.DataFrame(summaries[:1]), paths.tables / "q5_summary.csv", overwrite=True
    )
    candidates_path = save_table(
        pd.DataFrame(summaries), paths.tables / "q5_candidates.csv", overwrite=True
    )
    library_path = save_table(
        pd.DataFrame(library), paths.tables / "q5_candidate_library.csv", overwrite=True
    )
    rows = pd.DataFrame(_bomb_rows(best))
    bombs_path = save_table(rows, paths.tables / "q5_bombs.csv", overwrite=True)
    interval_rows = [
        {
            "missile": item.missile,
            "interval": number,
            "start_s": interval.start,
            "end_s": interval.end,
            "duration_s": interval.duration,
        }
        for item in best.missiles
        for number, interval in enumerate(item.intervals, 1)
    ]
    intervals_path = save_table(
        pd.DataFrame(interval_rows), paths.tables / "q5_intervals.csv", overwrite=True
    )
    workbook_path = paths.tables / "result3.xlsx"
    with pd.ExcelWriter(workbook_path, engine="openpyxl") as writer:
        rows.to_excel(writer, sheet_name="provisional_result3", index=False)
        pd.DataFrame(
            [
                {
                    "schema_status": "provisional",
                    "reason": "official result3.xlsx template is missing",
                    "mapping_action": "map explicit fields when official template is supplied",
                }
            ]
        ).to_excel(writer, sheet_name="schema_note", index=False)
```

```
    optimization_path = paths.logs / "q5_optimization.json"
    optimization_path.write_text(
        json.dumps(
            {
                "model": "q5-hierarchical-three-missile-joint-cover-v1",
                "objective": "J_sum=D1+D2+D3; J_min reported",
                "best": {**_summary(best, 1), "bombs": _bomb_rows(best)},
                "near_optimal_candidates": summaries,
                "solver": solver,
            },
            ensure_ascii=False,
            indent=2,
        )
        + "\n",
        encoding="utf-8",
    )
    validation_path = paths.logs / "q5_validation.json"
    validation_path.write_text(
        json.dumps(_validation(best, config), ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
    )
    return [
        summary_path,
        candidates_path,
        library_path,
        bombs_path,
        intervals_path,
        workbook_path,
        stage_path,
        optimization_path,
        validation_path,
    ]
```